
**Sistema híbrido de análisis musical y modelos de
lenguaje para la recuperación de información de
canciones populares iberoamericanas**
**A hybrid system combining music analysis and language
models for retrieving information on popular
Ibero-American songs**



**Trabajo de Fin de Máster
Curso 2025–2026**

Autor
María Sacher Carrasco

Director
Carlos León Aznar

Máster en Inteligencia Artificial
Facultad de Informática
Universidad Complutense de Madrid

Sistema híbrido de análisis musical y
modelos de lenguaje para la recuperación
de información de canciones populares
iberoamericanas

A hybrid system combining music analysis
and language models for retrieving
information on popular Ibero-American
songs

Trabajo de Fin de Máster en Inteligencia Artificial

Autor

María Sacher Carrasco

Director

Carlos León Aznar

Convocatoria: *Junio 2026*

Calificación:

Máster en Inteligencia Artificial

Facultad de Informática

Universidad Complutense de Madrid

18 de JUNIO de 2026

Dedicatória

TODO: completar

Agradecimientos

(TODO: completar)

Resumen

Sistema híbrido de análisis musical y modelos de lenguaje para la recuperación de información de canciones populares ibero-americanas

El estudio de grandes corpus musicales constituye una herramienta fundamental en disciplinas como la musicología histórica, la etnomusicología y el análisis musical. Tradicionalmente, estas investigaciones han dependido de procesos manuales de lectura y catalogación de partituras, una metodología rigurosa pero con severas limitaciones en términos de escalabilidad frente al creciente volumen de material musical digitalizado. A pesar de la aparición de formatos simbólicos estandarizados como MusicXML o MEI, persisten desafíos críticos relacionados con la heterogeneidad de los corpus, la inconsistencia en los estándares simbólicos, la complejidad algorítmica para detectar fenómenos rítmicos y la división entre el análisis estructural y semántico de las letras. Asimismo, los modelos de lenguaje (LLM) de propósito general presentan serias limitaciones de contexto y razonamiento para ejecutar análisis musicales especializados en tiempo real sobre colecciones extensas.

En este contexto, y como parte del proyecto europeo e iberoamericano EA-DIGIFOLK, este trabajo presenta el diseño e implementación de una arquitectura automatizada, robusta y escalable orientada al análisis de corpus musicales extensos y heterogéneos. El sistema propuesto supera las limitaciones actuales mediante una infraestructura intermedia que combina el procesamiento de partituras, la extracción y normalización automática de metadatos y su guardado posterior en una base de datos SQLite, y la integración del análisis musical y semántico.

Palabras clave

Musicología computacional, Corpus musicales, EA-DIGIFOLK, Extracción de características, Modelos de lenguaje (LLM), Music21, Música folclórica, Recuperación de información

Abstract

A hybrid system combining music analysis and language models for retrieving information on popular Ibero-American songs

The study of large musical corpora constitutes a fundamental tool in disciplines such as historical musicology, ethnomusicology, and musical analysis. Traditionally, this type of research has depended on manual processes of score reading and cataloging, a rigorous methodology but limited in terms of scalability against the growing volume of digitized musical material. Despite the emergence of standardized symbolic formats like MusicXML or MEI, critical challenges persist regarding the heterogeneity of the corpora, inconsistencies in symbolic standards, algorithmic complexity in detecting rhythmic phenomena, and the division between structural analysis and the semantic analysis of lyrics. Furthermore, general-purpose language models (LLMs) present severe limitations in context and reasoning when executing specialized musical analysis in real time over extensive collections.

In this context, and as part of the European and Ibero-American project EA-DIGIFOLK, this work presents the design and implementation of an automated, robust, and scalable architecture oriented towards the analysis of large and heterogeneous musical corpora. The proposed system overcomes current limitations through an intermediate infrastructure that combines score processing, automated metadata extraction and normalization along with its subsequent storage in an SQLite database, and the integration of musical and semantic analysis.

Keywords

Computational musicology, Musical corpora, EA-DIGIFOLK, Feature extraction, Language models (LLMs), Music21, Folk music, Information retrieval

Índice

1. Introducción	1
1.1. Motivación	2
1.2. Objetivos	3
1.2.1. Objetivos generales	4
1.2.2. Objetivos específicos	4
1.3. Plan de trabajo	4
1.4. Metodología y tecnología	6
1.5. Estructura del resto del documento	7
1.6. Uso de Inteligencia Artificial	8
2. Estado de la Cuestión	9
2.1. El giro computacional en el análisis de textos artísticos y el procesamiento simbólico musical	9
2.2. Modelos de Lenguaje de Gran Escala (LLMs) y el dominio musical simbólico	10
2.3. Recuperación de Información en Grandes Corpus: RAG Semántico frente a Enfoques Relacionales (Text-to-SQL)	10
2.3.1. Generación Aumentada por Recuperación (RAG)	10
2.3.2. Aproximaciones Relacionales y Text-to-SQL	11
2.3.3. Integración de Contexto y el Protocolo MCP	11
3. Captura de requisitos con musicólogas y formulación de preguntas para comprobar las capacidades de los modelos de lenguaje	13
3.1. Captura de requisitos con musicólogas a través de entrevistas personales . .	13
3.2. Captura de requisitos con musicólogas a través de formularios	14
3.3. Formulación de preguntas para comprobar las capacidades de los modelos de lenguaje	15
4. Sistema de indexación y recuperación de información basado en RAG Gemini File Search	19
4.1. Implementación del sistema de indexación y recuperación	19
4.1.1. Preparación e identificación de documentos	19
4.1.2. Creación y gestión del almacén de búsqueda de archivos	19
4.1.3. Procesamiento de archivos musicales MuseScore	20
4.1.4. Estrategia de carga e indexación	20

4.1.5. Recuperación de información mediante búsqueda semántica	20
5. Preprocesamiento del corpus musical para su extracción de metadatos y posterior almacenamiento en una base de datos SQLite y sistema de MCP para realizar consultas a la base de datos desde un modelo pequeño local	21
5.1. Introducción a la implementación y pipeline del sistema	21
5.2. Herramientas desarrolladas para la extracción de información de las obras .	22
5.2.1. Extracción de tempo	23
5.2.2. Extracción de compás	23
5.2.3. Extracción híbrida de la tonalidad	23
5.2.4. Extracción de título y autor	24
5.2.5. Inferencia del modo de la obra	25
5.2.6. Detección de hemiolias verticales	25
5.2.7. Detección de hemiolias horizontales	26
5.2.8. Detección de sínkopas	27
5.2.9. Extracción de letras	28
5.2.10. Análisis temático de la lírica de las obras con LLMs	29
5.2.11. Detección de cambio de resolución rítmica	29
5.2.12. Detección de desajustes en duraciones de notas	30
5.2.13. Detección de valores irregulares ocultos (grupillos)	31
5.2.14. Cálculo de densidad de eventos musicales por compás	32
5.2.15. Detección automática de polirritmias verticales	32
5.2.16. Extracción de la tesitura de la obra	33
5.2.17. Inferencia del género del autor a partir del nombre	34
5.2.18. Análisis automático de la perspectiva lírica	34
5.2.19. Extracción de notas y silencios	35
5.2.20. Extracción de la secuencia melódica de la obra	36
5.2.21. Análisis de la dirección de los intervalos melódicos	36
5.2.22. Clasificación de la tendencia melódica predominante	37
5.2.23. Extracción de correspondencias entre sílabas y duraciones musicales	37
5.2.24. Extracción y conteo de alteraciones accidentales	38
5.2.25. Extracción de metadatos de transcripción y procedencia digital . . .	39
5.2.26. Auditoría automatizada de control de calidad estructural	40
5.2.27. Extracción de intervalos melódicos	41
5.2.28. Detección y conteo de leitmotivs recurrentes	41
5.2.29. Detección de plicas anómalas	42
5.2.30. Extracción del volumen MIDI de la obra	42
5.2.31. Extracción de sustantivos y nombres propios (personas, lugares...) de la lírica de la obra	43
5.2.32. Identificación de saltos melódicos entre barras de compás (intervalos frontera)	44
5.2.33. Identificación del uso de la etiqueta <sb />(system break) en archivos MEI	45
5.3. Ventajas que ofrece la metodología escogida para la herramienta desarrollada	45
5.4. Desarrollo de aplicación web para facilitar el acceso a la herramienta	47

6. Evaluación y pruebas	49
6.1. Preguntas realizadas y respuestas de los modelos	50
6.1.1. Identifica piezas que se llamen El Carro	50
6.1.2. Identifica piezas cuyo autor sea Schumann	51
6.1.3. Identifica piezas con un compás de 4/4	52
6.1.4. Identifica piezas con hemiolias horizontales	53
6.1.5. Identifica piezas con hemiolias verticales	54
6.1.6. Identifica piezas con la tonalidad G mayor	55
6.1.7. Identifica piezas que estén en modo dórico	56
6.1.8. Identifica piezas con un bpm de 120	57
6.1.9. Identifica piezas con un volumen midi de 79	58
6.1.10. Identifica piezas cuya nota más grave sea A3	59
6.1.11. Identifica piezas cuya nota más aguda sea G5	59
6.1.12. Identifica piezas cuyo autor sea de género masculino	60
6.1.13. Identifica piezas cuyo software de codificación haya sido Sibelius 7.1.3	61
6.1.14. Identifica piezas cuya fecha de codificación sea el 2015-07-01	62
6.1.15. Identifica piezas con síncopas	63
6.1.16. Identifica piezas que traten la muerte como tema	64
6.1.17. Identifica piezas con polirritmia	64
6.1.18. Identifica piezas cuyo contenido lírico esté escrito en voz masculina	65
6.1.19. Identifica obras cuya tendencia melódica sea predominantemente descendente	66
6.1.20. Identifica piezas con 4 alteraciones accidentales	68
6.1.21. Identifica piezas con compases vacíos	68
6.1.22. Identifica piezas con notas de duración 0	69
6.1.23. Identifica piezas con plicas anómalas	70
6.1.24. Identifica piezas que usen la palabra “bondad”	71
6.1.25. Identifica piezas que mencionen el nombre propio “Alfonsito”	72
6.1.26. Identifica piezas cuyo intervalo frontera más frecuente sea +5	72
6.1.27. Identifica piezas en 2/4 que contengan síncopas ligadas entre el segundo pulso de un compás y el primero del siguiente de forma sistemática	74
6.1.28. Identifica piezas de tonalidad A menor que tengan hemiolias y que no superen los 90 bpm	75
6.1.29. Identifica piezas de tonalidad A menor que tengan hemiolias	76
6.1.30. Encuentra canciones donde el valor de dur.ppq cambie a mitad de la sección, indicando un cambio de resolución rítmica	77
6.1.31. Busca ejemplos de anacrusa en piezas que estén etiquetadas como “Nanas”	79
6.1.32. Localiza obras que tengan compases donde la suma de las duraciones de las etiquetas <note>no coincida con el meterSig declarado en el staffDef	80
6.1.33. ¿Qué piezas utilizan valores irregulares (por ejemplo, tresillos) no declarados explícitamente en un <time-modification>del MusicXML?	82
6.1.34. Compara la densidad de notas por compás entre una pieza de 4/4 y una de 2/4 del dataset; ¿cuál tiene mayor promedio de eventos musicales?	83

6.1.35. Identifica si existe algún uso de polirritmia en los archivos MEI . . .	84
6.1.36. Lista canciones que compartan la misma armadura de un bemol (F major/D minor) pero que traten temas distintos	85
6.1.37. Encuentra piezas recolectadas antes de 1950 que traten el tema de “la muerte” y que utilicen exclusivamente un rango melódico de sexta menor	86
6.1.38. ¿Qué diferencias de tesitura (nota más aguda vs. nota más grave) existen entre las canciones de nana?	88
6.1.39. Identifica canciones donde el autor sea una mujer, pero el contenido lírico esté escrito en voz masculina	91
6.1.40. Filtra piezas que usen la palabra “estrella” en la lírica y determina si su melodía es predominantemente ascendente o descendente	92
6.1.41. Busca canciones que traten temas de trabajo que compartan la mis- ma estructura de intervalos que las canciones de tema infantil	93
6.1.42. Localiza todas las notas que tengan bemoles accidentales y dime en qué compases aparecen en los archivos MEI encontrados	94
6.1.43. Identifica el uso de la etiqueta <sb /> (system break) y determina si coincide siempre con el final de una frase musical lógica	95
6.1.44. Compara el uso de las sílabas “duer” y “me” en los distintos archivos del dataset: ¿se asigna siempre a la misma duración de nota?	97
6.1.45. ¿Cuál es el intervalo melódico más frecuente entre la última nota de un compás y la primera del siguiente en las piezas de modo mixolidio?	98
6.1.46. Determina si las canciones con tempo superior a 120 BPM tienden a tener letras con palabras de menor longitud silábica	99
6.1.47. Identifica piezas donde la nota final sea la dominante (quinto grado) en lugar de la tónica, sugiriendo una estructura abierta	100
6.1.48. Identifica si existe algún patrón de silencios (<rest />) que se repi- ta sistemáticamente cada cuatro compases en las piezas de métrica ternaria	101
6.2. Resumen de preguntas formuladas y respuestas de los modelos	102
7. Discusión y análisis de limitaciones	107
8. Conclusiones y Trabajo Futuro	109
8.1. Conclusiones	109
8.2. Trabajo futuro	110
9. Introduction	113
9.1. Motivation	114
9.2. Objectives	115
9.2.1. General Objectives	115
9.2.2. Specific Objectives	115
9.3. Work Plan	116
9.4. Methodology and Technology	118
9.5. Structure of the Rest of the Document	119
9.6. Use of Artificial Intelligence	120
10. Conclusions and Future Work	121

10.1. Conclusions	121
10.2. Future Work	122
Bibliografía	125
A. Presentación del proyecto en congresos y seminarios	127

Índice de figuras

1.1. Diagrama de Gantt del plan de trabajo (octubre - junio).	5
5.1. Interfaz de la aplicación web desarrollada para interactuar con la herramienta.	47
9.1. Gantt chart of the work plan (October - June).	117

Índice de tablas

6.1.	Fragmento de la tabla devuelta tras consultar la tesitura en semitonos de distintas piezas del dataset	90
6.2.	Fragmento de la tabla de relaciones entre temas y tendencia melódica en piezas del dataset	94
6.3.	Fragmento de la tabla de relaciones entre BPM y número aproximado de sílabas	99
6.4.	Fragmento de la tabla de ejemplos de piezas en métrica ternaria que contienen silencios.	102
6.5.	Resumen de resultados obtenidos para las consultas de evaluación.	103

Introducción

“Many things we can’t do are simply because we think we can’t do them”

— Satoru Iwata

El estudio de grandes corpus musicales constituye una de las principales herramientas de investigación dentro de disciplinas como la musicología histórica, la etnomusicología, el análisis musical o la documentación del patrimonio sonoro. A través del análisis sistemático de conjuntos extensos de obras es posible identificar patrones estilísticos, estructuras compositivas recurrentes, relaciones entre repertorios y fenómenos culturales que difícilmente pueden observarse mediante el estudio aislado de piezas individuales.

Tradicionalmente, este tipo de investigaciones ha dependido de procesos manuales de lectura, catalogación y análisis de partituras, una metodología rigurosa pero limitada en términos de escalabilidad. El creciente volumen de material musical digitalizado, especialmente en ámbitos relacionados con la música tradicional y el patrimonio musical, ha puesto de manifiesto la necesidad de desarrollar herramientas capaces de automatizar parte de estas tareas y facilitar la exploración de grandes colecciones documentales.

La aparición de formatos simbólicos estandarizados para la representación musical, como MusicXML o MEI ha impulsado el desarrollo de la musicología computacional y ha permitido aplicar técnicas de análisis automatizado sobre repertorios cada vez más amplios. Sin embargo, la práctica demuestra que siguen existiendo importantes desafíos relacionados con la heterogeneidad de los corpus, la compatibilidad entre formatos, la extracción de información musicológica compleja, la integración entre contenido musical y textual, y la recuperación eficiente de conocimiento a partir de grandes volúmenes de datos.

De forma paralela, los avances recientes en inteligencia artificial y procesamiento del lenguaje natural han abierto nuevas posibilidades para la interacción con colecciones de datos extensas. No obstante, las particularidades de la información musical plantean limitaciones que dificultan la aplicación directa de estas tecnologías, especialmente cuando las consultas requieren conocimientos derivados de análisis musicales especializados que no se encuentran explícitamente representados en las fuentes originales.

En este contexto, este proyecto propone el diseño y desarrollo de una arquitectura orientada al análisis automatizado de corpus musicales extensos y heterogéneos. El sistema combina técnicas de procesamiento de partituras, extracción de características musicales, catalogación automática, recuperación de información y asistencia a través de modelos de lenguaje, con el objetivo de proporcionar una plataforma capaz de facilitar la investigación musicológica y mejorar el acceso al conocimiento contenido en grandes colecciones musicales.

En este contexto se desarrolla el proyecto "*EA-DIGIFOLK-An European and Ibero-American approach for the digital collection, analysis and dissemination of folk music (101086338)*", financiado por la Unión Europea, en el que participa la UCM¹, cuyo principal objetivo es preservar y compartir la cultura folclórica popular europea e iberoamericana, sobre todo del ámbito musical. EA-DIGIFOLK busca crear una plataforma que facilite la difusión de esta cultura a entornos educativos y públicos. Por otra parte, y más en relación con este proyecto, otro de los principales objetivos es el análisis musical y etnomusicológico del material audiovisual y escrito recopilado. Este trabajo se ha llevado a cabo como parte de este proyecto de investigación.

1.1. Motivación

El análisis musicológico de grandes volúmenes de música tradicional y/o folclórica se ha enfrentado históricamente a una limitación metodológica: la dependencia del análisis manual de partituras, lo que supone un proceso riguroso pero difícilmente escalable, ya que la automatización digital de este procedimiento resulta prácticamente imposible. La digitalización de archivos musicales en formatos simbólicos estructurados (como MusicXML o MEI) ha abierto las puertas a la musicología computacional, pero siguen persistiendo problemas fundamentales que dificultan cualquier metodología de automatización al digitalizar este tipo de contenido. Estos son algunos obstáculos y dificultades encontradas en el ámbito del análisis musical que han motivado el desarrollo de este proyecto:

- **Heterogeneidad en los corpus.** Los corpus musicales de gran tamaño, como con el que se ha trabajado a lo largo de este proyecto, suelen ser muy heterogéneos tanto a nivel estructural como a su catalogación. En primer lugar, las obras pueden encontrarse almacenadas en múltiples formatos de archivo (*MSCZ*, *MusicXML*, *MEI*, entre otros), cada uno con particularidades técnicas y semánticas que requieren procesos de extracción y normalización específicos. Esta diversidad impide la construcción de flujos de análisis completamente automáticos y obliga a desarrollar procedimientos de preprocesado adaptados a cada fuente de información.

Además, la heterogeneidad del corpus no se limita al formato de los ficheros, sino que también afecta a los criterios de catalogación y organización de los repositorios. En estos casos, pueden encontrarse con frecuencia colecciones donde ciertos metadatos fundamentales, como el título de la obra, el autor o la procedencia geográfica, se almacenan dentro de la propia estructura del archivo, mientras que en otros casos dicha información únicamente está disponible en el nombre del fichero o en la organización jerárquica de los directorios. La ausencia de estándares homogéneos de descripción documental dificulta la identificación automática de las obras y requiere mecanismos adicionales de inferencia y validación de metadatos.

- **Inconsistencia en los estándares simbólicos.** Muchas herramientas estándar utilizadas en el ámbito técnico del análisis musicológico, como *music21* Cuthbert y Ariza (2010) pueden experimentar fallos críticos o perder metadatos relevantes a la hora de interpretar determinadas fuentes de información. Esto ocurre, por ejemplo, con ciertas etiquetas presentes en archivos MEI (como el atributo *wordpos*, que indica la posición de una sílaba dentro de una palabra; sin embargo, no todos los valores que puede tomar no son interpretables por muchos programas como *music21*), cuya

¹<https://cordis.europa.eu/project/id/101086338>

implementación no está plenamente normalizada entre los distintos programas de edición de partituras.

- **Complejidad algorítmica en la detección de fenómenos rítmicos complejos.** En el mundo de la música existe una infinidad de estructuras expresivas y ambiguas, como las síncopas y las hemiolias (tanto horizontales como verticales), que requieren un modelado matemático con un estricto estudio previo basado en características muy concretas de las obras, como los tiempos de ataque y jerarquías métricas. Esto supone trabajar de cero en funciones que no están implementadas de forma nativa en las librerías analíticas convencionales.
- **División entre el análisis musical y el semántico.** Muchos sistemas de análisis musical automatizado aíslan el contenido estructural y rítmico del contenido textual de la lírica. Esta es una separación muy problemática, ya que la separación de estas dos estructuras tan relacionadas entre sí implica una grave pérdida de información. Por ello es necesario diseñar un sistema capaz de extraer características musicales analíticas y, al mismo tiempo, clasificar el contenido temático y poético de las canciones.
- **Limitaciones contextuales de los modelos de lenguaje en corpus extensos.** Cuando el tamaño del corpus es elevado, resulta inviable proporcionar al modelo toda la información como contexto de entrada, especialmente en modelos de menor tamaño, cuyos límites de contexto y capacidad de razonamiento son más reducidos. Incluso empleando estrategias de recuperación de información (*Retrieval-Augmented Generation*, RAG), muchas consultas musicológicas requieren previamente la extracción y el cálculo de características analíticas complejas sobre las partituras, tales como patrones rítmicos, estructuras melódicas o relaciones métricas. Dado que estos procesos no pueden ejecutarse de manera fiable en tiempo real por un modelo de lenguaje, se hace necesaria una arquitectura intermedia capaz de realizar análisis musicales especializados y almacenar sus resultados en estructuras optimizadas para la recuperación y consulta posterior.

El proyecto nace a partir de estas carencias, con el objetivo de resolverlas mediante el estudio y desarrollo de un sistema capaz de suplir las necesidades de las musicólogas y usuarias que necesiten herramientas más sofisticadas para enfrentarse a análisis de corpus de datos muy extensos y complejos.

El propósito principal de esta investigación se centra en el desarrollo de una arquitectura automatizada, robusta y escalable, capaz de analizar rasgos latentes en un corpus de obras musicales, integrando simultáneamente información estructural, rítmica, melódica y semántica para facilitar el estudio musicológico de grandes colecciones folclóricas.

1.2. Objetivos

A partir de las limitaciones identificadas en las metodologías actuales de análisis musicológico computacional, este proyecto busca desarrollar una infraestructura tecnológica que facilite el estudio de grandes corpus musicales mediante técnicas de análisis automatizado, recuperación de información, análisis musicológico técnico y procesamiento del lenguaje natural. Para ello se establecen los siguientes objetivos.

1.2.1. Objetivos generales

Desarrollar un sistema automatizado, robusto y escalable para el análisis, catalogación y consulta de grandes corpus musicales heterogéneos, capaz de extraer conocimiento musicológico a partir de un gran número de obras almacenadas en distintos formatos, facilitando así el trabajo de investigación, exploración y recuperación de información por parte de musicólogas, investigadoras y otras usuarias especializadas.

1.2.2. Objetivos específicos

- Diseñar una arquitectura capaz de procesar corpus musicales compuestos por distintos formatos de representación simbólica, garantizando la compatibilidad entre las diversas fuentes documentales.
- Desarrollar un sistema automático de extracción y normalización de metadatos musicales con el fin de facilitar la catalogación sistemática de las obras y la construcción de bases de datos fiables y consistentes.
- Implementar algoritmos de análisis musical que permitan detectar y cuantificar características estructurales, melódicas, rítmicas y métricas relevantes para la investigación musicológica.
- Integrar el análisis musical con el análisis semántico y textual de las letras, permitiendo estudiar conjuntamente los aspectos musicales y literarios de las obras.
- Automatizar tareas de procesamiento y análisis que habitualmente requieren una elevada inversión de tiempo o conocimientos técnicos especializados, reduciendo las barreras de acceso a herramientas avanzadas de investigación musical.
- Diseñar e implementar un sistema de recuperación de información que permita realizar búsquedas complejas sobre el corpus, incluyendo consultas basadas tanto en metadatos como en características musicales extraídas automáticamente.
- Evaluar las capacidades y limitaciones de los modelos de lenguaje de propósito general en tareas de análisis y consulta de corpus musicológicos extensos, identificando los problemas de escalabilidad, precisión y contexto que presentan estos sistemas.
- Proponer mecanismos complementarios que permitan superar las limitaciones observadas en dichos modelos mediante la incorporación de análisis musicológicos especializados y estructuras optimizadas para la recuperación de conocimiento.

1.3. Plan de trabajo

Aquí se describe el plan de trabajo a seguir para la consecución de los objetivos descritos en el apartado anterior.

Para la consecución de los objetivos propuestos se ha diseñado un plan de trabajo estructurado en seis fases principales, abarcando desde finales de octubre hasta principios de junio. Este cronograma contempla desde la investigación inicial y la captura de requisitos mediante trabajo de campo, hasta el desarrollo técnico de la arquitectura, su evaluación comparativa y la redacción final de la memoria de investigación.

A continuación, se detalla la planificación temporal mediante un diagrama de Gantt (Figura 9.1):



Figura 1.1: Diagrama de Gantt del plan de trabajo (octubre - junio).

A nivel operativo, el desarrollo se desglosa en las siguientes actividades clave ligadas directamente a los objetivos del proyecto:

- **Fase 1 (Octubre – Diciembre):** Dedicada al estudio profundo del estado del arte en musicología computacional.
- **Fase 2 (Marzo):** Dedicada de forma exclusiva a la captura de requisitos mediante la realización de entrevistas cualitativas a musicólogas e investigadoras del área. El objetivo de este hito es alinear las capacidades del sistema con las necesidades analíticas reales y cotidianas de las usuarias expertas.
- **Fase 3 (Diciembre – Febrero):** Centrada en el diseño e implementación de la infraestructura intermedia y el pipeline de normalización automática, encargados de unificar la persistencia de datos en el archivo local de SQLite independientemente de la heterogeneidad u organización original del corpus folclórico.
- **Fase 4 (Febrero – Abril):** Enfocada en el desarrollo de algoritmos basados en *music21* para la extracción de rasgos técnicos complejos (estructuras rítmicas y melódicas) y su integración con los modelos de lenguaje locales (*Llama3* a través de *Ollama*) destinados a la clasificación semántica y temática de la lírica de las obras.
- **Fase 5 (Abril – Mayo):** Destinada a la fase crítica de evaluación. Se realizan las pruebas sobre el sistema de recuperación de información implementando tanto la aproximación local de texto a SQL como el enfoque RAG en la nube mediante la funcionalidad *File Search* de la API de Gemini, permitiendo contrastar los límites de precisión, contexto y alucinaciones de ambas tecnologías.
- **Fase 6 (Abril – Junio):** Consistente en la redacción de la memoria de investigación y preparación de la documentación del software desarrollado para garantizar su total reproducibilidad técnica posterior.

1.4. Metodología y tecnología

Para desarrollar este proyecto se han seleccionado herramientas de código abierto y la versión gratuita de la API de Gemini. Esta decisión garantiza la reproducción íntegra del proyecto de manera local (exceptuando la utilización de la API de Gemini).

- **Python 3.** Utilizado como base para desarrollar todo el proyecto. Se ha seleccionado este lenguaje de programación de alto nivel y propósito general fundamentalmente por albergar el ecosistema de librerías más avanzado para la musicología computacional, la ciencia de datos y la inteligencia artificial. Su versatilidad permite articular en un único flujo de trabajo automatizado desde el preprocesamiento de partituras y la normalización de metadatos procedentes de corpus musicales heterogéneos, hasta la gestión de bases de datos locales, la comunicación con servidores de modelos de lenguaje mediante peticiones estructuradas y la posterior evaluación analítica de los resultados.
- **Music21.** Kit de herramientas líder para la musicología computacional. Permite realizar un análisis musical profundo de cada obra en torno a sus notas, intervalos, acordes, escalas, etc., lo que facilita el estudio de patrones armónicos, melódicos y rítmicos, difíciles de identificar con otras herramientas y, en este caso, con LLMs. Además, tiene soporte para múltiples formatos, como MIDI o XML, lo que resulta

muy útil en esta investigación, donde se trabaja con un extenso y heterogéneo corpus de obras en distintos formatos. Por otra parte, su integración con Python y otras librerías facilita la extracción y el análisis de la información del corpus musical.

- **Ollama.** Servidor local que permite utilizar modelos de lenguaje de manera local y personalizada. Se ha seleccionado *llama3* como LLM para ejecutar distintas tareas de procesamiento de lenguaje natural. Esto mitiga costes de APIs externas, protege la privacidad del corpus y facilita la invariabilidad de las salidas mediante la restricción nativa de respuestas en formato JSON estructurado.
- **API de Gemini (versión gratuita).** Plataforma de servicios en la nube basada en inteligencia artificial generativa desarrollada por Google. En este proyecto se ha seleccionado específicamente el modelo *Gemini 2.5 Flash* a través de su API libre de costes para implementar una aproximación de recuperación y consulta basada en arquitecturas RAG asistidas por la funcionalidad nativa *File Search*.
- **SQLite.** Librería que implementa un motor de base de datos SQL pequeño, gratuito y de código abierto. Se ha escogido específicamente esta herramienta para almacenar la información extraída porque, a diferencia de otras bases de datos como MySQL, PostgreSQL o SQL Server, que requieren instalar un software que corra de fondo en tu ordenador, configurar usuarios, contraseñas y puertos, SQLite no cuelga de un servidor, sino que crea la base de datos como un único archivo local.

1.5. Estructura del resto del documento

Para facilitar la lectura y comprensión del proyecto desarrollado, el contenido de esta memoria se organiza en diferentes capítulos de manera secuencial de acuerdo con el ciclo de vida del proyecto desarrollado:

- **Capítulo 2: Estado de la Cuestión.** Ofrece el marco teórico fundamental de la investigación. En él se contextualiza el fenómeno conocido como el “*giro computacional*” aplicado al análisis de textos artísticos, creativos y musicales, revisando las metodologías existentes y los desafíos actuales en el procesamiento simbólico musical.
- **Capítulo 3: Captura de requisitos con musicólogas y formulación de preguntas.** Detalla la fase de captura de requisitos llevada a cabo mediante trabajo de campo. Se describen las entrevistas personales y los formularios web realizados con musicólogas expertas con el fin de definir las necesidades del usuario final. Además, se presenta el banco final de preguntas diseñado para someter a prueba las capacidades analíticas de los modelos de lenguaje y los sistemas implementados.
- **Capítulo 4: Sistema de indexación y recuperación de información basado en RAG Gemini File Search.** Expone la implementación de la primera aproximación tecnológica de este proyecto. Se detalla el pipeline encargado de preparar e indexar los documentos del corpus y la estrategia de recuperación mediante búsqueda semántica utilizando el motor de indexación de *Gemini*.
- **Capítulo 5: Preprocesamiento del corpus musical para su extracción de metadatos.** Constituye el núcleo metodológico y el desarrollo técnico principal de la investigación. Describe detalladamente la implementación del pipeline que extrae de forma automatizada diversas variables musicológicas, estructurales, líricas y de

calidad a partir de la librería *music21*. Asimismo, se explica la configuración del entorno *Model Context Protocol* (MCP) que habilita las consultas relacionales directas sobre un archivo local *SQLite* empleando un modelo pequeño local (*Llama3.1:8B*) y concluye con la implementación de la interfaz de la aplicación web desarrollada.

- **Capítulo 6: Evaluación y pruebas.** Presenta los resultados del proyecto a través de la ejecución del banco de pruebas en ambas arquitecturas. Se contrastan y analizan de forma explícita las respuestas e inferencias proporcionadas por los modelos de lenguaje ante cada una de las cuestiones musicales complejas formuladas.
- **Capítulo 7: Discusión y análisis de limitaciones.** Aborda de manera crítica la comparativa cualitativa y cuantitativa entre el enfoque determinista relacional (*Text-to-SQL*) y la aproximación semántica (*RAG*), discutiendo abiertamente las tasas de alucinación encontradas, los problemas de contexto y las limitaciones técnicas resueltas durante el desarrollo de la arquitectura.
- **Capítulo 8: Conclusiones y Trabajo Futuro.** Sintetiza los hallazgos principales derivados de la investigación, validando el cumplimiento de los objetivos generales y específicos propuestos. Paralelamente, define las líneas de investigación prioritarias e inmediatas, tales como la implementación de arquitecturas multiagente (A2A), técnicas de *GraphRAG* o la explotación de variables semiestructuradas JSON mediante funciones nativas.

Finalmente, la memoria incorpora un **Apéndice A**, donde se recopilan las actividades de transferencia del conocimiento científico, detallando la presentación y el impacto del proyecto en congresos nacionales e internacionales y seminarios académicos del área. Cabe destacar que, para cumplir con las directrices de difusión internacional de la investigación, los capítulos correspondientes a la **Introducción** y las **Conclusiones y Trabajo Futuro** se replican en lengua inglesa en los capítulos 9 y 10, respectivamente.

1.6. Uso de Inteligencia Artificial

Este trabajo ha sido desarrollado con el apoyo puntual de herramientas de inteligencia artificial como asistencia técnica y de redacción. Su uso se ha limitado a tareas auxiliares, tales como la organización de ideas, la mejora de la claridad en la redacción y el apoyo en la implementación de ciertas partes del código. En todos los casos, las decisiones de diseño, la validación de resultados y la responsabilidad final sobre el contenido y la implementación recaen exclusivamente sobre la autora.

Estado de la Cuestión

El desarrollo de una arquitectura capaz de automatizar el análisis, catalogación e interrogación de grandes corpus de música folclórica mediante Inteligencia Artificial requiere converger disciplinas históricamente distantes. Este capítulo examina el marco conceptual y los antecedentes científicos organizados en tres ejes fundamentales: el giro computacional en las humanidades y el procesamiento simbólico de partituras, la emergencia de las tecnologías basadas en Modelos de Lenguaje de Gran Escala (LLMs), y los paradigmas contrapuestos de recuperación de información simbólica y relacional.

2.1. El giro computacional en el análisis de textos artísticos y el procesamiento simbólico musical

En las últimas décadas, las disciplinas humanísticas han experimentado lo que la literatura científica denomina el “*giro computacional*” (*computational turn*), un desplazamiento metodológico que transita desde el análisis microscópico de obras aisladas hacia la explotación macroscópica de grandes volúmenes de datos mediante técnicas de *Distant Reading* Moretti (2013). En el dominio de la musicología, este cambio de paradigma se ha consolidado bajo la disciplina de la musicología computacional, habilitando el estudio de patrones estilísticos, estructuras melódicas y evoluciones folclóricas sobre colecciones que exceden la capacidad de absorción del análisis manual analógico Cook (2004).

El éxito de este enfoque está íntimamente ligado a la estandarización de formatos de representación simbólica digital. Estructuras lógicas y lenguajes de marcado como *MusXML* y, más recientemente, el estándar de la *Music Encoding Initiative* (MEI), han permitido codificar no solo las propiedades acústicas o de rendimiento de una obra (a la manera del formato MIDI estándar), sino su semántica textual y ontología musical compleja, integrando elementos líricos, métricos y estructurales dentro de esquemas arbóreos jerárquicos basados en XML Roland (2002).

No obstante, la manipulación directa de estos ficheros impone una elevada complejidad algorítmica. Para superar esta barrera, la comunidad científica ha desarrollado *frameworks* orientados a objetos específicos para la manipulación musicológica. Entre ellos, el ecosistema **music21** se ha consolidado como el estándar de facto en el ámbito de la investigación Cuthbert y Ariza (2010). Concebido como un conjunto de herramientas de código abierto basadas en Python, *music21* abstrae la complejidad de los formatos de marcado subyacentes, permitiendo a los investigadores programar algoritmos analíticos para la búsqueda de patrones de intervalos, la extracción de líneas melódicas y el cómputo de jerarquías

métricas. A pesar de su robustez, el uso de estos entornos sigue requiriendo competencias avanzadas de programación, lo que introduce una brecha técnica crítica para los investigadores de corte humanístico y musicológico tradicional.

2.2. Modelos de Lenguaje de Gran Escala (LLMs) y el dominio musical simbólico

La llegada de la arquitectura *Transformer* Vaswani et al. (2017) revolucionó el procesamiento del lenguaje natural (NLP) y, de forma paulatina, ha comenzado a influir en las metodologías de análisis de datos estructurados y creativos. La capacidad intrínseca de los Modelos de Lenguaje de Gran Escala (LLMs) para modelar dependencias a largo plazo y capturar representaciones semánticas complejas ha impulsado el desarrollo de iniciativas recientes orientadas a la comprensión y generación de música simbólica.

Trabajos pioneros en este campo exploraron el entrenamiento de modelos de lenguaje autorregresivos sobre representaciones textualizadas de partituras (como notación ABC o secuencias linearizadas de eventos MIDI). Sin embargo, la investigación contemporánea ha detectado limitaciones severas en estos enfoques debido a la naturaleza multidimensional y polifónica de la música, características que la sintaxis puramente secuencial del lenguaje de texto no puede codificar de manera directa Ji et al. (2023).

Recientemente, el paradigma ha evolucionado hacia sistemas multimodales capaces de alinear codificadores musicales especializados con modelos fundacionales de texto. Un ejemplo representativo es la emergencia de arquitecturas de instrucción como *MIDI-LLaMA* o adaptaciones basadas en *MusicBERT*, las cuales buscan que el modelo comprenda estructuras armónicas latentes mediante técnicas de alineamiento de características y ajuste fino conversacional Chou y et al. (2026). A pesar de estos avances de frontera, los modelos generales de última generación (como los pertenecientes a las familias *Gemini* o *Llama*) siguen manifestando deficiencias estructurales cuando operan de manera directa sobre partituras complejas codificadas en lenguaje natural: la densidad sintáctica de los formatos XML satura con rapidez las ventanas de contexto operativo y degrada los mecanismos de atención, provocando fallos sistemáticos en tareas que involucran razonamiento musical abstracto, cálculos matemáticos métricos o análisis de fenómenos expresivos sutiles como sínkopas y hemiolias.

2.3. Recuperación de Información en Grandes Corpus: RAG Semántico frente a Enfoques Relacionales (Text-to-SQL)

Ante la imposibilidad técnica de inyectar colecciones masivas de datos musicales en el contexto de un modelo de lenguaje, la ingeniería de la IA ha bifurcado sus esfuerzos de recuperación de información en dos aproximaciones distintas:

2.3.1. Generación Aumentada por Recuperación (RAG)

El enfoque RAG (*Retrieval-Augmented Generation*) se basa en segmentar el corpus de documentos, proyectar dichos fragmentos en un espacio vectorial continuo mediante modelos de *embeddings* y recuperar de forma semántica las secciones más relevantes de cara a fundamentar la respuesta del generador Lewis et al. (2020). Implementaciones avanzadas proporcionadas por la industria, tales como el motor nativo *File Search* de Google Gemini, han facilitado el despliegue de estos sistemas sobre repositorios documentales extensos.

No obstante, en contextos científicos de alta especificidad (como la musicología técnica), el RAG tradicional vectorizado o basado en texto presenta serios problemas de fiabilidad. Al carecer de capacidades intrínsecas de cómputo estructurado, los modelos de lenguaje tienden a la generación de fenómenos de alucinación, deduciendo métricas inexistentes o inventando referencias líricas o de autoría ausentes en el conjunto original de datos Huang y et al. (2023).

2.3.2. Aproximaciones Relacionales y Text-to-SQL

Con el fin de limitar la variabilidad de los modelos conversacionales puros, la literatura científica ha propuesto el paradigma de traducción de lenguaje natural a lenguaje de consultas estructuradas (*Text-to-SQL*). En este diseño de arquitectura, el modelo de lenguaje no actúa como el almacén o el recuperador indexado de los datos, sino como un agente traductor (un sintetizador de consultas) intercalado entre la interfaz de usuario y un motor de base de datos relacional determinista como *SQLite* Katsogiannis-Meimarakis y Koutrika (2023). Almacenar las variables musicológicas extraídas previamente mediante un procesamiento algorítmico estricto (por ejemplo, computadas previamente con *music21*) garantiza que la fase de recuperación de información sea determinista, auditable y libre de alucinaciones factuales. Sin embargo, este enfoque desplaza la fuente de error hacia la capacidad de inferencia del LLM: si el modelo comete fallos sintácticos al construir la *query* SQL, o si es incapaz de plasmar una condición musicológica abstracta dentro de las restricciones de un esquema de base de datos relacional, el sistema comete fallos críticos de recuperación.

2.3.3. Integración de Contexto y el Protocolo MCP

Como respuesta a la necesidad de estandarizar la conexión segura, bidireccional y eficiente entre agentes de IA y fuentes de datos relacionales locales (o APIs externas), la industria del software ha introducido frameworks de conectividad avanzados como el **Model Context Protocol (MCP)** impulsado a finales de 2024 Anthropic PBC (2024). MCP define un protocolo abierto cliente-servidor que formaliza cómo un modelo de lenguaje puede descubrir y ejecutar herramientas y recursos específicos en su entorno operativo. En lugar de desarrollar conectores acoplados para cada base de datos, el protocolo MCP permite desacoplar la base de datos (por ejemplo, encapsulando las operaciones de extracción *SQLite*) en un servidor independiente con el que el modelo interactúa dinámicamente según el contexto de la consulta planteada por el usuario. La investigación sobre la viabilidad de este protocolo combinado con modelos locales pequeños (SLMs, como *Llama 3.1:8B*) para la ejecución de pipelines científicos específicos y la construcción de aplicaciones web guiadas representa uno de los campos de desarrollo más activos y con mayor potencial de transferencia tecnológica en la ingeniería de software actual.

Captura de requisitos con musicólogas y formulación de preguntas para comprobar las capacidades de los modelos de lenguaje

3.1. Captura de requisitos con musicólogas a través de entrevistas personales

Para recopilar información sobre las necesidades, expectativas y posibles usos de la herramienta desarrollada, se realizaron entrevistas presenciales con musicólogas de la Facultad de Geografía e Historia de la Universidad Complutense de Madrid. Estas entrevistas permitieron obtener información cualitativa más detallada sobre los criterios de búsqueda, análisis y contextualización considerados relevantes en el ámbito de la musicología y el estudio de la música tradicional y folclórica.

Entre los principales intereses identificados destacó la necesidad de realizar búsquedas musicales avanzadas a partir de elementos concretos de las obras, como los incipits musicales (los primeros compases instrumentales o vocales), estructuras melódicas, letras, instrumentación o características interpretativas. Asimismo, las investigadoras señalaron la importancia de disponer de información musicológica detallada, incluyendo aspectos como autoría, fecha, duración, tonalidad, tempo, rango vocal o presencia de grabaciones sonoras.

Otro de los aspectos recurrentes fue el interés por analizar la evolución y transmisión de las obras a lo largo del tiempo, especialmente en el contexto de la música tradicional y folclórica. En este sentido, se destacó la relevancia de estudiar las variaciones entre versiones, los procesos de creación y transmisión oral, así como las relaciones existentes entre músicas de distintas regiones, comunidades y contextos culturales.

Las entrevistas también pusieron de manifiesto la importancia de incorporar funcionalidades orientadas a la comparación y análisis relacional de obras musicales, tales como la identificación de patrones compartidos, la frecuencia de aparición de determinados incipits o leitmotifs musicales dentro de un corpus o la comparación simultánea de canciones dentro de una misma interfaz.

Asimismo, las participantes señalaron la necesidad de integrar información procedente de diferentes catálogos y bases de datos especializadas (como BNE, ISNI, Wikidata o SGAE), incluyendo variantes de nombres y pseudónimos de autores e intérpretes con el fin de facilitar procesos de recuperación y contextualización de la información musical.

Finalmente, las entrevistas evidenciaron el valor de conservar metadatos relacionados con la recopilación y procedencia de las obras, tales como el lugar de recuperación, el contexto de documentación o los datos asociados al proceso de recogida de la información, aspectos especialmente relevantes en investigaciones vinculadas al patrimonio musical y la tradición oral.

3.2. Captura de requisitos con musicólogas a través de formularios

Además de las entrevistas realizadas de manera presencial, se diseñó un formulario estructurado dirigido a potenciales usuarias pertenecientes al ámbito académico e investigador. El objetivo principal de este instrumento fue identificar los perfiles de los participantes, conocer los tipos de consultas que realizarían con la herramienta y comprender el uso previsto de los resultados obtenidos. El cuestionario fue creado y difundido mediante Google Forms, lo que permitió una distribución sencilla y accesible, así como la recopilación organizada de las respuestas.

El cuestionario incluyó, en primer lugar, una pregunta destinada a caracterizar el perfil académico de los participantes, ofreciendo las categorías de investigador/a, estudiante, profesor/a u otras situaciones profesionales relacionadas. Asimismo, se solicitó información sobre el área de investigación de cada encuestada con el fin de contextualizar sus respuestas y analizar posibles diferencias entre disciplinas.

A continuación se incorporaron dos preguntas abiertas orientadas a identificar las consultas que las usuarias podrían considerar relevantes a realizar mediante la herramienta y a documentar el uso que las académicas darían a los datos proporcionados por el sistema tras realizar dicha consulta. La información recopilada permitió, por una parte, recoger ejemplos concretos de necesidades de búsqueda, análisis o recuperación de información, así como otros posibles escenarios de uso no previstos inicialmente por los investigadores y, por otra parte, explorar el potencial impacto de la herramienta en actividades de investigación, docencia, análisis, documentación o generación de nuevo conocimiento.

Las respuestas obtenidas mediante el formulario permitieron identificar distintos intereses y necesidades relacionadas con la investigación musicológica y el desarrollo de herramientas de recuperación y análisis de información musical. Las participantes pertenecían principalmente al ámbito investigador y docente, con especialización en áreas como la música latinoamericana de los siglos XIX y XX, la producción musical y el teatro lírico, así como la musicología y los estudios sobre flamenco y folclore.

Uno de los aspectos más destacados fue el interés por disponer de funcionalidades de búsqueda avanzadas que permitan localizar distintas versiones de una misma obra, clasificar repertorios por estilos o estructuras musicales y acceder a diferentes interpretaciones y grabaciones relacionadas con un mismo material musical. Asimismo, varias respuestas señalaron la importancia de incorporar conexiones con otros recursos externos, como archivos, bibliotecas o bases de datos especializadas, con el fin de contextualizar las obras y establecer relaciones entre distintos repertorios musicales y literarios.

En el ámbito de la música tradicional y el folclore, las respuestas pusieron de manifiesto la relevancia de incluir criterios de búsqueda relacionados con aspectos temáticos, geográficos y socioculturales. Entre ellos destacan la posibilidad de identificar referencias a lugares concretos, analizar temáticas recurrentes en las letras o estudiar variaciones lingüísticas y culturales en función de la región, el idioma o la época histórica.

Asimismo, las musicólogas señalaron el interés de utilizar los datos recuperados para

la construcción de bases de datos propias, el desarrollo de proyectos de investigación, la elaboración de materiales docentes y la creación de repertorios especializados. También se destacó el potencial de la herramienta para generar visualizaciones y mapas interactivos que permitan representar la distribución geográfica y la evolución de determinadas tradiciones musicales.

Otro aspecto recurrente fue la importancia atribuida a la documentación del proceso de recopilación y transmisión de la música tradicional. Se consideró relevante incorporar información sobre la labor desarrollada por folcloristas y etnomusicólogos, incluyendo datos sobre las regiones estudiadas, los repertorios recopilados y las metodologías de recogida empleadas.

En conjunto, los resultados del formulario evidencian la necesidad de una herramienta flexible y adaptada, capaz de integrar información procedente de múltiples fuentes y de ofrecer funcionalidades avanzadas de búsqueda, análisis y contextualización musical. Las respuestas obtenidas refuerzan además la importancia de abordar la música no únicamente como un objeto textual o sonoro aislado, sino también como un fenómeno cultural, histórico y social.

3.3. Formulación de preguntas para comprobar las capacidades de los modelos de lenguaje

Como parte del diseño experimental inicial, se elaboró un conjunto de preguntas teórico-musicales orientadas a evaluar las capacidades potenciales de los modelos de lenguaje pensados para la herramienta en distintos ámbitos de la musicología, el análisis musical y la recuperación avanzada de información musical. El objetivo de este proceso fue definir un repertorio amplio y representativo de consultas que permitiera cubrir tanto aspectos estrictamente musicales —como ritmo, tonalidad, métrica o melodía— como dimensiones documentales, textuales, históricas y técnicas relacionadas con los formatos de codificación musical utilizados en el corpus.

Las preguntas fueron formuladas con distintos niveles de complejidad y especificidad, combinando criterios musicales, metadatos descriptivos, análisis lírico y aspectos estructurales derivados de formatos como MEI, MusicXML o MSCZ. De este modo, se buscó evaluar la viabilidad de consultas complejas capaces de relacionar múltiples dimensiones analíticas dentro de un mismo entorno de recuperación de información.

Para facilitar su organización y análisis, las preguntas fueron agrupadas en diferentes bloques temáticos en función del ámbito musicológico o técnico abordado.

- **Ritmo, métrica y tempo.** Preguntas centradas en la acentuación, figuras rítmicas, síncopas, polirritmias, velocidad (BPM) y la relación matemática/temporal de las notas dentro del compás. Preguntas incluidas:
 - *Identifica piezas en 2/4 que contengan síncopas ligadas entre el segundo pulso de un compás y el primero del siguiente de forma sistemática*
 - *Compara la densidad de notas por compás entre una pieza de 4/4 y una de 2/4 del dataset; ¿cuál tiene mayor promedio de eventos musicales?*
 - *Identifica si existe algún uso de polirritmia en los archivos MEI*
 - *Identifica si existe algún patrón de silencios (<rest />) que se repita sistemáticamente cada cuatro compases en las piezas de métrica ternaria*
 - *Identifica piezas con un compás de 4/4*

- *Identifica piezas con hemiolias horizontales*
 - *Identifica piezas con hemiolias verticales*
 - *Identifica piezas con un bpm de 120*
 - *Identifica piezas con síncopas*
 - *Identifica piezas con polirritmia*
- **Análisis melódico y armónico.** Consultas orientadas a los intervalos musicales, tesituras, movimientos melódicos (ascendentes/descendentes), alteraciones accidentales (sostenidos, bemoles y becuadros), tonalidades y grados de resolución (tónica/dominante). Preguntas incluidas:
- *Localiza todas las notas que tengan bemoles accidentales y dime en qué compases aparecen en los archivos MEI encontrados*
 - *¿Cuál es el intervalo melódico más frecuente entre la última nota de un compás y la primera del siguiente en las piezas de modo mixolidio?*
 - *Identifica piezas donde la nota final sea la dominante (quinto grado) en lugar de la tónica, sugiriendo una estructura abierta*
 - *Identifica piezas con la tonalidad G mayor*
 - *Identifica piezas que estén en modo dórico*
 - *Identifica piezas cuya nota más grave sea A3*
 - *Identifica piezas cuya nota más aguda sea G5*
 - *Identifica obras cuya tendencia melódica sea predominantemente descendente*
 - *Identifica piezas con 4 alteraciones accidentales*
 - *Identifica piezas cuyo intervalo frontera más frecuente sea +5*
- **Relación métrica, lírica y texto.** Preguntas híbridas que analizan cómo interactúa el texto (la lírica, el número de sílabas, palabras clave) con los aspectos musicales del tempo, la altura de las notas o la duración rítmica. Preguntas incluidas:
- *Identifica canciones donde el autor sea una mujer, pero el contenido lírico esté escrito en voz masculina*
 - *Filtra piezas que usen la palabra “estrella” en la lírica y determina si su melodía es predominantemente ascendente o descendente*
 - *Compara el uso de las sílabas “duer” y “me” en los distintos archivos del dataset: ¿se asigna siempre a la misma duración de nota?*
 - *Determina si las canciones con tempo superior a 120 BPM tienden a tener letras con palabras de menor longitud silábica*
 - *Identifica piezas cuyo contenido lírico esté escrito en voz masculina*
 - *Identifica piezas que usen la palabra “bondad”*
 - *Identifica piezas que mencionen el nombre propio “Alfonsito”*
- **Contexto sociocultural, temático y geográfico.** Preguntas complejas que cruzan la etnomusicología o el análisis musical comparativo con datos del entorno (temas como “Nanas” o “Trabajo” o fechas de recolección históricas). Preguntas incluidas:
- *Busca ejemplos de anacrusa en piezas que estén etiquetadas como “Nanas”*

- *Encuentra piezas recolectadas antes de 1950 que traten el tema de “la muerte” y que utilicen exclusivamente un rango melódico de sexta menor*
 - *¿Qué diferencias de tesitura (nota más aguda vs. nota más grave) existen entre las canciones de nana?*
 - *Busca canciones que traten temas de trabajo que compartan la misma estructura de intervalos que las canciones de tema infantil*
 - *Identifica piezas que se llamen El Carro*
 - *Identifica piezas cuyo autor sea Schumann*
 - *Identifica piezas cuyo autor sea de género masculino*
 - *Identifica piezas que traten la muerte como tema*
 - *Identifica piezas de tonalidad A menor que tengan hemiolias y que no superen los 90 bpm*
 - *Identifica piezas de tonalidad A menor que tengan hemiolias*
 - *Lista canciones que compartan la misma armadura de un bemol (F major/D minor) pero que traten temas distintos*
- **Estructura analítica de datos y formatos técnicos.** Este bloque engloba las preguntas que se centran puramente en el código o metadatos de los archivos musicales (MEI, MusicXML, XML, MSCZ), discrepancias de sintaxis musical y atributos técnicos de renderizado (como plicas, volúmenes MIDI o transcripciones). Preguntas incluidas:
- *Encuentra canciones donde el valor de dur.ppq cambie a mitad de la sección, indicando un cambio de resolución rítmica*
 - *Localiza obras que tengan compases donde la suma de las duraciones de las etiquetas <note>no coincida con el meterSig declarado en el staffDef*
 - *¿Qué piezas utilizan valores irregulares (por ejemplo, tresillos) no declarados explícitamente en un <time-modification>del MusicXML?*
 - *Identifica el uso de la etiqueta <sb />(system break) y determina si coincide siempre con el final de una frase musical lógica*
 - *Identifica piezas con un volumen midi de 79*
 - *Identifica piezas cuyo software de codificación haya sido Sibelius 7.1.3*
 - *Identifica piezas cuya fecha de codificación sea el 2015-07-01*
 - *Identifica piezas con compases vacíos*
 - *Identifica piezas con notas de duración 0*
 - *Identifica piezas con plicas anómalas*

Sistema de indexación y recuperación de información basado en RAG Gemini File Search

4.1. Implementación del sistema de indexación y recuperación

Con el objetivo de habilitar la consulta semántica del corpus musical heterogéneo utilizado en el proyecto, se desarrolló un sistema de indexación de documentos basado en la infraestructura de búsqueda de archivos proporcionada por Gemini. El sistema permite la incorporación automática de documentos musicales y metadatos asociados, así como su posterior recuperación mediante consultas en lenguaje natural.

4.1.1. Preparación e identificación de documentos

El proceso comienza con la exploración recursiva del conjunto local de datos que contiene todas las partituras. Se admiten diversos formatos de archivo, incluyendo texto plano (.txt), Markdown (.md), JSON (.json), CSV (.csv), XML (.xml), MEI (.mei) y archivos de partituras comprimidas de MuseScore (.mscz). Cada extensión se asocia explícitamente con un tipo MIME apropiado para facilitar su correcta interpretación por parte del sistema de indexación.

Con el fin de garantizar la compatibilidad con los servicios remotos y evitar problemas derivados del uso de caracteres especiales, todos los nombres de archivo son normalizados mediante una transformación Unicode a ASCII. Posteriormente se eliminan caracteres potencialmente conflictivos, conservando únicamente letras, números y un conjunto reducido de símbolos seguros.

4.1.2. Creación y gestión del almacén de búsqueda de archivos

La infraestructura utiliza un almacén de búsqueda (File Search Store) como repositorio centralizado de documentos indexados. Antes de iniciar la carga de nuevos archivos, el sistema verifica la existencia previa del almacén y, en caso necesario, crea una nueva instancia.

Para evitar la indexación redundante de documentos ya procesados, se recupera el

listado de archivos previamente almacenados. Los nombres normalizados de dichos documentos se emplean como mecanismo de comparación para identificar duplicados y excluirllos de posteriores cargas.

4.1.3. Procesamiento de archivos musicales MuseScore

Los archivos MuseScore comprimidos (.mscz) requieren un tratamiento específico, ya que son contenedores ZIP que almacenan internamente la representación estructurada de la partitura en formato XML (.mscx). Durante la fase de preparación, estos archivos son descomprimidos temporalmente y se extrae el documento XML principal. De esta forma, el contenido musical estructurado puede ser interpretado directamente por el sistema de indexación, mejorando significativamente la capacidad de recuperación semántica frente al almacenamiento de archivos binarios comprimidos.

Los documentos extraídos son renombrados con extensión XML antes de ser enviados al servicio de indexación, manteniendo así la coherencia entre el formato real del contenido y la información de metadatos asociada.

4.1.4. Estrategia de carga e indexación

La incorporación de documentos al sistema de almacenamiento se realiza procesándolos por lotes. Esta estrategia reduce la carga sobre la API, facilita la monitorización del progreso y minimiza la probabilidad de alcanzar límites de uso del servicio.

Para cada documento, el sistema genera una copia temporal con nombre normalizado y envía tanto el archivo como sus metadatos asociados al almacén de búsqueda. Cada operación de carga produce un identificador de proceso asíncrono que es monitorizado periódicamente hasta que la indexación finaliza correctamente.

Tras completar cada lote, el sistema introduce una pausa controlada antes de iniciar el siguiente conjunto de documentos. Este mecanismo actúa como medida preventiva frente a restricciones de frecuencia de acceso y contribuye a mejorar la estabilidad del proceso de ingestión.

4.1.5. Recuperación de información mediante búsqueda semántica

Una vez completada la indexación, el sistema habilita una interfaz de consulta interactiva con el modelo Gemini. Las preguntas formuladas por el usuario se envían al modelo junto con una herramienta de búsqueda conectada al almacén de partituras previamente construido.

Durante la generación de respuestas, el modelo recupera automáticamente los documentos más relevantes para la consulta planteada y los utiliza como contexto adicional. Este enfoque combina técnicas de recuperación de información con RAG, permitiendo responder preguntas complejas sobre el corpus musical sin necesidad de incorporar explícitamente todo el contenido documental en cada solicitud.

Capítulo 5

Preprocesamiento del corpus musical para su extracción de metadatos y posterior almacenamiento en una base de datos SQLite y sistema de MCP para realizar consultas a la base de datos desde un modelo pequeño local

5.1. Introducción a la implementación y pipeline del sistema

Derivar archivos de formato MEI y XML directamente como texto a un LLM general como Gemini dificulta mucho la extracción de información; el modelo tiende a perderse entre la jerarquía de etiquetas de estos ficheros. Procesar los documentos de formato MSCZ supone un desafío mayor para estos modelos, ya que no representan directamente las partituras, sino que son archivos binarios comprimidos que contienen la partitura correspondiente, así como otros metadatos relacionados con la obra y archivos de configuración.

Si, en lugar de intentar analizar una sola partitura, se realizan búsquedas complejas sobre términos musicológicos y técnicos, que exijan cálculos métricos y musicales avanzados, sobre todas las obras del dataset, entonces, los modelos de lenguaje generales resultan incapaces de enfrentarse a estos desafíos.

Para solucionar estas limitaciones, se ha diseñado e implementado un sistema de análisis musical automatizado basado en una arquitectura con herramientas robustas y orientadas a la teoría y análisis de música folclórica y popular. Ya que estas herramientas se encargan de analizar simbólicamente las obras musicales, los modelos no tienen que encargarse de realizar las tediosas búsquedas entre las miles de obras que componen el corpus, sino que su labor se facilita, pudiendo realizar consultas simples en una base de datos compuesta por la información extraída previamente de las canciones. Para obtener la información que describe las obras se ha optado por utilizar la librería *music21*.

Una vez realizado el preprocesamiento del corpus, el usuario ya puede realizar consultas a la herramienta. Para responder, el modelo accede a la base de datos con la información sobre las obras a través de una arquitectura MCP (*Model Context Protocol*) que proporciona

los recursos necesarios para extraer la información relevante con *queries*.

Durante el proyecto se ha desarrollado un sistema de análisis musical bajo una arquitectura secuencial dividida en cinco fases diferenciadas:

- **Fase de descubrimiento de archivos.** Inspección recursiva del directorio raíz del corpus para identificar y catalogar archivos con extensiones compatibles (.xml, .musicxml, .mxl, .mei, etc.).
- **Fase de preprocesamiento y normalización de archivos.** Limpieza de cadenas de texto para corregir la aparición de, por ejemplo, caracteres no aptos o espacios en los nombres de archivo de las obras, e incompatibilidades de sintaxis entre herramientas como *music21* y los formatos utilizados antes de la llamada al intérprete musical.
 - **Procesamiento específico de documentos MSCZ.** Los ficheros MSCZ son archivos binarios comprimidos por MuseScore que contienen una partitura en formato MSCX, metadatos sobre la obra, como archivos de configuración, audio, renderizado, y otros recursos. Al ofrecer este tipo de documentos a los modelos, estos no interpretan la partitura comprimida, sino que analizan el fichero binario que envuelve el resto de información.
 Respuesta de Gemini 2.5 Flash al intentar procesar un documento MSCZ: *I cannot provide information about "the song" without more specific details. The search results contain technical file information such as 'score_style.mss', '568.msx', 'audiosettings.json', and 'viewsettings.json', which appear to be related to a music score file and its settings, but do not identify a particular song by title, artist, or content.*
 Para procesar y extraer la información musicológica de la obra, se descomprime primero el documento para separar la partitura y posteriormente se analiza de manera genérica, siguiendo el procedimiento utilizado para otros tipos de archivos, como MEI o XML.
- **Fase de análisis simbólico y extracción de metadatos.** Carga de la información sobre cada obra en memoria para extraer de ella datos como la tonalidad, el compás, el tempo, la inferencia híbrida de la tonalidad y la detección de patrones rítmicos.
- **Fase de análisis semántico de la lírica mediante LLMs locales.** Extracción del texto lírico estructurado de cada obra y su posterior envío a un LLM local para su etiquetado. El modelo determina qué temas trata la letra, asignándole, como máximo, cuatro temáticas a cada canción (Nana, Religión, Naturaleza, Picaresca, Amor, etc.).
- **Fase de consolidación, estructuración y almacenamiento en la base de datos SQLite.** Compilación de los diccionarios que contienen las características mencionadas con anterioridad y su exportación a una base de datos SQLite.

5.2. Herramientas desarrolladas para la extracción de información de las obras

Para mejorar el rendimiento de la búsqueda de características concretas en el corpus se han diseñado e implementado una serie de funciones que abordan de manera más específica y técnica, basándose en teoría musical, los desafíos de extracción musical y semántica que presentan los motores de búsqueda con modelos de IA generales.

5.2.1. Extracción de tempo

Con el objetivo de caracterizar el tempo global de las obras del corpus, se implementó un sistema de extracción automática de la indicación de tempo a partir de la representación simbólica de la partitura.

El método utilizado para esta extracción de tempo es `_extract_bpm`.

El método recorre la estructura jerárquica del objeto musical en busca de elementos de tipo `MetronomeMark`, los cuales contienen información explícita sobre la velocidad de ejecución expresada en pulsaciones por minuto (BPM). Dado que una obra puede presentar múltiples cambios de tempo a lo largo de su desarrollo, el algoritmo selecciona la primera marca de metrónomo válida encontrada durante el recorrido de la partitura.

Cuando se identifica una indicación de tempo con un valor numérico definido, este se extrae y se redondea al entero más cercano con el fin de obtener una representación estandarizada del tempo. Este valor se interpreta como una aproximación del tempo inicial o predominante de la obra.

En caso de que no exista ninguna indicación explícita en la partitura, el método devuelve un valor nulo, indicando la ausencia de información de tempo disponible en la codificación.

El resultado final consiste en un valor entero que representa el tempo en BPM.

5.2.2. Extracción de compás

Con el objetivo de identificar el compás de cada obra del corpus, se implementó un procedimiento de extracción de la indicación de compás a partir de la estructura simbólica de la partitura.

El método utilizado para esta extracción de compás es `_extract_compas`.

El procedimiento recorre la representación interna del objeto musical en busca de elementos de tipo `TimeSignature`, que contienen la información relativa a la métrica escrita en la partitura. Dado que una obra puede incluir múltiples indicaciones de compás a lo largo de su desarrollo, el algoritmo selecciona la primera ocurrencia válida encontrada en el recorrido jerárquico de la estructura musical.

Una vez identificada una indicación métrica válida, el sistema extrae su representación en forma de ratio (por ejemplo, $3/4$, $6/8$ o $4/4$), la cual describe la relación entre pulsos y unidad de tiempo dentro del compás. Esta información se considera representativa de la métrica inicial o predominante de la obra.

En caso de no encontrarse ninguna indicación de compás válida, el procedimiento devuelve un valor nulo, indicando la ausencia de información métrica explícita en la partitura analizada.

El resultado final consiste en una cadena de texto que representa el compás de la obra.

5.2.3. Extracción híbrida de la tonalidad

Con el objetivo de determinar la tonalidad de las obras del corpus, se implementaron dos sistemas complementarios de inferencia tonal: un método basado en análisis armónico automático y otro basado en heurísticas estructurales que combinan información de armadura y distribución de alturas.

El método principal utilizado para esta extracción de la tonalidad es `_extract_tonality`.

Este sistema emplea las capacidades de análisis tonal integradas en la librería `music21`. A partir de la partitura completa, se ejecuta un algoritmo de estimación de tonalidad que

identifica la relación más probable entre la tónica y el modo (mayor o menor) en función del contenido armónico global de la obra. El resultado se expresa como una combinación de nota fundamental y modalidad.

En caso de que el análisis no sea posible o no se disponga de información suficiente, el método devuelve un valor nulo, garantizando así la robustez del proceso ante entradas incompletas o mal formateadas.

De forma complementaria, se implementó un segundo método denominado `_infer_tonality_custom`, diseñado para reforzar o sustituir la estimación automática en aquellos casos en los que la información tonal es ambigua o incompleta.

Este método parte de la armadura de clave expresada en número de alteraciones (sostenidos o bemoles), la cual se extrae directamente de la partitura. A partir de este valor se establece un conjunto de tonalidades candidatas, que incluye tanto la tonalidad mayor como su relativa menor correspondiente. Esta relación sigue el sistema tonal clásico basado en armaduras.

Posteriormente, el algoritmo analiza la última nota de la obra como posible indicador de resolución tonal, bajo la hipótesis de que muchas tradiciones musicales tienden a finalizar en la tónica. Esta nota final se utiliza como criterio heurístico para discriminar entre las dos tonalidades candidatas.

En los casos en los que la última nota no coincide claramente con ninguna de las tónicas posibles, se recurre a un criterio estadístico basado en la frecuencia de aparición de las alturas. Concretamente, se contabiliza la ocurrencia de las notas correspondientes a las tónicas mayor y menor, seleccionando aquella que presenta una mayor presencia relativa dentro de la obra.

5.2.4. Extracción de título y autor

Con el objetivo de normalizar la información descriptiva asociada a cada obra del corpus, se implementó un procedimiento de extracción automática del título y del compositor o autor de la pieza.

El método utilizado para esta extracción de metadatos es `_extract_title_and_author`.

El procedimiento toma como entrada un objeto estructurado de tipo Score junto con la ruta original del archivo. En primer lugar, se accede a los metadatos internos del objeto musical. A partir de dicha información se extraen el título de la obra (o el nombre del movimiento) y el compositor o entidad responsable de la composición.

Dado que los metadatos en colecciones musicales digitalizadas pueden contener inconsistencias o valores generados automáticamente durante procesos de conversión, el algoritmo incorpora un conjunto de reglas de filtrado y depuración. En particular, se consideran inválidos aquellos títulos que corresponden a valores temporales o de sistema, tales como cadenas prefijadas con “tmp”, así como aquellos derivados directamente del nombre del archivo original. Estos últimos casos se descartan.

En los casos en los que el título no está disponible o ha sido descartado por los criterios de limpieza, se utiliza como alternativa el nombre del archivo (sin extensión) como identificador de la obra. Esta estrategia garantiza la existencia de un identificador mínimo para cada elemento del corpus, incluso en ausencia de metadatos formales.

El resultado final del procedimiento consiste en un par de valores normalizados: el título de la obra y el nombre del autor o compositor, cuando este último está disponible.

5.2.5. Inferencia del modo de la obra

Con el objetivo de caracterizar la organización modal de las piezas del corpus, se implementó un sistema de inferencia automática de modo musical basado en la distribución temporal de las alturas. Esta metodología permite estimar tanto la tónica de referencia como el modo predominante de una obra a partir de la duración acumulada de los distintos grados.

El método utilizado para esta inferencia del modo es `_infer_mode_custom`.

Este método parte de la identificación de todas las notas presentes en la partitura. En ausencia de información explícita sobre la tonalidad o el centro modal, se adopta una estrategia heurística que considera la última nota de la obra como posible candidata a tónica. Esta decisión se fundamenta en prácticas frecuentes en repertorios de tradición modal y en música de transmisión oral, donde la resolución final tiende a coincidir con el centro tonal percibido.

Una vez establecida la tónica de referencia, todas las alturas del conjunto se proyectan al espacio de clases de altura mediante su reducción módulo 12, obteniendo así una representación cromática independiente de la octava. A continuación, se calcula la duración total asociada a cada intervalo respecto a la tónica, sumando la duración de todas las notas que comparten la misma clase de altura.

Este proceso genera un perfil de distribución temporal de las alturas, en el que cada intervalo (de 0 a 11 semitonos) queda asociado a la cantidad total de tiempo musical que ocupa dentro de la obra. Esta representación permite cuantificar la relevancia estructural de cada grado en función de su duración acumulada, en lugar de tener sólo en cuenta su frecuencia de aparición.

Posteriormente, el algoritmo compara este perfil con los patrones interválicos característicos de los siete modos diatónicos tradicionales: jónico, dórico, frigio, lidio, mixolidio, eólico y locrio. Cada modo se define como un conjunto de grados escalares relativos a la tónica, lo que permite construir plantillas modales teóricas.

Para cada modo se calcula una puntuación global que suma las duraciones correspondientes a los grados que lo componen. De este modo, los intervalos que forman parte de un modo contribuyen positivamente a su puntuación en función del tiempo musical que ocupan en la obra.

El modo final se obtiene seleccionando aquel cuya plantilla acumula la mayor cantidad total de duración, interpretándose este valor como el mejor ajuste entre el material musical observado y los modelos modales teóricos considerados.

5.2.6. Detección de hemiolias verticales

Con el objetivo de identificar fenómenos de superposición métrica simultánea dentro de una partitura, se implementó un sistema de detección automática de hemiolias verticales. Esta metodología busca localizar compases en los que diferentes voces presentan organizaciones rítmicas incompatibles, generando una coexistencia de agrupaciones binarias y ternarias dentro de una misma estructura métrica.

El método utilizado para esta detección de hemiolias verticales es `detectar_hemiolias_verticales`.

La detección se centra en los compases de $3/4$ y $6/8$, ya que constituyen los contextos métricos más habituales para la aparición de este tipo de hemiolias. Ambos compases poseen una duración temporal equivalente, pero difieren en la organización de sus pulsos principales. Mientras que el compás de $3/4$ se articula mediante tres pulsos regulares, el

compás de 6/8 suele estructurarse como dos pulsos principales con subdivisión ternaria. La interacción simultánea entre ambas formas de organización constituye el fundamento de la hemiolia vertical.

Como paso inicial, la partitura se carga y procesa para acceder a su estructura interna. Posteriormente, el algoritmo recorre todas las partes y analiza individualmente cada compás cuya indicación métrica corresponda a 3/4 o 6/8. Los compases pertenecientes a otros contextos métricos son excluidos del análisis.

Para que pueda existir una hemiolia vertical, es necesario que el compás contenga al menos dos voces independientes. Por esta razón, el sistema descarta aquellos compases con una sola voz donde no es posible establecer relaciones simultáneas entre diferentes patrones de acentuación.

Una vez identificadas las voces presentes en el compás, el algoritmo extrae las posiciones temporales de todos los ataques sonoros contenidos en cada una de ellas. Estas posiciones se representan mediante conjuntos de desplazamientos temporales (*offsets*) expresados en unidades musicales normalizadas.

A continuación, el método compara todas las combinaciones posibles de voces presentes en el compás. Para cada par de voces se evalúa si sus patrones de ataque corresponden a agrupaciones binarias o ternarias. La clasificación se realiza mediante funciones especializadas que identifican configuraciones características de cada organización métrica a partir de la distribución de los ataques sonoros.

La detección de la hemiolia vertical se produce cuando dos voces simultáneas presentan agrupaciones rítmicas incompatibles, concretamente cuando una voz exhibe una organización binaria mientras otra manifiesta una organización ternaria dentro del mismo compás. Esta coexistencia genera una percepción simultánea de diferentes estructuras métricas y constituye el criterio operativo utilizado para definir la hemiolia vertical.

Cuando se identifica al menos una combinación de voces que satisface esta condición, el número del compás se registra como un caso de hemiolia vertical. El proceso continúa examinando el resto de la obra hasta completar el análisis de todas las partes y compases.

Finalmente, el método devuelve una lista ordenada con los números de los compases donde se ha detectado este fenómeno.

5.2.7. Detección de hemiolias horizontales

Con el objetivo de identificar fenómenos de reinterpretación métrica sucesiva dentro de una partitura, se implementó un sistema de detección automática de hemiolias horizontales. Esta metodología busca localizar compases en los que, manteniéndose constante la indicación de compás escrita, se produce un cambio significativo en el patrón interno de acentuación rítmica, generando una percepción métrica diferente a la esperada.

El método utilizado para esta detección de hemiolias horizontales es `detectar_hemiolias_horizontales`.

La detección se centra en la relación tradicional entre los compases de 6/8 y 3/4, dos estructuras métricas que poseen la misma duración total pero distribuyen los acentos de manera diferente. Mientras que el compás de 6/8 suele organizarse como dos pulsos principales subdivididos ternariamente, el compás de 3/4 se articula como tres pulsos regulares de subdivisión binaria. La alternancia perceptiva entre ambas organizaciones constituye una de las manifestaciones más características de la hemiolia.

Como paso inicial, la partitura se carga y procesa para obtener todas las partes y compases de la obra. El análisis se restringe exclusivamente a compases escritos en 6/8 o 3/4, ya que son las configuraciones métricas sobre las que se fundamenta la definición

operativa utilizada por el algoritmo.

Para cada compás se recogen las posiciones temporales de los ataques sonoros presentes. Estas posiciones se utilizan como indicadores de los puntos de apoyo rítmico predominantes y permiten inferir la organización métrica que emerge de la distribución de los eventos musicales.

La clasificación del patrón métrico percibido se basa en la presencia de ataques en posiciones estratégicas del compás. Se considera que un compás presenta una articulación característica de $3/4$ cuando aparecen apoyos en los tres pulsos principales de la estructura ternaria, correspondientes a las posiciones 0.0, 1.0 y 2.0 expresadas en unidades de negra. Por el contrario, se interpreta una organización propia de $6/8$ cuando los ataques principales se concentran en las posiciones 0.0 y 1.5, representativas de los dos pulsos dominantes del compás compuesto.

A partir de esta información, el algoritmo asigna a cada compás una categoría que representa su métrica percibida. Esta clasificación describe cómo se organiza realmente la acentuación interna, independientemente de la indicación de compás escrita en la partitura.

Los compases que satisfacen estos criterios se registran como casos de hemiolia horizontal y se incorporan al conjunto de resultados. Finalmente, el método devuelve una lista ordenada con los números de los compases donde se ha detectado este fenómeno.

5.2.8. Detección de síncopas

Con el objetivo de identificar fenómenos de desplazamiento acentual dentro de una partitura, se implementó un sistema de detección automática de síncopas. Esta metodología busca localizar compases en los que una nota o acorde comienza en una posición métrica débil y prolonga su duración atravesando una posición de mayor jerarquía métrica, generando una suspensión temporal del acento esperado.

El método utilizado para esta detección de síncopas es `detectar_sincopas`.

Como paso inicial, la partitura se carga y procesa para obtener una representación estructurada de sus elementos musicales. Posteriormente, el algoritmo recorre todas las partes instrumentales y analiza cada compás de forma independiente.

Para cada compás, el sistema examina todas las notas y acordes presentes, ignorando los silencios y cualquier otro elemento que no represente un evento sonoro. De cada evento se recuperan su posición de inicio dentro del compás y su duración, ambas expresadas en unidades de negra.

La detección de síncopas se fundamenta en la estructura jerárquica de acentos implícita en la indicación de compás. Para cada evento musical se calcula la fuerza métrica asociada a su punto de inicio. Esta magnitud representa el nivel de acentuación relativo de la posición donde comienza la nota o el acorde dentro del compás.

A continuación, el algoritmo determina si la duración del evento atraviesa alguno de los puntos de articulación métrica presentes en el compás. Para ello se genera una serie de referencias temporales correspondientes a los distintos pulsos que lo subdividen. Cada uno de estos puntos es un posible candidato a representar una posición de mayor relevancia métrica.

Para cada pulso atravesado por el evento se evalúa su fuerza métrica mediante la información proporcionada por la indicación de compás. Cuando se identifica un punto cuya jerarquía acentual es superior a la del instante de inicio de la nota, el algoritmo interpreta que el evento está desplazando el acento musical esperado. En consecuencia, se considera que existe una síncopa.

La detección se basa, por tanto, en dos condiciones simultáneas. En primer lugar, el

evento debe comenzar en una posición relativamente débil dentro de la estructura métrica. En segundo lugar, su duración debe extenderse más allá de un punto de mayor acentuación sin que se produzca un nuevo ataque sonoro en dicho instante. Esta definición permite distinguir las síncopas de otras configuraciones rítmicas que simplemente presentan notas en posiciones débiles pero no generan una transferencia efectiva del acento.

Cuando se detecta al menos una síncopa dentro de un compás, el número de dicho compás se incorpora al conjunto de resultados. Finalmente, el método devuelve tres indicadores: una variable binaria que señala la presencia o ausencia de síncopas en la obra, una lista con los números de los compases afectados y el número total de compases donde se ha identificado este fenómeno rítmico.

5.2.9. Extracción de letras

Con el objetivo de incorporar información textual al proceso de análisis musical, se implementó un sistema de extracción automática de letras capaz de operar sobre diversos formatos de representación musical digital. Esta metodología permite recuperar el texto cantado de una obra y reconstruirlo en forma legible a partir de la información silábica almacenada en la partitura.

El método principal utilizado para esta extracción es `_extract_lyrics`.

Dado que cada formato musical emplea estructuras internas diferentes para representar las letras, el sistema incorpora mecanismos específicos de extracción para archivos MEI, MusicXML y MuseScore.

En todos los casos, el algoritmo identifica los elementos textuales contenidos en la partitura y los agrupa según el número de verso o estrofa al que pertenecen. Este paso resulta esencial para evitar la mezcla de textos procedentes de diferentes líneas poéticas o variantes simultáneas de una misma canción.

La reconstrucción de las palabras se realiza utilizando la información por sílabas almacenada en cada formato. Los elementos textuales pueden aparecer etiquetados como palabras completas o como sílabas que representan el inicio, la continuación o el final de una palabra. A partir de esta información, el algoritmo concatena las sílabas correspondientes hasta reconstruir cada término completo respetando su orden original dentro de la partitura.

En el caso de los archivos MEI, el sistema analiza los elementos `verse` y `syl`. Cada sílaba incorpora un atributo `wordpos` que indica su posición dentro de la palabra. Los valores correspondientes a inicio, interior y final permiten reconstruir secuencias silábicas completas antes de incorporarlas al texto final. Las distintas estrofas se procesan de forma independiente y posteriormente se concatenan respetando su numeración original.

Para los archivos MusicXML, la extracción se basa en los elementos `lyric`. La posición silábica se obtiene mediante el elemento `syllabic`, cuyos valores distinguen palabras completas de sílabas iniciales, intermedias y finales. Además, se realizan tareas de normalización textual destinadas a eliminar ciertos marcadores de notación fonética o melismática que podrían dificultar los análisis posteriores del contenido lingüístico.

En el caso de MuseScore XML (`.mscx`), el algoritmo procesa los elementos `Lyrics` y utiliza la información de numeración de estrofas y sílabas para reconstruir las palabras. El procedimiento sigue una estrategia equivalente a la utilizada en MusicXML, adaptada a la estructura específica del formato MuseScore.

Los archivos comprimidos MuseScore (`.mscz`) requieren una etapa adicional de procesamiento. En estos casos, el sistema localiza y extrae temporalmente el documento `.mscx` contenido en el archivo comprimido y aplica posteriormente el mismo procedimiento de

reconstrucción textual descrito para dicho formato.

Una vez completada la reconstrucción de todas las estrofas, el sistema genera una representación textual continua de la letra completa de la canción. Durante este proceso se eliminan espacios redundantes y se normalizan ciertos caracteres especiales con el fin de obtener una salida homogénea e independiente del formato original de almacenamiento.

El resultado final consiste en una cadena de texto que contiene la letra reconstruida de la obra.

5.2.10. Análisis temático de la lírica de las obras con LLMs

Con el objetivo de enriquecer la descripción semántica de las canciones del corpus, se implementó un sistema de extracción automática de temas basado en LLMs. Esta metodología permite identificar de forma automática los tópicos predominantes presentes en el texto de una canción a partir de su título y de su letra.

El método utilizado para este análisis es `_analizar_temas_con_ollama`.

El procedimiento recibe como entrada el título y la letra de una canción. Cuando ambos campos se encuentran vacíos, el análisis se omite y el método devuelve un resultado nulo. En caso contrario, la información textual se utiliza para construir una consulta dirigida a un modelo de lenguaje ejecutado localmente mediante Ollama.

Para guiar el proceso de clasificación, el algoritmo genera un mensaje de instrucción que incorpora explícitamente tanto el título como la letra de la canción. Dicho mensaje solicita la identificación de los temas más representativos de la obra, limitando la respuesta a un máximo de cuatro categorías. Entre los ejemplos de referencia proporcionados al modelo se incluyen etiquetas habituales en el ámbito del folclore musical, como nana, naturaleza, muerte, religión, amor, trabajo, picaresca, fiesta, sátira, infancia, guerra o viaje.

Además de la consulta principal, se define un mensaje de sistema que establece el comportamiento esperado del modelo. Este mensaje especifica que el análisis debe realizarse desde una perspectiva musicológica y folclórica, y restringe estrictamente el formato de salida.

La comunicación con el modelo se realiza mediante una llamada a la interfaz local de Ollama. El sistema solicita una respuesta completa en una única operación y activa el modo de generación estructurada en formato JSON, reduciendo así la probabilidad de respuestas ambiguas o difíciles de procesar automáticamente.

Una vez recibida la respuesta, el algoritmo interpreta el contenido JSON generado por el modelo y recupera la lista de temas identificados. Estas etiquetas se incorporan posteriormente al conjunto de metadatos que describen la canción.

En caso de producirse errores durante la comunicación con el modelo, fallos de procesamiento o respuestas inválidas, el procedimiento devuelve una lista vacía sin interrumpir el flujo general de análisis.

5.2.11. Detección de cambio de resolución rítmica

Con el objetivo de evaluar la consistencia de la representación temporal interna de las partituras codificadas en formato MEI, se implementó un sistema para detectar cambios de resolución rítmica durante el transcurso de una obra. Esta metodología permite identificar modificaciones en la relación entre las figuras musicales y los pulsos utilizados para representar su duración.

El método utilizado para esta detección es `_detectar_cambio_resolucion_ppq`.

La métrica se basa en el concepto de resolución temporal expresada mediante PPQ

(*Pulses Per Quarter Note*), un parámetro que define el número de pulsos internos que corresponden a una negra. Cuando la resolución permanece estable, una misma figura musical mantiene siempre la misma equivalencia en pulsos. Por el contrario, si esta relación cambia dentro de una misma sección de la obra, pueden aparecer inconsistencias que afecten a la interpretación temporal de los datos.

Como paso inicial, el algoritmo verifica que el archivo analizado corresponda al formato MEI y procesa su estructura XML. Posteriormente, la obra se divide en sus distintas secciones musicales (**section**) que se analizan de forma independiente.

Para cada sección se mantienen dos mecanismos de control. El primero consiste en registrar el valor de resolución PPQ declarado explícitamente en las definiciones de partitura (**scoreDef**) o pentagrama (**staffDef**). El segundo almacena la correspondencia observada entre cada figura rítmica nominal y el número de pulsos asignados mediante el atributo **dur.ppq**.

La detección de cambios se realiza mediante dos estrategias complementarias. La primera estrategia examina las redefiniciones explícitas de resolución que puedan aparecer a lo largo de la partitura. Durante el recorrido secuencial de los compases, el algoritmo inspecciona las cabeceras locales contenidas en cada compás. Cuando se encuentra un atributo **ppq** cuyo valor difiere de la resolución previamente activa, el compás correspondiente se registra como un posible punto de cambio de resolución temporal.

La segunda estrategia analiza directamente las duraciones codificadas en los eventos musicales. Para cada nota, acorde o silencio que disponga simultáneamente de los atributos **dur** y **dur.ppq**, el algoritmo recupera la figura rítmica y el número de pulsos efectivos asociados a dicha figura. A continuación, se construye un mapa que relaciona cada figura musical con su equivalencia en pulsos dentro de la sección analizada.

Cuando una figura previamente observada reaparece asociada a un número de pulsos diferente, el algoritmo interpreta que la resolución temporal ha cambiado. Por ejemplo, si una negra se codifica inicialmente con un determinado número de pulsos y posteriormente aparece representada con otro valor distinto dentro de la misma sección, se considera que existe una inconsistencia en la resolución PPQ utilizada para describir la obra.

Todos los compases donde se detecta alguna de estas situaciones son registrados y añadidos al conjunto de resultados. Finalmente, el método devuelve un indicador binario que señala la presencia o ausencia de cambios de resolución temporal y una lista con los números de los compases afectados.

5.2.12. Detección de desajustes en duraciones de notas

Con el objetivo de evaluar la consistencia métrica de las partituras, se implementó un sistema de detección automática de desajustes entre el contenido rítmico de los compases y la indicación de compás declarada en la obra. Esta metodología permite identificar errores de codificación, omisiones o inconsistencias que provocan que la duración de una voz no coincida con la capacidad temporal establecida por el compás vigente.

El método utilizado para esta detección es **detectar_desajustes_meter_sig**.

Este método mantiene en todo momento un estado interno que representa la indicación de compás activa. Inicialmente, este estado se obtiene a partir de las definiciones globales de la obra. Posteriormente, durante el recorrido secuencial de los compases, el algoritmo actualiza dicha información cuando detecta nuevas definiciones métricas locales. De este modo, se garantiza que los cambios de compás se tengan en cuenta exactamente en el punto donde aparecen en la partitura.

Para cada compás se calcula primero su duración teórica esperada en unidades de negra.

Por ejemplo, un compás de $3/4$ posee una duración total de tres negras, mientras que un compás de $6/8$ equivale igualmente a tres negras.

Una vez determinada la capacidad temporal esperada del compás, el algoritmo analiza cada voz de manera independiente. Este enfoque permite detectar inconsistencias que podrían quedar ocultas al considerar únicamente la suma global de todas las voces.

El cálculo de la duración real de cada voz se realiza recorriendo secuencialmente los elementos musicales que la componen. Se consideran notas individuales, acordes y silencios. En el caso de los acordes, la duración se obtiene exclusivamente del elemento contenedor **chord**, evitando contabilizar varias veces las notas internas que lo componen.

Las duraciones de todos los eventos pertenecientes a una misma voz se acumulan para obtener la duración total del compás. Posteriormente, este valor se compara con la duración teórica calculada a partir de la indicación métrica vigente.

Cuando la duración total de una voz difiere significativamente de la duración esperada, el compás se considera inconsistente desde el punto de vista métrico. En ese caso, el número de compás se registra como un caso de desajuste. Basta con que una única voz presente la discrepancia para que el compás completo sea marcado como potencialmente erróneo.

Finalmente, el método devuelve un indicador binario que señala la presencia o ausencia de desajustes métricos en la partitura y una lista de los compases afectados.

5.2.13. Detección de valores irregulares ocultos (grupillos)

Con el objetivo de evaluar la calidad y consistencia de la codificación rítmica de las partituras, se implementó un procedimiento para la detección de valores irregulares ocultos. Estos casos corresponden a eventos musicales cuyas duraciones representan subdivisiones no convencionales (como tresillos, quintillos o septillos) pero que no han sido declaradas explícitamente mediante los mecanismos estándar de notación.

El método utilizado para esta detección es `detectar_valores_irregulares_ocultos`.

Tras cargar la partitura, el método recorre todas las partes instrumentales y examina cada uno de sus compases. Para cada compás se inspeccionan las notas y acordes presentes, ignorando aquellos elementos que no representan eventos sonoros válidos o cuya duración sea nula.

El análisis se centra en la duración de cada evento musical, expresada en unidades de negra. Con el fin de minimizar posibles errores derivados de la representación en coma flotante, cada duración se transforma en una fracción cuyo denominador queda limitado a un valor máximo predefinido.

A continuación, el algoritmo examina el denominador de cada fracción obtenida. En la notación rítmica convencional, las subdivisiones regulares se basan exclusivamente en potencias de dos, por lo que sus denominadores adoptan valores como 1, 2, 4, 8, 16 o 32. Mediante una comprobación binaria eficiente se determina si el denominador corresponde a una potencia de dos. Cuando esta condición no se cumple, la duración se considera matemáticamente irregular, indicando la posible presencia de agrupaciones como tresillos, quintillos u otras subdivisiones no binarias.

La detección definitiva de un valor irregular oculto requiere una segunda verificación. Además de presentar una duración irregular, el evento no debe contener información explícita de grupillo en sus metadatos internos. Cuando ambas condiciones se satisfacen simultáneamente, el algoritmo interpreta que existe una irregularidad rítmica implícita u oculta en la codificación de la partitura.

Los compases donde se detecta al menos un caso de este tipo son registrados y añadidos al conjunto de resultados. Finalmente, el método devuelve tres indicadores: una variable

binaria que señala la presencia o ausencia de valores irregulares ocultos en la obra, una cadena de texto con los números de los compases afectados y el número total de compases en los que se ha identificado esta anomalía.

5.2.14. Cálculo de densidad de eventos musicales por compás

Con el objetivo de cuantificar la actividad musical presente en una partitura, se implementó un sistema de cálculo de densidad de eventos. Esta métrica permite estimar el grado de concentración de eventos sonoros a lo largo de la obra a través de la relación entre el número total de eventos musicales y la cantidad de compases de la pieza.

El método utilizado para esta extracción de densidad de eventos es `_calcular_densidad_eventos`.

Este método recibe como entrada una partitura completa representada como un objeto `Score`. En primer lugar, se realiza un recorrido exhaustivo de toda la estructura musical para localizar los eventos sonoros presentes en la obra. Para ello, se consideran exclusivamente notas individuales y acordes, mientras que los silencios son descartados del análisis al no aportar información sobre la actividad sonora.

A continuación, el algoritmo contabiliza el número total de eventos identificados. Cada nota y cada acorde se consideran una única unidad de análisis, independientemente de su duración o complejidad interna. El resultado de este paso proporciona una medida absoluta de la cantidad de material musical contenido en la partitura.

Posteriormente, se determina el número total de compases reales de la obra. Dado que una partitura puede contener múltiples voces, partes instrumentales o capas simultáneas que comparten la misma numeración de compás, el procedimiento agrupa los compases mediante un conjunto de valores únicos. Para ello, se recorren todos los objetos de tipo compás presentes en la partitura y se registra únicamente su número identificador. Esto evita contabilizar varias veces un mismo compás cuando aparece replicado en distintas partes instrumentales.

Una vez obtenidos el número total de eventos y el número de compases, se calcula la densidad media de la obra como el cociente entre ambas magnitudes. Esta relación expresa el número promedio de eventos musicales por compás y constituye una medida global de la densidad estructural de la partitura. Con el fin de facilitar su interpretación, el resultado se redondea a dos cifras decimales.

Finalmente, el método devuelve tres valores: el número total de eventos musicales detectados, el número total de compases únicos presentes en la obra y la densidad media calculada.

5.2.15. Detección automática de polirritmias verticales

Con el objetivo de identificar fenómenos de superposición rítmica compleja dentro de una partitura, se implementó un sistema de detección automática de polirritmias verticales. Esta metodología busca localizar compases en los que coexisten simultáneamente subdivisiones rítmicas incompatibles entre distintas voces o partes instrumentales, tales como tresillos frente a subdivisiones binarias regulares o combinaciones más complejas como 3:4, 3:5 o 5:7.

El método utilizado para esta detección de polirritmias es `detectar_polirritmias`.

Este método recibe como entrada una partitura y la procesa para facilitar su análisis. Posteriormente, todas las medidas de la obra se agrupan según su número de compás, permitiendo examinar conjuntamente el contenido rítmico que ocurre de manera simultánea

en cada instante de la obra.

Para cada compás, el algoritmo inspecciona todas las notas y acordes presentes en las distintas voces. El análisis se realiza sobre las duraciones musicales expresadas en unidades de negra, excluyendo aquellos eventos cuya duración resulta inválida o nula. A partir de esta información se identifican las subdivisiones rítmicas utilizadas en cada contexto.

La detección de subdivisiones irregulares se lleva a cabo mediante dos estrategias. La primera consiste en examinar explícitamente la información de grupillos (tresillos, dosillos, seisillos...) almacenada en la partitura. Cuando un evento contiene metadatos que describen agrupaciones irregulares, se recupera la relación entre el número de notas ejecutadas y el número de notas esperadas según la subdivisión convencional. Toda discrepancia entre ambos valores se interpreta como una subdivisión no regular y se registra para el análisis posterior.

La segunda estrategia actúa como mecanismo de respaldo para aquellos casos en los que la información de los grupillos no aparece de manera explícita en la partitura. En este caso, las duraciones se transforman en fracciones racionales y se analiza su denominador. El algoritmo elimina sucesivamente todos los factores asociados a potencias de dos, que representan subdivisiones binarias convencionales. Los factores restantes permiten identificar subdivisiones irregulares basadas en otros números primos, como tresillos, quintillos o septillos. De esta manera, es posible detectar irregularidades rítmicas incluso cuando no se encuentran anotadas explícitamente en los metadatos de la partitura.

La detección final se basa en la coexistencia de subdivisiones incompatibles dentro del mismo compás. Se consideran dos escenarios principales. El primero corresponde a la presencia simultánea de dos o más subdivisiones irregulares diferentes, como ocurre en relaciones del tipo 3:5 o 5:7. El segundo se produce cuando una subdivisión irregular coexiste con subdivisiones binarias regulares, generando configuraciones equivalentes a relaciones como 3:2 o 5:4. En ambos casos se interpreta que existe una superposición de estructuras métricas incompatibles y, por tanto, una polirritmia vertical.

Los compases que satisfacen alguno de estos criterios se registran como polirritmias y se incorporan al conjunto de datos. Finalmente se devuelve una lista ordenada de números de compás donde se han detectado estas polirritmias.

5.2.16. Extracción de la tesitura de la obra

Con el objetivo de identificar el registro de la composición, se implementó un sistema destinado a determinar la tesitura de una partitura. En este caso la tesitura se define como el rango comprendido entre la altura más grave y la altura más aguda registradas en la obra.

El método utilizado para esta extracción de tesitura es `_calcular_tesitura`.

Este método recibe como entrada una partitura representada mediante la estructura `Score` de `Music21` y realiza un recorrido recursivo sobre todos los eventos musicales que contienen información de altura. El análisis considera tanto notas individuales como acordes, garantizando que todas las alturas presentes en la obra participen en el cálculo de los límites de la tesitura.

Cuando el evento corresponde a una nota aislada, su altura se incorpora directamente al conjunto de datos analizado. En el caso de los acordes, el sistema extrae todas las alturas que forman parte del evento armónico y las incorpora individualmente al conjunto de observaciones.

Una vez recopiladas todas las alturas de la obra, el algoritmo identifica los valores extremos utilizando la representación numérica `Pitch Space` (`ps`). La nota más grave se

determina como aquella que presenta el valor mínimo de Pitch Space, mientras que la nota más aguda corresponde al valor máximo observado.

El resultado final consiste en un par de valores que describen los límites inferior y superior del registro utilizado por la composición.

En aquellos casos en los que la partitura no contiene eventos con altura definida, el procedimiento devuelve valores nulos simbólicos que indican la imposibilidad de calcular la tesitura.

5.2.17. Inferencia del género del autor a partir del nombre

Con el objetivo de incorporar información descriptiva adicional sobre los autores presentes en el corpus musical, se implementó un sistema muy básico de inferencia del género asociado al nombre del autor.

El método utilizado para esta inferencia es `_clasificar_genero_autor`.

Este método recibe como entrada una cadena de texto correspondiente al nombre del autor y genera una clasificación en una de tres categorías: masculino, femenino o desconocido. Antes de realizar el análisis, se lleva a cabo una fase de validación destinada a identificar casos en los que la información disponible no permite efectuar una inferencia razonable. En particular, los autores anónimos, tradicionales o aquellos cuya identificación no está especificada son clasificados directamente como desconocidos.

Posteriormente, el algoritmo extrae el primer componente nominal del nombre completo. Sobre este lexema se aplican una serie de reglas heurísticas basadas en terminaciones frecuentemente asociadas a nombres propios masculinos y femeninos en lengua española.

La clasificación femenina se asigna cuando el nombre presenta determinadas terminaciones tradicionalmente vinculadas a nombres de mujer. Con el fin de reducir errores sistemáticos, se incorporan excepciones explícitas para algunos nombres cuya terminación podría inducir clasificaciones incorrectas. De manera análoga, la clasificación masculina se determina a partir de un conjunto de terminaciones habitualmente asociadas a nombres de hombre.

Cuando ninguna de las reglas definidas proporciona evidencia suficiente para una asignación clara, el sistema devuelve la categoría desconocido.

5.2.18. Análisis automático de la perspectiva lírica

Con el objetivo de caracterizar la voz narrativa presente en el texto de una canción, se implementó un sistema de clasificación semántica orientado a identificar la perspectiva lírica expresada en la letra. Esta metodología analiza las marcas lingüísticas presentes en el discurso con el fin de determinar si la voz que interpreta la obra presenta rasgos predominantemente masculinos, femeninos, neutros o insuficientes para realizar una clasificación fiable.

El método utilizado para este análisis es `_analizar_perspectiva_lirica`.

Este método recibe como entrada el texto completo de la letra y realiza una validación inicial para comprobar que la cantidad de contenido disponible sea suficiente para el análisis. Los textos excesivamente breves son clasificados automáticamente como indeterminados debido a la ausencia de evidencia lingüística suficiente.

Para determinar la voz del narrador se usa un modelo de lenguaje local, construyendo una instrucción específica que solicita al modelo la identificación de la voz lírica a partir de indicadores morfosintácticos y semánticos presentes en el texto. Entre estos indicadores destacan las marcas de género gramatical asociadas a adjetivos, participios y otras cons-

trucciones lingüísticas que permiten inferir la identidad del sujeto. El modelo recibe la letra completa como contexto y devuelve una clasificación restringida a una de cuatro categorías predefinidas: masculino, femenino, neutro o desconocido.

La salida generada por el modelo se valida posteriormente mediante un proceso de verificación sintáctica. Únicamente se aceptan respuestas que respetan el formato estructurado especificado y que pertenecen al conjunto cerrado de categorías permitidas.

Con el fin de garantizar la operatividad del sistema se incorpora un mecanismo de respaldo muy básico basado en reglas lingüísticas explícitas. Este procedimiento alternativo se activa cuando el modelo de lenguaje no está disponible o cuando se produce algún error durante la comunicación con el servicio local. En esta modalidad, el análisis se fundamenta en la búsqueda de palabras clave asociadas a formas masculinas y femeninas frecuentes en español. El algoritmo contabiliza la presencia de términos representativos de cada categoría y asigna la clasificación correspondiente en función de la frecuencia observada. Cuando ninguna categoría presenta predominio claro, la voz lírica se clasifica como neutra.

El resultado final consiste en una etiqueta categórica que resume la perspectiva narrativa inferida a partir de la letra.

5.2.19. Extracción de notas y silencios

Con el propósito de obtener una representación textual compacta de la estructura musical de una partitura, se implementó un sistema de extracción secuencial de eventos sonoros y silencios. El método transforma la información simbólica contenida en la partitura en una cadena de texto lineal que preserva el orden cronológico de aparición de los eventos musicales, facilitando su inspección visual, almacenamiento y análisis.

El método utilizado para esta extracción es `_extract_notes_and_rests`.

Este método recibe como entrada una partitura representada mediante la estructura Score de Music21 y realiza un recorrido cronológico sobre todos los eventos musicales relevantes. Para garantizar una secuencia temporal única, la estructura jerárquica de la obra se aplanan previamente mediante una transformación que integra en un único flujo los eventos procedentes de diferentes voces, pentagramas o niveles.

Durante el recorrido se consideran tres tipos fundamentales de eventos: notas individuales, silencios y acordes. Otros elementos de notación, como claves, armaduras de clave, indicaciones métricas o anotaciones editoriales, son excluidos deliberadamente del análisis por no formar parte de la secuencia sonora.

Cuando el evento corresponde a una nota individual, el procedimiento extrae su nombre en notación científica, incluyendo la clase de altura y la octava correspondiente. Con el fin de mejorar la legibilidad de la representación textual, los bemoles se expresan mediante la notación convencional basada en la letra “b”, facilitando y normalizando así la representación de las alteraciones.

Los silencios se codifican mediante una etiqueta textual explícita que indica la ausencia temporal de sonido. Esta decisión permite conservar la estructura temporal básica de la partitura y distinguir claramente entre eventos sonoros y pausas musicales.

En el caso de los acordes, el procedimiento conserva la información de simultaneidad agrupando todas las alturas que componen el evento armónico. Las notas se representan de manera conjunta y se conectan mediante un separador. Esta codificación permite diferenciar los eventos polifónicos de las sucesiones melódicas ordinarias sin perder la información relativa a las alturas que participan en el evento.

Una vez procesados todos los eventos, los elementos obtenidos se concatenan siguiendo el orden temporal original de la obra para generar una única cadena textual. El resulta-

do constituye una representación simbólica simplificada de la partitura que mantiene la secuencia de notas, silencios y acordes, preservando las relaciones cronológicas entre ellos.

5.2.20. Extracción de la secuencia melódica de la obra

Con el objetivo de obtener una representación lineal de la melodía principal de una partitura, se implementó un sistema de extracción cronológica de alturas musicales basado en la secuencia de eventos sonoros presentes en la obra.

El método utilizado para esta extracción es `_extract_melody_pitches`.

El método recibe como entrada una partitura representada mediante la estructura `Score` de `Music21` y genera una secuencia ordenada de alturas (pitches). Para garantizar la correcta preservación del orden temporal de los eventos, la estructura jerárquica de la partitura se transforma previamente en una representación lineal mediante una operación de aplanamiento (flattening). Esta transformación permite integrar en una única secuencia los eventos procedentes de diferentes voces, pentagramas o niveles estructurales, manteniendo su disposición cronológica dentro de la obra.

A continuación, el algoritmo recorre todos los eventos musicales identificados como notas o acordes. Cuando el evento corresponde a una nota individual, su altura se incorpora directamente a la secuencia melódica resultante. En cambio, cuando el evento corresponde a un acorde, se selecciona la nota más aguda de este, asociada normalmente a la melodía.

El resultado final es una secuencia cronológicamente ordenada de alturas musicales que representa una estimación de la línea melódica principal de la obra.

5.2.21. Análisis de la dirección de los intervalos melódicos

Con el objetivo de examinar el comportamiento direccional de una melodía, se implementó un procedimiento destinado a analizar la orientación de los intervalos entre alturas consecutivas. Este enfoque permite describir el perfil general del movimiento melódico mediante la cuantificación de ascensos, descensos y repeticiones de altura, independientemente de la magnitud exacta de los intervalos involucrados.

El método utilizado para este análisis es `_analyze_interval_directions`.

Este método recibe como entrada una secuencia ordenada de alturas musicales (pitches). A partir de esta secuencia se examinan todos los pares de notas consecutivas presentes en la línea melódica. Cuando la secuencia contiene menos de dos eventos, el procedimiento concluye inmediatamente, ya que no existen intervalos a analizar.

Para cada par de alturas consecutivas, la comparación se realiza utilizando la representación numérica `Pitch Space (ps)`, que asigna un valor continuo a cada altura musical. Esta representación permite efectuar comparaciones directas entre notas.

La dirección de cada intervalo se determina comparando el valor de la segunda nota con el de la primera. Cuando la altura posterior es superior, el intervalo se clasifica como ascendente. Cuando la altura posterior es inferior, el intervalo se clasifica como descendente. Finalmente, cuando ambas alturas poseen el mismo valor, el evento se registra como una repetición de nota.

A lo largo del recorrido, el sistema mantiene contadores independientes para cada una de las tres categorías. El resultado final consiste en un conjunto de frecuencias que resume la distribución de movimientos melódicos observados en la obra.

5.2.22. Clasificación de la tendencia melódica predominante

Con el fin de caracterizar el comportamiento global del movimiento melódico, se implementó un sistema de clasificación basado en la distribución relativa de intervalos ascendentes y descendentes. Este método permite resumir el perfil direccional de una obra mediante una única etiqueta descriptiva acompañada de indicadores cuantitativos.

El método utilizado para esta clasificación es `_determine_predominant_trend`.

El procedimiento recibe como entrada los conteos previamente obtenidos para las tres categorías fundamentales de movimiento melódico: intervalos ascendentes, intervalos descendentes y repeticiones de altura. A diferencia de enfoques que consideran todas las transiciones por igual, el método se centra exclusivamente en los movimientos que implican un cambio efectivo de altura, excluyendo las repeticiones del cálculo.

En primer lugar se calcula el número total de movimientos activos como la suma de los intervalos ascendentes y descendentes. Cuando no existe ningún movimiento de altura, es decir, cuando la obra está compuesta únicamente por repeticiones o carece de material melódico suficiente, el sistema clasifica automáticamente la melodía como estática y asigna valores nulos a los porcentajes de ascenso y descenso.

En caso contrario se calcula la proporción relativa de movimientos ascendentes y descendentes respecto al total de movimientos activos. Una vez obtenidos los porcentajes, el procedimiento aplica un criterio de clasificación basado en un umbral de tolerancia. Se considera que una melodía presenta una distribución equilibrada cuando la diferencia absoluta entre ambos porcentajes no supera el 6 %. Este margen tiene como objetivo evitar clasificaciones excesivamente sensibles a pequeñas fluctuaciones estadísticas que carecen de relevancia musical significativa.

Cuando la diferencia entre ambas proporciones excede dicho umbral, la obra se clasifica según la dirección predominante observada. Si el porcentaje de movimientos ascendentes es superior al de movimientos descendentes, la melodía se considera predominantemente ascendente. En caso contrario, se clasifica como predominantemente descendente.

El resultado final consiste en un conjunto de indicadores compuesto por la categoría descriptiva asignada y los porcentajes correspondientes a cada dirección de movimiento.

5.2.23. Extracción de correspondencias entre sílabas y duraciones musicales

Con el objetivo de analizar la relación entre el texto cantado y su realización rítmica, se implementó un método destinado a extraer las correspondencias entre las sílabas presentes en una partitura vocal y las duraciones musicales asociadas a cada una de ellas. Este enfoque permite identificar patrones de asignación temporal del texto.

El método utilizado para esta extracción es `_extract_syllable_duration_mapping`.

El método recibe como entrada una partitura representada mediante la estructura `Score` de `Music21` y realiza un recorrido lineal sobre todos los eventos melódicos presentes en la obra. El análisis se limita a aquellas notas que contienen información lírica explícita, excluyendo automáticamente los eventos instrumentales o las notas sin texto asociado.

Para cada nota vocal identificada, el algoritmo recupera su duración musical utilizando la representación estándar basada en unidades de negra. Esta representación proporciona una medida normalizada del valor rítmico de cada evento, independientemente de la notación específica empleada en la partitura.

Posteriormente, se examinan todas las etiquetas líricas asociadas a la nota. Antes de su almacenamiento, las cadenas de texto son sometidas a una fase de normalización mediante

una función de limpieza específica, cuyo objetivo es eliminar artefactos de codificación, signos auxiliares o inconsistencias de representación que puedan afectar a los análisis posteriores.

Una vez obtenida la forma normalizada de la sílaba, el sistema registra la duración musical correspondiente dentro de una estructura de agrupación indexada por texto.

Tras recorrer por completo la partitura, los conjuntos de duraciones se transforman en listas ordenadas para facilitar su almacenamiento y consulta. El resultado final consiste en un diccionario donde cada sílaba se asocia con el conjunto de duraciones musicales que adopta a lo largo de la pieza.

Finalmente, la estructura se codifica en formato JSON utilizando una configuración que preserva los caracteres originales. Esta estrategia evita la conversión de caracteres acentuados o específicos de determinadas lenguas a secuencias de escape.

5.2.24. Extracción y conteo de alteraciones accidentales

Con el objetivo de caracterizar el uso de alteraciones accidentales dentro de una partitura, se desarrolló un método de detección y agregación de alteraciones basado en la estructura jerárquica de la representación musical. Esta metodología permite identificar aquellas alteraciones que constituyen modificaciones explícitas en la obra musical, diferenciándolas de las alteraciones implícitas derivadas de la armadura de clave.

Los métodos utilizados para detectar estas anomalías son `_extract_accidental_events` y `_format_accidentals_report`.

- **Extractor de alteraciones accidentales `_extract_accidental_events`.** Recorre la partitura por compases, identificando la armadura recuperándola de la correspondiente estructura `KeySignature`. Esta información se utiliza posteriormente para distinguir entre alteraciones originales de la tonalidad y alteraciones introducidas explícitamente por el compositor o editor. A continuación, se inspeccionan todos los eventos musicales contenidos en cada compás, incluyendo tanto notas individuales como acordes. En el caso de los acordes, el análisis se realiza sobre cada una de las alturas que los componen, garantizando así la detección de alteraciones presentes en estructuras polifónicas. Para cada altura musical se verifica la existencia de una alteración asociada. El procedimiento se centra exclusivamente en sostenidos, bemoles y becuadros, ignorando las alteraciones naturales de acuerdo con los criterios establecidos para el análisis. Una vez identificada una posible alteración, se determina si esta corresponde realmente a una modificación accidental o si forma parte de la armadura de clave vigente. Cuando la alteración observada coincide exactamente con la definida por la tonalidad, el evento se considera parte de la armadura y se excluye del análisis. Por el contrario, cuando existe una discrepancia o no se encuentra una alteración equivalente en la armadura, el evento se clasifica como una alteración accidental explícita y se registra junto con el número de compás en el que aparece. Cada evento almacena la nota alterada, incluyendo su octava y su localización.
- **Contador de alteraciones accidentales `_format_accidentals_report`.** Almacena inicialmente los compases por conjuntos para que la representación resultante refleje únicamente la presencia de la alteración en cada compás independientemente de cuántas veces aparezca dentro de este. Posteriormente, los conjuntos de compases se convierten en listas ordenadas para garantizar la compatibilidad con formatos como JSON. Esto produce una estructura indexada en la que cada nota alterada se asocia explícitamente con los compases donde aparece, facilitando consultas poste-

riores y análisis automatizado. Además de la representación estructurada, el procedimiento genera un resumen textual orientado a la visualización inmediata de los resultados. Para cada nota alterada se construye una expresión legible que incluye la identificación de la nota y la relación de compases correspondientes. Cuando la alteración aparece en un único compás se utiliza una referencia singular, mientras que las apariciones múltiples se representan mediante una enumeración ordenada de compases. El resultado final consiste en una cadena de texto que sintetiza toda la información detectada. Finalmente, el método devuelve tres elementos complementarios: una representación JSON de las alteraciones agrupadas por nota, adecuada para almacenamiento en la base de datos; un resumen textual diseñado para la inspección directa; y un indicador que indica el número total de eventos detectados durante el análisis.

5.2.25. Extracción de metadatos de transcripción y procedencia digital

Con el objetivo de reconstruir información relativa al proceso de codificación y transformación de las partituras digitales, se desarrolló un procedimiento de extracción automática de metadatos de transcripción. Este método permite identificar el software empleado durante la generación o conversión del documento, recuperar información temporal asociada a su codificación y determinar el formato de origen del archivo analizado.

El método utilizado para esta extracción de metadatos es `_extract_transcription_metadata`.

El procedimiento recibe como entrada la ruta de un archivo musical en formato XML, MusicXML o MEI y realiza una inspección directa de su contenido textual. A diferencia de otros métodos centrados en la estructura musical, este análisis se orienta exclusivamente a los metadatos de procedencia incorporados por los programas de edición, conversión o publicación digital.

En los documentos codificados en formato MEI, el procedimiento examina la sección de descripción de la codificación, buscando elementos asociados a aplicaciones de software responsables de la creación o transformación del archivo. Cuando esta información está disponible, se registra el nombre de la herramienta empleada. También se inspeccionan los elementos relacionados con fechas de creación o codificación para recuperar información cronológica relevante.

En los archivos MusicXML, el método analiza los metadatos de codificación definidos por el estándar, prestando especial atención a los elementos que describen el software utilizado y la fecha de generación del documento. Estos campos suelen ser incorporados automáticamente por los programas de edición musical y constituyen una fuente útil para la reconstrucción del historial de procesamiento del archivo.

Ya que los archivos MSCZ han sido generados específicamente por el programa MuseScore, su procedencia es trivial, por lo que este tipo de documento no se procesa explícitamente.

Además del análisis estructurado de metadatos, se realiza una búsqueda textual global dentro del documento con el fin de detectar referencias explícitas a herramientas de conversión. Se presta especial atención a la identificación de Verovio, una plataforma ampliamente utilizada para la conversión y visualización de documentos musicales digitales, ya que, en una de las preguntas de test se pregunta específicamente por este estándar. La detección puede producirse tanto mediante metadatos específicos como a través de menciones textuales presentes en comentarios, cabeceras o información auxiliar del archivo.

Este método genera un conjunto normalizado de metadatos que incluye el software de

codificación identificado, un indicador binario de conversión mediante Verovio, la fecha de codificación cuando está disponible y el formato de origen del documento. En ausencia de información explícita, se asignan valores por defecto que garantizan la consistencia de la salida y permiten continuar el procesamiento sin interrupciones.

5.2.26. Auditoría automatizada de control de calidad estructural

Con el fin de evaluar la integridad estructural de las partituras digitales y detectar posibles errores derivados de procesos de transcripción, digitalización o conversión entre formatos, se implementó un procedimiento automatizado de control de calidad. Este procedimiento genera un conjunto de indicadores para medir la consistencia básica de la representación musical y a señalar posibles anomalías que puedan comprometer análisis posteriores.

El método utilizado para evaluar estructuralmente la partitura es `_extract_quality_control_metadata`.

Este método recibe como entrada una partitura representada mediante la estructura Score de Music21 y realiza una inspección sistemática de todos los compases presentes en la obra. Durante este proceso se analizan tanto los eventos musicales (notas y silencios) como determinadas propiedades temporales asociadas a cada nota.

La primera comprobación se centra en la identificación de compases compuestos exclusivamente por silencios, es decir, compases vacíos. Para ello, se recopilan todos los eventos contenidos en cada compás y se verifica si la totalidad de ellos corresponde a silencios. Aunque este tipo de compases puede ser musicalmente válido en determinados contextos, en este caso concreto de análisis de obras folclóricas, una frecuencia elevada de ocurrencias puede indicar problemas de importación, pérdidas de información durante la conversión o fragmentos incompletos de la partitura original. El número total de compases detectados bajo esta condición se registra como indicador de calidad.

La segunda comprobación busca notas con duración nula o negativa. Este tipo de evento constituye una inconsistencia estructural, ya que toda nota debe poseer una duración estrictamente positiva para poder ser interpretada correctamente por los sistemas de notación y reproducción. La presencia de duraciones nulas suele asociarse a errores de codificación, corrupciones del archivo o fallos en los procesos de transformación de datos musicales.

A partir de los resultados obtenidos, el procedimiento calcula una puntuación global de integridad inicializada en un valor máximo de 100 puntos. Las anomalías detectadas generan penalizaciones acumulativas que dependen de la naturaleza y frecuencia de los problemas encontrados. Los compases vacíos producen una reducción progresiva de la puntuación hasta un límite máximo predefinido, mientras que las notas con duración nula generan una penalización independiente. Este mecanismo permite sintetizar múltiples indicadores de calidad en una única métrica.

Finalmente, la puntuación resultante se utiliza para asignar una clasificación cualitativa del estado de la partitura. Los documentos con puntuaciones elevadas se consideran estructuralmente consistentes, mientras que puntuaciones intermedias generan advertencias preventivas sobre posibles desajustes. Cuando la puntuación desciende por debajo de un umbral crítico, el sistema recomienda una revisión manual debido a la posible existencia de errores graves o información incompleta.

5.2.27. Extracción de intervalos melódicos

Con el objetivo de obtener una representación de la estructura melódica de una partitura, se implementó un procedimiento de extracción de intervalos melódicos basado en las distancias en semitonos entre eventos musicales consecutivos. Este enfoque permite describir el movimiento de la melodía independientemente de su altura absoluta, facilitando la comparación entre fragmentos musicales transpuestos a diferentes tonalidades.

El método utilizado para identificar el volumen MIDI de la obra es `_extract_melodic_intervals`.

El algoritmo recibe como entrada una partitura y genera una secuencia ordenada de alturas expresadas en valores MIDI. Para ello, la partitura se transforma previamente en una representación lineal mediante el aplanamiento de su estructura jerárquica, preservando el orden temporal de los eventos musicales y omitiendo los silencios. De este modo, únicamente se consideran notas individuales y acordes presentes en la secuencia musical.

Cuando el evento corresponde a una nota aislada, se extrae directamente su altura en formato MIDI. En el caso de los acordes, se selecciona la nota más aguda como representante del evento. Esta decisión se fundamenta en que, en gran parte de la música tonal occidental, la voz superior suele desempeñar la función melódica principal, permitiendo aproximar el contorno melódico incluso en presencia de textura armónica.

Una vez obtenida la secuencia de alturas, se calculan los intervalos melódicos entre pares consecutivos de eventos. Los valores positivos indican movimientos ascendentes, los valores negativos movimientos descendentes y el valor cero la repetición de una misma altura. El resultado final consiste en una secuencia ordenada de intervalos que describe el perfil melódico de la obra sin depender de la tonalidad, la tesitura o el registro específicos.

5.2.28. Detección y conteo de leitmotivs recurrentes

Con el objetivo de identificar patrones melódicos recurrentes dentro de una partitura, se implementó un procedimiento basado en el análisis de n-gramas sobre secuencias de intervalos melódicos. Este enfoque permite detectar fragmentos musicales que reaparecen a lo largo de una obra independientemente de su altura absoluta, favoreciendo la identificación de motivos característicos y posibles estructuras temáticas recurrentes.

El método utilizado para identificar el volumen MIDI de la obra es `_detect_leitmotivs_ngrams`.

Este método toma como punto de partida la secuencia de intervalos melódicos obtenida a partir de la sucesión lineal de eventos musicales. Dado que los intervalos representan relaciones entre alturas consecutivas y no alturas absolutas, la representación resultante es invariante frente a transposiciones, lo que facilita la detección de patrones equivalentes en diferentes contextos.

A continuación, se aplica una estrategia de ventana deslizante sobre la secuencia de intervalos para generar todos los n-gramas posibles de longitud fija. El tamaño de cada n-grama se determina a partir del número de notas que se desea considerar en el motivo.

Para cada longitud considerada, se generan todos los patrones interválicos presentes en la obra y se calcula su frecuencia de aparición mediante un procedimiento de conteo. Posteriormente se seleccionan únicamente aquellos patrones cuya frecuencia alcanza o supera un umbral predefinido. En la configuración empleada por defecto, un motivo debe aparecer al menos tres veces para ser considerado recurrente.

Con el fin de reducir la detección de patrones carentes de interés musical, el procedimiento excluye secuencias compuestas exclusivamente por intervalos nulos. Este filtrado

evita que repeticiones mecánicas de una misma nota sean clasificadas como motivos relevantes.

Los patrones identificados se representan mediante una notación textual basada en la sucesión de intervalos expresados en semitonos. Por ejemplo, una secuencia de intervalos ([+4, -2, +1]) indica un ascenso de cuatro semitonos seguido de un descenso de dos semitonos y un posterior ascenso de un semitono. Cada patrón se almacena junto con su frecuencia de aparición dentro de la obra.

Finalmente, los resultados se devuelven en formato JSON, permitiendo su almacenamiento directo en la base de datos. La estructura generada contiene los motivos detectados y el número de veces que cada uno aparece en la partitura.

5.2.29. Detección de plicas anómalas

Con el propósito de evaluar la coherencia gráfica de la notación musical, se implementó un sistema de detección automática de anomalías en la dirección de las plicas de las notas. El método se basa en las convenciones tradicionales de escritura musical, según las cuales las notas situadas por debajo de la tercera línea del pentagrama suelen representarse con la plica orientada hacia arriba, mientras que las notas situadas por encima de dicha línea se representan habitualmente con la plica orientada hacia abajo. Las desviaciones respecto a esta convención se consideran potenciales anomalías de notación.

Los métodos utilizados para detectar estas anomalías son `_detect_stem_direction_anomalies` y `_get_middle_line_midi`.

El algoritmo recibe como entrada una partitura representada mediante la estructura Score de Music21 y recorre de forma exhaustiva todas las notas individuales presentes en la obra. Para cada nota se recupera la dirección de la plica almacenada en el documento original. Únicamente se consideran aquellas notas cuya dirección ha sido especificada explícitamente (up o down), excluyéndose las plicas automáticas o ausentes, ya que en estos casos no es posible determinar una decisión editorial concreta.

Posteriormente, el método identifica la clave activa en el contexto exacto de cada nota con el fin de determinar la posición correspondiente a la tercera línea del pentagrama. Esta referencia se transforma a una representación numérica basada en valores MIDI, permitiendo comparar de forma homogénea la altura de la nota con la posición central del pentagrama independientemente de la clave utilizada.

Una vez establecida esta referencia, se verifica la correspondencia entre la altura de la nota y la orientación de su plica. Se registra una anomalía cuando una nota situada por debajo de la tercera línea presenta una plica orientada hacia abajo o cuando una nota situada por encima de la tercera línea presenta una plica orientada hacia arriba. Cada ocurrencia detectada incrementa un contador global y queda asociada al compás correspondiente para facilitar su posterior localización.

Como resultado, este procedimiento genera un conjunto de indicadores descriptivos que incluyen la presencia o ausencia de anomalías, el número total de casos detectados y la identificación de los compases en los que se producen.

5.2.30. Extracción del volumen MIDI de la obra

Con el fin de obtener información sobre la dinámica de reproducción de las partituras analizadas, se desarrolló un procedimiento de extracción automática del volumen MIDI a partir de distintos formatos de representación musical digital. Ya que los tres formatos procesados en este corpus (MEI, XML y MSCZ) representan de distinta manera la información

relativa a la partitura, el método diseñado se adapta a cada tipo de documento, proporcionando una interfaz unificada para la recuperación de este parámetro independientemente de la estructura interna de la obra.

El método utilizado para identificar el volumen MIDI de la obra es `_extract_midi_volume`.

El algoritmo recibe como entrada la ruta del archivo a analizar y verifica inicialmente que su extensión se corresponda con alguno de los formatos soportados. En el caso de archivos MSCZ, el procedimiento accede al contenedor comprimido y localiza el archivo de configuración de audio (`audiosettings.json`). A continuación, analiza su contenido en formato JSON y extrae el valor asociado al atributo `volumeDb` dentro de la configuración maestra de audio. Este valor representa el nivel global de volumen definido para la reproducción de la partitura.

Para los formatos basados en XML (MEI y MusicXML), el documento se procesa mediante un analizador sintáctico XML y se recorre de forma exhaustiva la estructura jerárquica de elementos. En los documentos MEI, el volumen MIDI se identifica a través del atributo `midi.volume`, presente en elementos `instrDef`, mientras que en MusicXML se localiza en elementos `volume` contenidos dentro de estructuras `midi-instrument`. Una vez recuperado el valor, se eliminan posibles caracteres de formato (como símbolos de porcentaje) y se realiza una conversión a representación numérica. Finalmente, el valor se redondea y almacena como un entero para garantizar una representación uniforme entre formatos.

5.2.31. Extracción de sustantivos y nombres propios (personas, lugares...) de la lírica de la obra

Con el objetivo de enriquecer el análisis temático de las obras, se ha implementado una herramienta que recorre la letra de cada canción para identificar y extraer los sustantivos y nombres propios presentes en ella. Esta información puede ser crucial para determinar los temas tratados en la obra, así como para realizar análisis comparativos entre distintas canciones, autores o regiones.

El método utilizado para hacer esta identificación de etiquetas es `_extraer_sustantivos_y_propios`.

Esta implementación se diseñó con el objetivo de identificar y extraer automáticamente los sustantivos comunes y los nombres propios presentes en las líricas de las canciones en español del corpus. Para ello se apoya en técnicas de procesamiento del lenguaje natural implementadas mediante la biblioteca `spaCy` y su modelo lingüístico para español.

En una primera fase, el texto es sometido a un análisis morfosintáctico que permite obtener información gramatical de cada token. Los términos clasificados como sustantivos comunes (NOUN) son normalizados mediante lematización y convertidos a minúsculas con el fin de unificar diferentes variantes flexivas bajo una misma representación léxica. Esta normalización reduce la dispersión terminológica y facilita posteriores tareas de análisis cuantitativo o comparación semántica.

Paralelamente, los elementos identificados como nombres propios (PROPN) son extraídos conservando su forma original. Esta decisión metodológica permite mantener la integridad de la información nominal y preservar las mayúsculas características de entidades específicas nombradas.

Con el objetivo de mejorar la cobertura de la detección de entidades, se incorpora una segunda etapa basada en reconocimiento de entidades nombradas (Named Entity Recognition). En esta fase se identifican entidades correspondientes a personas, ubicaciones y

organizaciones, incluyendo expresiones compuestas formadas por múltiples palabras. Este mecanismo complementa la clasificación gramatical tradicional y permite capturar entidades complejas.

Los elementos extraídos se almacenan eliminando automáticamente posibles duplicados. Posteriormente, los resultados son ordenados y agregados en dos categorías diferenciadas: sustantivos comunes y nombres propios. La salida final se presenta en forma de diccionario estructurado que facilita su integración en etapas posteriores del flujo de procesamiento.

Adicionalmente, el procedimiento incorpora un mecanismo de respaldo para entornos en los que el modelo lingüístico no se encuentre disponible. En tales situaciones, la identificación de nombres propios se realiza mediante una heurística basada en patrones ortográficos (detección de palabras que comienzan por letra mayúscula). Aunque este enfoque ofrece una aproximación básica para la identificación de entidades nominales, no proporciona la precisión necesaria para la extracción fiable de sustantivos comunes, por lo que dicha funcionalidad queda supeditada a la disponibilidad del modelo de PLN.

5.2.32. Identificación de saltos melódicos entre barras de compás (intervalos frontera)

Otra característica útil a identificar recoge los intervalos entre compases, lo que puede aportar al análisis general de la obra información relativa a los fraseos y las estructuras melódicas de la obra. Esto también puede ayudar a identificar estructuras melódicas características de ciertas culturas y regiones, permitiendo así realizar un análisis comparativo entre distintas músicas tradicionales. Además, para responder a una de las preguntas de test orientadas a probar el sistema, con esta implementación también se pretende identificar cuál es intervalo frontera más frecuente.

Con esta implementación no se pretende sólo identificar el salto melódico entre compases, sino también poder identificar las ligaduras que unen notas pertenecientes a distintos compases. En este caso, no existe un salto melódico ni un nuevo ataque en el compás posterior, sino una extensión rítmica. Esta herramienta filtra también esos casos.

Los métodos utilizados para hacer esta identificación de intervalos son `_extract_measure_boundary_intervals` y `_calculate_most_frequent_boundary_interval`.

- **Extractor de intervalos frontera** `_extract_measure_boundary_intervals`. Recorre secuencialmente la obra y extrae los intervalos melódicos que conectan el último evento sonoro de un compás con el primer evento sonoro del compás siguiente. Para evitar ambigüedades derivadas de la escritura polifónica, el análisis se realiza sobre las partes o voces disponibles de manera independiente. En cada cambio de compás sólo se analizan las notas y acordes presentes; los silencios se excluyen. Con el fin de identificar únicamente movimientos melódicos reales, la función ignora aquellos casos en los que la primera nota del compás siguiente constituye la continuación de una ligadura de prolongación. En tales situaciones no existe un nuevo ataque sonoro y, por tanto, no puede considerarse que se produzca un nuevo intervalo melódico. Cuando la frontera es válida para el análisis, se calcula la distancia en semitonos entre la última nota del compás precedente y la primera del compás siguiente. En el caso de los acordes, se toma como referencia la nota más aguda de cada uno. Los intervalos obtenidos se almacenan como valores enteros con signo, donde los valores positivos indican movimiento ascendente, los negativos movimiento descendente y el valor cero la repetición de la misma altura.

- **Contador de intervalos frontera `_calculate_most_frequent_boundary_interval`.**
Recibe la colección de intervalos obtenidos y determina cuál de ellos aparece con mayor frecuencia a lo largo de la obra. Para facilitar la interpretación de los resultados, los intervalos se representan mediante una notación con signo explícito (por ejemplo, «+2», «-5» o «0»). Posteriormente se contabiliza la frecuencia de aparición de cada intervalo y se identifica el más común junto con el número de veces que ocurre.

5.2.33. Identificación del uso de la etiqueta `<sb />`(system break) en archivos MEI

Una de las preguntas orientadas a probar el sistema de búsqueda basándose en la estructura interna del archivo de la obra buscaba identificar si, en algunos casos, la aparición de etiqueta `<sb />`(system break) en archivos MEI coincide con el final de una frase musical lógica. Para comprobar que esta condición se cumple, se verifica la puntuación de la última sílaba cantada antes del salto.

Aunque `music21` es excelente para extraer alturas e intervalos, a menudo abstrae o separa los elementos de diseño visual (como `<sb />` o saltos de sistema) en capas independientes, perdiendo su posición secuencial exacta respecto a las etiquetas de compás (`<measure>`) del archivo original. Por ello, y dado que la condición específica que se busca sólo aparece en archivos con formato MEI, la solución ideal es diseñar una herramienta que parsee directamente el archivo como un árbol XML mediante `xml.etree.ElementTree`.

El método utilizado para hacer esta identificación de etiquetas es `_analizar_frases_sb_mei`.

Tras procesar inicialmente el archivo XML, se localizan todas las secciones musicales (`<section>`) y se analizan secuencialmente sus elementos en busca de etiquetas `<sb />`. Por cada salto de sistema identificado, la función localiza el compás (`<measure>`) inmediatamente anterior y extrae las sílabas líricas (`<syl>`) contenidas en él. A continuación, examina la última sílaba del compás y comprueba si finaliza con algún signo de puntuación considerado delimitador de frase (punto, coma, punto y coma, signo de exclamación, signo de interrogación o puntos suspensivos). Cuando esta condición se cumple, el salto de sistema se considera coincidente con el final de una frase lógica.

El procedimiento contabiliza tanto el número total de etiquetas `<sb />` presentes en el documento como el número de aquellas que cumplen el criterio anterior. Finalmente, devuelve un resumen indicando si el archivo contiene saltos de sistema, cuántos se han localizado, cuántos coinciden con finales de frase y si la totalidad de ellos cumple esta correspondencia.

La función incluye además mecanismos de validación para asegurar que el archivo analizado posee extensión `.mei` y para gestionar posibles errores durante la lectura o el procesamiento del XML. En ausencia de etiquetas `<sb />`, informa explícitamente de esta circunstancia en el resultado devuelto.

5.3. Ventajas que ofrece la metodología escogida para la herramienta desarrollada

El sistema diseñado e implementado se basa en un proceso de extracción previa de metadatos musicales y textuales a partir de los archivos originales que contienen las partituras y otros detalles sobre las obras, seguido de su almacenamiento estructurado en una base

de datos relacional. Este enfoque presenta diversas ventajas frente a sistemas que delegan todo el análisis directamente en modelos de lenguaje durante el momento de la consulta.

- **Escalabilidad del corpus.** Ya que la complejidad computacional sólo se aprecia en la fase de preprocesamiento, al analizar de manera lineal todas las obras, a la hora de formular las preguntas, el sistema se limita a hacer consultas concretas a la base de datos SQLite, ahorrando así tiempo y recursos. Esto permite que el coste computacional de las consultas permanezca prácticamente constante incluso cuando el tamaño del corpus aumenta considerablemente, favoreciendo la incorporación de nuevas obras sin afectar significativamente al rendimiento del sistema.
- **Precisión en las respuestas.** Al recibir directamente las respuestas desde una base de datos llena de información fiable extraída mediante algoritmos específicamente diseñados basándose en estándares de teoría musical, resulta menos probable que el modelo invente datos sobre las obras, ya que todo lo que debe responder está incorporado en la salida de la consulta. Este proceso permite reducir el fenómeno conocido como "alucinación".
- **Rapidez y ligereza en el almacenamiento.** El contexto proporcionado al modelo pasa de estar compuesto por miles de archivos musicales que requieren demasiado espacio de almacenamiento a ocupar unos pocos bytes de JSON estructurado. Además de ser un beneficio para el almacenamiento, también lo es para la velocidad de respuesta.
- **Interpretabilidad de los resultados.** Cada dato almacenado en la base de datos puede asociarse directamente con un procedimiento analítico concreto. Esto permite conocer exactamente qué algoritmo produjo cada metadato, facilitando la trazabilidad de los resultados y la comprensión de las respuestas generadas por el sistema. A diferencia de los enfoques puramente neuronales, las conclusiones obtenidas pueden justificarse mediante reglas y criterios explícitos.
- **Reproducibilidad científica.** Los metadatos se generan mediante algoritmos deterministas que producen los mismos resultados cuando se aplican sobre las mismas partituras. Esto favorece la reproducibilidad de los experimentos y permite verificar independientemente los resultados obtenidos.
- **Independencia respecto al modelo de lenguaje.** La mayor parte del conocimiento musical utilizado por la herramienta no reside en el modelo conversacional, sino en la base de datos construida durante el preprocesamiento. Por lo tanto, el sistema puede adaptarse fácilmente a diferentes modelos de lenguaje sin necesidad de repetir el análisis musical completo, garantizando una mayor flexibilidad y facilitando futuras actualizaciones.
- **Capacidad para consultas musicológicas complejas.** La estructuración previa de los metadatos permite realizar búsquedas especializadas sobre características musicales concretas, como tonalidades, modos, compases, patrones rítmicos, síncopas, polirritmias, hemiolias o temáticas textuales. Estas consultas resultarían considerablemente más costosas si tuvieran que calcularse desde cero para cada interacción con el usuario.
- **Separación entre análisis y consulta.** La arquitectura distingue claramente entre la fase de extracción de conocimiento y la fase de interacción con el usuario. Esta

separación facilita el mantenimiento del sistema, simplifica la incorporación de nuevos algoritmos de análisis y permite ampliar progresivamente el conjunto de metadatos sin modificar la lógica de consulta ya implementada.

- **Escalabilidad en la extracción de metadatos.** La arquitectura modular empleada permite incorporar nuevos algoritmos de análisis sin necesidad de modificar significativamente el resto del sistema. Cada característica musical se obtiene mediante funciones independientes especializadas en la detección de fenómenos concretos, como aspectos rítmicos, melódicos, armónicos, textuales o estructurales. Esta organización facilita la ampliación progresiva de la base de conocimiento mediante la incorporación de nuevos métodos de extracción de metadatos, permitiendo adaptar la herramienta a futuras necesidades de investigación o a nuevos objetivos analíticos. De este modo, el sistema puede evolucionar para identificar estructuras musicales más complejas o incorporar nuevos criterios musicológicos sin comprometer la compatibilidad con los análisis ya realizados.

5.4. Desarrollo de aplicación web para facilitar el acceso a la herramienta

Con el objetivo de facilitar el acceso a la herramienta técnica por parte de los usuarios, se ha diseñado e implementado una interfaz gráfica de usuario en formato de aplicación web accesible desde cualquier navegador. Aunque la arquitectura intermedia basada en el protocolo MCP (*Model Context Protocol*) y la abstracción relacional en *SQLite* resuelven con éxito la complejidad del procesamiento musicológico, su ejecución nativa a través de interfaces de línea de comandos o scripts aislados introduce una barrera técnica crítica para el perfil de usuario objetivo del proyecto, compuesto mayoritariamente por musicólogas, etnomusicólogos e investigadores de las humanidades.

La aplicación web actúa como una capa extra que encapsula la lógica interna del sistema y proporciona un entorno visual intuitivo, interactivo y guiado.

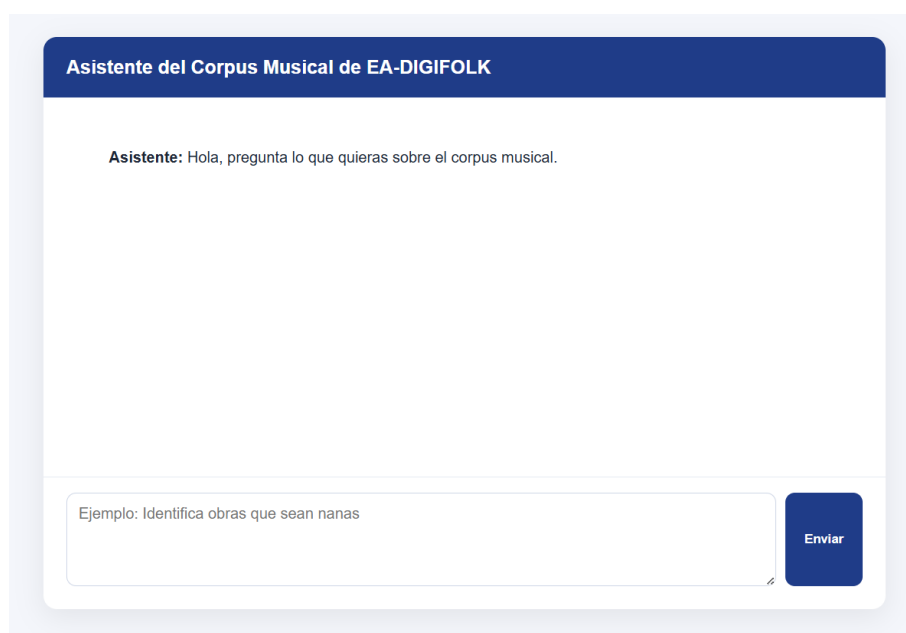


Figura 5.1: Interfaz de la aplicación web desarrollada para interactuar con la herramienta.

Esta interfaz contiene un chat con el que se establece la conexión con el LLM local, permitiendo enviar preguntas desde la barra de búsqueda.

Capítulo 6

Evaluación y pruebas

Como se expuso en el capítulo 3, se elaboró un conjunto de preguntas para evaluar el funcionamiento de los distintos modelos e implementaciones propuestas.

Dicho conjunto cubre distintos niveles de complejidad, desde consultas basadas en metadatos explícitos hasta preguntas que requieren inferencia sobre estructuras musicales, relaciones rítmicas y características analíticas no directamente codificadas de manera explícita en los documentos.

Todas las pruebas se llevaron a cabo mediante *one-shot*, presentando cada consulta de manera aislada al modelo sin historial previo de interacción ni ejemplos adicionales. Cada pregunta se pregunta una única vez a cada modelo, escogiendo la respuesta obtenida como resultado definitivo independientemente de si es correcta o no. La elección de este enfoque responde a la necesidad de evaluar de forma controlada la capacidad intrínseca de los sistemas para recuperar y razonar sobre la información contenida en el corpus, evitando la influencia de estrategias conversacionales, ajustes progresivos del contexto o dependencia de interacciones previas.

En el caso particular del modelo basado en consultas a una base de datos SQLite mediante MCP (Model Context Protocol), la información recuperada se utiliza como mecanismo de RAG para la construcción de la respuesta final. Dado que el corpus utilizado en este caso está compuesto por aproximadamente 12.000 piezas musicales, se observó que la cantidad de resultados devueltos por algunas consultas podía ser excesiva para ser incluida de forma completa en el contexto del modelo. Esta situación resultaba especialmente problemática en el caso del modelo de menor tamaño (8B parámetros), el cual mostraba una tendencia a la degradación de la calidad de las respuestas y a la generación de alucinaciones cuando se le proporcionaban contextos demasiado extensos o ruidosos.

Por este motivo y para este caso en concreto, se estableció una restricción de recuperación mediante LIMIT 10 en las consultas a la base de datos. Esta decisión permite controlar el tamaño del contexto proporcionado al modelo, priorizando los resultados más relevantes según el criterio de ordenación de la consulta, y reduciendo el riesgo de sobrecarga contextual.

6.1. Preguntas realizadas y respuestas de los modelos

6.1.1. Identifica piezas que se llamen El Carro

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** “I found references to El Carro in a few pieces.

One piece contains the lyrics: Échale hierro al carro, échale hierro, échale volanderas de fino acero. and No trastumbes el carro, llévalo siento, mira, galán, que llevas, la dama dentro.

Another piece is titled 53. Déjame subir al carro and includes lyrics such as: Déjame subir al carro, carretero de Zamora, déjame subir al carro, que quiero ver a mi novia.

There is also a mention of El tocar el pan which appears to contain El car as part of the lyrics.

Additionally, one piece mentions Los carboneros which also references Los car.”

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** “* 291. El carro * 283. ¿Dónde vas con el carro? (tonada de muelas, medida)”
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
WHERE LOWER(p.titulo) LIKE '%el carro%';
```

Tras ejecutar la consulta se obtuvieron dos resultados: ‘* 291. El carro’y ‘* 283. ¿Dónde vas con el carro? (tonada de muelas, medida)’.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. Aunque la consulta formulada requiere localizar piezas cuyo título contenga la expresión “El Carro”, el modelo no realiza una búsqueda orientada al campo de título, sino que recupera fragmentos textuales procedentes del contenido de distintas canciones en los que aparece la palabra “carro”. Como consecuencia, la respuesta mezcla referencias a letras, títulos parcialmente relacionados y coincidencias léxicas irrelevantes. Este comportamiento muestra limitación del enfoque basado exclusivamente en recuperación semántica sobre documentos completos: el sistema recupera fragmentos textualmente similares a la consulta, pero carece de mecanismos para imponer restricciones estructuradas sobre campos concretos de los documentos. En este caso, la búsqueda semántica prioriza la presencia del término “carro” en el texto musical frente a la identificación precisa de títulos de obras.

Por el contrario, la respuesta generada por el modelo local llama3.1:8b con acceso a la base de datos SQLite coincide exactamente con el resultado esperado. El modelo identifica correctamente las dos piezas cuyo título contiene la expresión “El carro”: “291. El carro” y “283. ¿Dónde vas con el carro? (tonada de muelas, medida)”. La precisión de la respuesta se debe a que la consulta puede traducirse de forma directa a una operación SQL sobre el campo `titulo`, permitiendo filtrar explícitamente los registros relevantes. Este ejemplo ilustra una de las principales ventajas del enfoque basado en bases de datos estructuradas frente a la recuperación documental

tradicional: cuando la información requerida corresponde a metadatos explícitos almacenados en columnas concretas, las consultas SQL permiten obtener resultados exactos y verificables sin depender de la similitud semántica entre textos.

Por tanto, para esta pregunta se considera que el modelo Gemini 2.5 Flash produce una respuesta incorrecta, mientras que el sistema basado en SQLite proporciona una respuesta completamente correcta.

6.1.2. Identifica piezas cuyo autor sea Schumann

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** Robert Schumann es el autor de las siguientes piezas musicales:

* 54. CANCIÓN DE CUNA, * 57. DÍA DE ASUETO, * 63. EL DÍA DE MAMÁ, * 55. MI LINDA MUÑECA, * 56. VILLANCICO"

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con el autor Schumann:

* 53. LAS ABEJAS, * 55. MI LINDA MUÑECA, * 56. VILLANCICO, * 57. DÍA DE ASUETO, * 63. EL DÍA DE MAMÁ

Estas piezas tienen como autor a Robert Schumann, un compositor alemán del siglo XIX."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.autor
FROM piezas p
WHERE LOWER(p.autor) LIKE '%schumann%';
```

Tras ejecutar la consulta se obtuvieron seis resultados: 53. LAS ABEJAS, 54. CANCIÓN DE CUNA, 55. MI LINDA MUÑECA, 56. VILLANCICO, 57. DÍA DE ASUETO, 63. EL DÍA DE MAMÁ.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es parcialmente correcta. El modelo identifica correctamente cinco de las seis obras cuyo autor es Robert Schumann: "54. CANCIÓN DE CUNA", "55. MI LINDA MUÑECA", "56. VILLANCICO", "57. DÍA DE ASUETO" y "63. EL DÍA DE MAMÁ". Sin embargo, omite la pieza "53. LAS ABEJAS", que también cumple la condición especificada en la consulta. Aunque la respuesta no contiene falsos positivos, la recuperación resulta incompleta. Este comportamiento es coherente con las limitaciones observadas en sistemas RAG basados en búsqueda semántica, donde la recuperación depende de los fragmentos documentales seleccionados por el motor de búsqueda y no existe garantía de exhaustividad sobre todos los documentos relevantes del corpus.

La respuesta generada por el modelo llama3.1:8b con acceso a la base de datos SQLite también es parcialmente correcta. El sistema recupera correctamente las piezas "53. LAS ABEJAS", "55. MI LINDA MUÑECA", "56. VILLANCICO", "57. DÍA DE

ASUETO” y “63. EL DÍA DE MAMÁ”, pero omite “54. CANCIÓN DE CUNA”. Tras revisar los datos almacenados en la base de datos, se observó que esta obra presenta el valor “Schumann.” en el campo de autor, incluyendo un punto final. Si el modelo generó una consulta basada en una coincidencia exacta con el valor “Schumann”, la presencia de este carácter adicional habría impedido recuperar dicho registro. En este caso, el error no parece estar relacionado con la ausencia de información en la base de datos, sino con la generación de una consulta insuficientemente robusta frente a pequeñas variaciones en los datos textuales.

Este resultado pone de manifiesto un aspecto relevante en sistemas basados en bases de datos estructuradas: la calidad y normalización de los metadatos pueden influir significativamente en la recuperación de información. Diferencias aparentemente menores, como espacios, signos de puntuación o variantes ortográficas, pueden provocar la pérdida de resultados cuando se realizan comparaciones exactas. El uso de operadores más flexibles, como LIKE o procedimientos de normalización previos, permite mitigar este tipo de problemas.

6.1.3. Identifica piezas con un compás de 4/4

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Se ha identificado una pieza musical con un compás de 4/4:

* **474**:

Esta obra fue compuesta por Vicente T. Mendoza y forma parte de su Panorama de la música tradicional en México, publicado por la UNAM en 1956. El archivo MusicXML para esta pieza, MX-1956-00-VM-01474.musicxml.xml, indica explícitamente un compás de 4/4."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes obras con compás de 4/4:

1. IE-2019-D-HLS-001, 2. IE-2019-D-HLS-002, 3. IE-2019-D-HLS-003, 4. IE-2019-D-HLS-007, 5. IE-2019-D-HLS-010, 6. IE-2019-D-HLS-011, 7. IE-2019-D-HLS-013, 8. IE-2019-D-HLS-016, 9. IE-2019-D-HLS-017, 10. IE-2019-D-HLS-018"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.compas
FROM piezas p
WHERE p.compas = '4/4';
```

Tras ejecutar la consulta se obtuvieron 1690 obras de compás 4/4.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es correcta en cuanto a la identificación del compás solicitado, ya que la única pieza devuelta efectivamente presenta un compás de 4/4. Sin embargo, la respuesta es claramente incompleta, ya que el corpus contiene un total de 1690 obras con esta característica. En este caso, el sistema no garantiza cobertura completa del espacio de búsqueda, lo que conduce a una recuperación parcial incluso cuando la información está disponible.

Por otro lado, la respuesta generada por el modelo llama3.1:8b con acceso a la base de datos SQLite es correcta y completa en el contexto de la evaluación realizada. El modelo recupera correctamente las 10 primeras obras con compás de 4/4, coherentes con el límite impuesto por la consulta de recuperación (LIMIT 10). Aunque esta limitación impide mostrar el conjunto completo de 1690 resultados, la ejecución de la consulta SQL es correcta y devuelve resultados válidos y representativos del conjunto total. A diferencia del sistema basado en File Search, este enfoque garantiza exhaustividad respecto a la porción de datos consultada, al operar directamente sobre una base de datos estructurada.

En consecuencia, la respuesta de Gemini 2.5 Flash se considera correcta pero incompleta, mientras que la respuesta del modelo basado en SQLite se considera correcta dentro de las restricciones de recuperación establecidas por el sistema.

6.1.4. Identifica piezas con hemiolias horizontales

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Basado en los resultados de la búsqueda, los fragmentos proporcionados son principalmente datos de notación musical en formatos como MEI y MusicXML. Estos formatos describen las notas, duraciones y agrupaciones rítmicas de las piezas. Sin embargo, no contienen una anotación explícita o un análisis textual que identifique directamente la presencia de hemiolias horizontales en ninguna de las obras.

Uno de los documentos (ES-1948-ES-FP-017a.mei) incluye una dirección que indica No hay ejemplos en las Secciones II y III, lo que sugiere que podría ser parte de un estudio o análisis musical, pero no proporciona ejemplos positivos de hemiolias en las secciones consultadas.

Para identificar piezas con hemiolias horizontales, se necesitaría un análisis musical de los fragmentos, lo cual no es posible realizar directamente con la información textual disponible en los resultados de la búsqueda."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con hemiolias horizontales: IE-2019-D-HLS-027, IE-2019-D-HLS-030, A Stor Mo Chroi, The Wild Rover, 575, 580, 583, 585, 608 y Muiñeira de Poio."
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.tiene_hemiolia_horizontal = 1;
```

Tras realizar la consulta, se obtuvieron 220 piezas con hemiolias horizontales.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta en términos de recuperación, ya que el modelo no identifica ninguna pieza con hemiolias horizontales. En su lugar, ofrece una explicación general

sobre la naturaleza de los datos musicales presentes en el corpus y concluye que no es posible determinar la existencia de este rasgo a partir de la información recuperada.

Por otro lado, la respuesta generada por el modelo llama3.1:8b con acceso a la base de datos SQLite es correcta, ya que identifica adecuadamente piezas que contienen hemiolias horizontales según el campo estructurado `tiene_hemiolia_horizontal`. El modelo recupera una lista de obras coherente con la condición de la consulta, incluyendo tanto identificadores numéricos como títulos.

En consecuencia, el sistema basado en File Search falla al no recuperar ninguna instancia relevante del corpus, mientras que el sistema basado en SQLite ofrece una respuesta correcta pero parcial en términos de cobertura, aunque adecuada dentro del contexto de visualización limitada de resultados.

6.1.5. Identifica piezas con hemiolias verticales

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "La búsqueda de archivos proporcionó varias partituras en formato MusicXML y MEI. Sin embargo, estos archivos contienen la notación musical en sí y no incluyen un análisis explícito que identifique la presencia de hemiolias verticales. Para determinar si una pieza contiene hemiolias verticales, sería necesario analizar la estructura rítmica y métrica de cada partitura.

Una hemiolia vertical es un dispositivo rítmico en el que dos o más voces o líneas melódicas interpretan patrones rítmicos que sugieren métricas diferentes simultáneamente. Un ejemplo común se da cuando un compás de 6/8 (que se siente en dos pulsos de tres subdivisiones cada uno) es interpretado por una voz como si estuviera en 3/4 (tres pulsos de dos subdivisiones cada uno), creando una tensión o superposición rítmica."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con hemiolias verticales:

* Drum, * Garryowen, * Go to the devil and shake yourself, * Jackson's Favorite, * Jackson's Morning Brush, * Kennedys's Jig, * King Charle's Jig, * Kissing and drinking, * Kitty of Coleraine, * Paddy O'Carrol

Estas piezas tienen hemiolias verticales en su análisis musical."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.tiene_hemiolia_vertical = 1;
```

Tras ejecutar la consulta, se obtuvieron 86 obras con hemiolias verticales.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta en términos de recuperación, ya que el modelo no devuelve

ninguna pieza concreta con hemiolias verticales. En su lugar, proporciona una explicación general del concepto de hemiolia vertical y describe su interpretación musicológica, sin ofrecer resultados específicos del corpus. Este comportamiento sugiere que el sistema, al no encontrar evidencia explícita del fenómeno en los fragmentos recuperados, opta por una respuesta descriptiva basada en conocimiento general en lugar de realizar una recuperación efectiva de instancias del dataset.

Por otro lado, la respuesta generada por el modelo llama3.1:8b con acceso a la base de datos SQLite es correcta en cuanto a la identificación de piezas con hemiolias verticales, ya que todas las obras listadas cumplen la condición definida por el campo estructurado `tiene_hemiolia_vertical`. Según la consulta de referencia, el conjunto completo está compuesto por 86 obras, por lo que la respuesta representa únicamente una muestra parcial del total.

En consecuencia, el sistema basado en File Search falla al no recuperar instancias del corpus, mientras que el sistema basado en SQLite produce resultados correctos pero limitados en número debido a restricciones en la estrategia de consulta, aunque suficientes para evaluar correctamente la presencia del fenómeno.

6.1.6. Identifica piezas con la tonalidad G major

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Las siguientes piezas se encuentran en la tonalidad de Sol mayor (G major):

* MX-1910-00-EA-00004.musicxml.xml, * MX-1910-00-EA-00019.musicxml.xml, * MX-1910-00-EA-00002.musicxml.xml, * MX-1910-00-EA-00012.musicxml.xml

Estas partituras están escritas para acordeón y cada una tiene una armadura de clave con un sostenido, lo que corresponde a la tonalidad de Sol mayor. "

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con la tonalidad de G mayor:

1. IE-2019-D-HLS-001, 2. IE-2019-D-HLS-002, 3. IE-2019-D-HLS-003, 4. IE-2019-D-HLS-004, 5. IE-2019-D-HLS-005, 6. IE-2019-D-HLS-007, 7. IE-2019-D-HLS-008, 8. IE-2019-D-HLS-009, 9. IE-2019-D-HLS-010

Estas piezas tienen la tonalidad de G mayor."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.tonalidad
FROM piezas p
WHERE LOWER(TRIM(p.tonalidad)) = 'g major';
```

Tras realizar la consulta, se obtuvieron 2722 obras en G Major.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta, ya que sólo dos de las piezas identificadas presentan tonalidad de G major, mientras que el resto pertenecen a otras tonalidades distintas.

Por otro lado, la respuesta generada por el modelo llama3.1:8b con acceso a la base de datos SQLite es correcta en términos de recuperación, ya que todas las piezas listadas cumplen la condición de pertenecer a la tonalidad de G mayor según el campo estructurado `tonalidad`. Sin embargo, al igual que en casos anteriores, la respuesta es necesariamente parcial debido a la limitación impuesta por la estrategia de recuperación (`LIMIT 10`), lo que implica que únicamente se muestran los primeros resultados del conjunto completo de 2722 obras.

En consecuencia, el sistema basado en File Search presenta una recuperación incorrecta, mientras que el sistema basado en SQLite ofrece resultados correctos y consistentes, aunque restringidos en su presentación por limitaciones deliberadas de muestreo.

6.1.7. Identifica piezas que estén en modo dórico

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Las siguientes piezas musicales podrían estar en modo dórico, basándose en sus armaduras de clave:

* **CO-1948-00-BG-00015.musicxml.xml***: Esta pieza tiene una armadura de clave con un sostenido, lo cual es consistente con el modo D dórico (Re dórico). * **MX-1910-00-EA-00023.musicxml.xml***: Esta pieza presenta una armadura de clave con dos sostenidos, lo que podría indicar el modo E dórico (Mi dórico). * **MX-1910-00-EA-00002.musicxml.xml***: Al igual que la primera, esta pieza tiene una armadura de clave con un sostenido, sugiriendo que podría estar en modo D dórico. * **MX-1910-00-EA-00006.musicxml.xml***: Con una armadura de clave de un sostenido, esta pieza también es compatible con el modo D dórico. * **CO-1948-00-BG-00001.musicxml.xml***: Esta pieza también tiene una armadura de clave con un sostenido, lo que la hace una candidata para estar en modo D dórico.

Es importante señalar que la determinación del modo dórico en estas piezas se basa en la interpretación de las armaduras de clave, ya que los archivos no declaran explícitamente el modo musical."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas en modo dórico:

1. IE-2019-D-HLS-006, 2. IE-2019-D-HLS-012, 3. IE-2019-D-HLS-015, 4. IE-2019-D-HLS-016, 5. IE-2019-D-HLS-025, 6. IE-2019-D-HLS-027, 7. IE-2019-D-HLS-029, 8. IE-2019-D-HLS-035, 9. IE-2019-D-HLS-049, 10. Naña"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.modos
FROM piezas p
WHERE LOWER(p.modos) LIKE '%dórico%';
```

Tras realizar la consulta se obtuvieron 1658 obras en modo dórico.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search básico con Gemini 2.5 Flash no coincide con los resultados esperados. El modelo no realizó una

búsqueda sobre el campo `modo`, sino que infirió posibles casos de modo dórico a partir de las armaduras de clave presentes en los archivos MusicXML. Este razonamiento resulta problemático porque una misma armadura puede corresponder a distintos modos o tonalidades, por lo que la armadura por sí sola no permite determinar con fiabilidad el modo musical. Como consecuencia, las obras devueltas no corresponden necesariamente a piezas clasificadas como dóricas en la base de datos. De hecho, al contrastar los resultados con la consulta SQL, se comprobó que todas las obras propuestas por el modelo eran en realidad piezas en modo jónico, excepto la última, que estaba catalogada como eólica. El error se debe a que el modelo sustituyó una búsqueda explícita sobre los metadatos por una inferencia musicológica basada en información parcial de las partituras.

Por el contrario, la respuesta generada por el modelo local con acceso a SQLite fue correcta. Todas las obras listadas estaban efectivamente clasificadas en modo dórico y coincidían con los resultados de la consulta SQL. Aunque únicamente se muestran diez resultados, la respuesta se considera satisfactoria debido al límite máximo de elementos impuesto en las consultas realizadas por el sistema.

6.1.8. Identifica piezas con un bpm de 120

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Se han identificado las siguientes piezas musicales con un BPM de 120:

* 120. María la pastora (a), * Una pieza del archivo "IE-1909-D-PWJ-120.xml" y "IE-1909-D-PWJ-120.mei", * Una pieza del archivo "goodman_vol_1_120.xml"

Cabe mencionar que la pieza 120. Ayer te quise.xml tiene un tempo de 99, no de 120, a pesar de que 120 aparece en el nombre del archivo. "

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con un BPM de 120:

1. IE-2019-D-HLS-014, 2. IE-2019-D-HLS-028, 3. IE-2019-D-HLS-037, 4. IE-2019-D-HLS-049, 5. IE-2019-D-HLS-050, 6. IE-2019-D-HLS-058, 7. IE-2019-D-HLS-059, 8. IE-2019-D-HLS-014 (con ID 9684), 9. IE-2019-D-HLS-028 (con ID 9698), 10. IE-2019-D-HLS-037 (con ID 9707)

Nota: Hay dos piezas con el mismo título pero diferentes IDs, lo que sugiere que pueden ser versiones o revisiones de la misma pieza."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.bpm
FROM piezas p
WHERE p.bpm = 120;
```

Tras realizar la consulta se obtuvieron 27 obras con un bpm 120.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search básico con Gemini 2.5 Flash resulta incorrecta debido a que no se basa en el atributo estructurado `bpm`, sino en coincidencias textuales en nombres de archivos y descripciones.

Esto provoca falsos positivos, como piezas que incluyen “120” en el título o en el nombre del fichero, pero cuyo tempo real no corresponde a 120 BPM. Incluso el propio modelo reconoce este problema al indicar explícitamente un caso con BPM 99, lo que evidencia la falta de consistencia en el criterio de filtrado. En consecuencia, los resultados no son fiables como consulta estructurada sobre la base de datos.

En contraste, la respuesta del modelo local con acceso a SQLite es correcta en cuanto a la identificación de piezas con `bpm = 120`, ya que opera directamente sobre el campo numérico correspondiente. Sin embargo, el listado se ve afectado por el límite de salida (top-10 resultados), lo que impide mostrar la totalidad de las 27 obras existentes en la base de datos. Aun así, el conjunto devuelto es coherente con la consulta SQL de referencia, por lo que se considera una respuesta válida y completa dentro de las restricciones del sistema.

6.1.9. Identifica piezas con un volumen midi de 79

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Se han identificado varias piezas con un volumen MIDI muy cercano a 79 (específicamente 78.7402). Estas piezas son las siguientes:

* Una pieza titulada 379, compuesta por Vicente T. Mendoza, donde la parte es de piano. * Otra pieza titulada 479, también compuesta por Vicente T. Mendoza y con una parte de piano. * Una obra titulada 79. Toque militar, la cual incluye una parte para gran piano."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con un volumen midi de 79:

1. 5, 2. 8, 3. Allegretto, 4. Allegro Moderato, 5. NANA, 6. LA LLUVIA, 7. AVE MARIS STELLA, 8. EL TURPIALITO"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.midi_volume
FROM piezas p
WHERE p.midi_volume = 79;
```

Tras realizar la consulta se obtuvieron 1549 obras con un volumen midi de 79.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash puede considerarse parcialmente correcta, ya que identifica obras cuyo volumen MIDI es cercano al valor solicitado y proporciona ejemplos concretos. Sin embargo, no responde exactamente a la consulta formulada, que requería localizar todas las piezas con el valor `midi_volume = 79`. Además, la recuperación se basa en coincidencias encontradas en los archivos musicales y devuelve únicamente tres ejemplos, sin evidenciar una búsqueda exhaustiva sobre el conjunto de datos. Por tanto, la respuesta es correcta en términos aproximados, pero incompleta respecto al criterio exacto solicitado.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las obras listadas presentan un volumen MIDI igual a 79 según la base de datos

y constituyen una muestra válida de los resultados esperados. Aunque únicamente devuelve ocho ejemplos debido a la limitación de longitud impuesta al sistema, las respuestas recuperadas coinciden con los registros obtenidos mediante la consulta SQL.

6.1.10. Identifica piezas cuya nota más grave sea A3

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** Según la información disponible, no se pudo identificar ninguna pieza musical cuya nota más grave sea específicamente A3. Algunos documentos muestran la presencia de la nota A3, pero también contienen notas más graves como A2 o A1. Otros documentos presentan notas más agudas como la más grave, como A4, D4 o G4.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** “Se encontraron las siguientes piezas cuya nota más grave es A3:
* ID 64, * ID 211, * ID 221, * ID 1219, * ID 1535, * ID 1562, * ID 1645, * ID 1718, * ID 1736, * ID 1742”
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.nota_mas_grave
FROM piezas p
WHERE p.nota_mas_grave = 'A3';
```

Tras realizar la consulta se obtuvieron 403 obras cuya nota más grave es A3.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. Aunque el modelo analiza algunos documentos y detecta la presencia de la nota A3 en determinadas obras, concluye erróneamente que no puede identificarse ninguna pieza cuya nota más grave sea exactamente A3.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las obras listadas cumplen la condición de tener A3 como nota más grave, de acuerdo con los registros almacenados en la base de datos. No obstante, únicamente se muestran diez resultados debido a la limitación impuesta sobre el número máximo de respuestas devueltas por el sistema.

6.1.11. Identifica piezas cuya nota más aguda sea G5

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Basándome en la información proporcionada, he identificado varios archivos de música que contienen la nota G5. Sin embargo, no puedo confirmar que G5 sea la nota más aguda en ninguna de estas piezas en su totalidad, ya que algunos fragmentos también muestran la presencia de notas más altas como A5.

Los archivos que contienen la nota G5 incluyen: * 'MX-1952-00-CR-00002.musicxml.xml' * 'IE-2019-D-HLS-015.xml' * 'goodman_vol_1_495.xml' * 'goodman_vol_2_132.xml' * 'goodman_vol_1_505.xml'

Es importante señalar que, en los archivos 'MX-1952-00-CR-00002.musicxml.xml' y 'IE-2019-D-HLS-015.xml', la nota A5 también aparece después de G5, lo que indica que G5 no es la nota más aguda en esas secciones específicas de las piezas. Para determinar la nota más aguda de una pieza completa, sería necesario un análisis exhaustivo de la partitura entera."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas cuya nota más aguda es G5: IE-2019-D-HLS-001, IE-2019-D-HLS-003, IE-2019-D-HLS-004, IE-2019-D-HLS-008, IE-2019-D-HLS-012, IE-2019-D-HLS-024, IE-2019-D-HLS-026, IE-2019-D-HLS-027, IE-2019-D-HLS-031 y IE-2019-D-HLS-033."
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.nota_mas_aguda
FROM piezas p
WHERE p.nota_mas_aguda = 'G5';
```

Tras ejecutar la consulta se obtuvieron 1529 obras cuya nota más aguda es G5.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. Aunque el modelo identifica varias piezas que contienen la nota G5, la consulta solicitaba específicamente localizar aquellas obras cuya nota más aguda fuese G5. El modelo no logra verificar esta condición y, de hecho, señala que algunas de las obras recuperadas contienen notas más agudas, como A5, por lo que no cumplen el criterio solicitado. En consecuencia, la respuesta se limita a detectar la presencia de la nota G5 dentro de las partituras recuperadas, sin determinar correctamente si constituye el registro máximo de cada obra.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las piezas listadas presentan G5 como nota más aguda según los registros almacenados en la base de datos. No obstante, únicamente se muestran diez resultados debido a la limitación establecida sobre el número máximo de respuestas devueltas por el sistema.

6.1.12. Identifica piezas cuyo autor sea de género masculino

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Según la información disponible, no se pueden identificar piezas cuyos autores sean de género masculino de manera concluyente.

Las obras encontradas incluyen: * ASSERRIN, ASERRAN, cuyo autor está listado como Compositor / arreglista. * 80- A BELÉN CORRAMOS, también atribuida a un Compositor / arreglista. * Mis Señoras a donde van ustedes, que fue dictada por la niña Fernanda F. Rabanada, indicando una autora femenina. * 163. Los números retornados, para la cual no se menciona un compositor específico. * Gaffney's Favorite Son, cuyo autor se indica como Compositor / arreglista.

Dado que Compositor / arreglista no especifica el género del autor, y en el caso de F. Rabanada se menciona explícitamente niña Fernanda, no es posible confirmar que alguna de estas piezas sea de un autor masculino basándose únicamente en los datos proporcionados."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas cuyo autor sea de género masculino:

* ID: 2514, * ID: 2518, * ID: 2532, * ID: 2562, * ID: 3004, * ID: 3010, * ID: 3014, * ID: 3025, * ID: 3030, * ID: 3037

Es posible que haya más piezas con autores de género masculino en la base de datos.
"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.autor_genero
FROM piezas p
WHERE p.autor_genero = 'masculino';
```

Tras ejecutar la consulta se obtuvieron 1451 obras con autor de género masculino.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. El modelo no consigue identificar correctamente las obras cuyo autor tiene género masculino y concluye que no es posible determinar dicha información a partir de los documentos recuperados. En su lugar, analiza referencias textuales presentes en algunas partituras y metadatos parciales, infiriendo erróneamente que la ausencia de una indicación explícita del género impide responder a la consulta.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las obras listadas presentan el valor `masculino` en el campo `autor_genero`, por lo que cumplen la condición solicitada. No obstante, únicamente se muestran diez resultados debido a la limitación establecida sobre el número máximo de respuestas devueltas por el sistema.

6.1.13. Identifica piezas cuyo software de codificación haya sido Sibelius 7.1.3

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** No se han encontrado piezas cuyo software de codificación sea Sibelius 7.1.3 en la información proporcionada. Los resultados de la búsqueda indican que las piezas encontradas fueron codificadas con MuseScore 4.2.1 o MuseScore 4.1.1.

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas cuyo software de codificación haya sido Sibelius 7.1.3:

* Fáinne Geal an Lae (ID: 1998), * An Clár Bog Déil (ID: 1999), * Reel (ID: 2000, ID: 2005, ID: 2006), * Miss Monaghan's Reel (ID: 2001), * Betsy Baker (ID: 2002), * Miss Johnson (ID: 2003), * The Wind that Shakes the Barley (ID: 2004), * The Lady's Cup of Tea (ID: 2007)"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.software_codificacion
FROM piezas p
WHERE p.software_codificacion = 'Sibelius 7.1.3';
```

Tras realizar la consulta se obtuvieron 4641 obras codificadas con Sibelius 7.1.3.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. El modelo concluye que no existen piezas codificadas con Sibelius 7.1.3 porque, entre los documentos recuperados durante la búsqueda, únicamente encuentra referencias a versiones de MuseScore. Sin embargo, esta conclusión se deriva de una muestra limitada de archivos recuperados y no de una consulta exhaustiva sobre los metadatos del conjunto de datos. Como demuestra la consulta SQL, existen numerosas obras cuyo campo `software_codificacion` tiene exactamente el valor Sibelius 7.1.3.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las obras listadas presentan Sibelius 7.1.3 como software de codificación según los registros almacenados en la base de datos. No obstante, únicamente se muestran algunos ejemplos debido a la limitación impuesta sobre el número máximo de resultados devueltos por el sistema.

6.1.14. Identifica piezas cuya fecha de codificación sea el 2015-07-01

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** No se encontraron piezas con la fecha de codificación 2015-07-01 en los resultados de la búsqueda. Los archivos encontrados, como “ES-1948-CB-CO1-015.mei”, “ES-1948-CB-CO-015.mei”, “ES-1948-CB-CO1-415.mei” y “ES-2000-AS-EMT-015.mei”, indican una fecha de aplicación de “2023-02-22” para el proceso de transcodificación, no para la codificación original de la pieza.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** “Se encontraron las siguientes piezas cuya fecha de codificación sea el 2015-07-01:
1. Ceann Dubh Dileas, 2. The Steam Packet, 3. The Bantry Jig, 4. The Sandy Banks of Rush, 5. Lough Erne’s Water, 6. The Humours of Redgap, 7. When Mary was a Young Thing, 8. A Highland Reel, 9. The Gipsy, 10. The Basket of Oysters”
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, p.fecha_codificacion
FROM piezas p
WHERE p.fecha_codificacion = '2015-07-01';
```

Tras realizar la consulta se obtuvieron 1607 obras codificadas el 2015-07-01.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. El modelo no identifica correctamente las obras con fecha de codificación 2015-07-01 y concluye erróneamente que no existen coincidencias, basándose en metadatos de otros procesos de transcodificación con fechas distintas

(por ejemplo, 2023-02-22). Esto indica una confusión entre la fecha de codificación original de la pieza y la fecha de procesamiento posterior de los archivos, lo que provoca un falso negativo respecto a la consulta solicitada.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las obras listadas cumplen la condición de tener el campo `fecha_codificacion` igual a 2015-07-01. No obstante, la salida muestra únicamente un subconjunto de los resultados totales debido a la limitación en el número de elementos devueltos por la consulta o por el formato de respuesta.

6.1.15. Identifica piezas con síncopas

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Los resultados de la búsqueda proporcionan archivos MusicXML que contienen datos de partituras, pero no identifican piezas musicales específicas por su título que sean conocidas por sus síncopas. Los nombres de los archivos son genéricos como Partitura sin título o identificadores numéricos, lo que no permite identificar las obras.

Para identificar piezas con síncopas, sería necesario analizar la estructura rítmica dentro de cada uno de estos archivos MusicXML, lo cual no es posible directamente a partir de los fragmentos de texto proporcionados."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con síncopas:

* IE-2019-D-HLS-012, * IE-2019-D-HLS-014, * IE-2019-D-HLS-018, * IE-2019-D-HLS-027, * IE-2019-D-HLS-031, * IE-2019-D-HLS-032, * IE-2019-D-HLS-043, * IE-2019-D-HLS-048, * IE-2019-D-HLS-053, * IE-2019-D-HLS-058

Estas piezas tienen síncopas según el análisis musical."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.tiene_sincopas = 1;
```

Tras realizar la consulta se obtuvieron 538 obras con síncopas.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. El modelo no identifica ninguna pieza concreta con síncopas y argumenta que la información disponible no permite realizar la clasificación solicitada, ya que los archivos recuperados no contienen metadatos explícitos sobre este rasgo rítmico a nivel de obra.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las piezas listadas cumplen la condición `tiene_sincopas = 1` según el análisis musical almacenado en la tabla `analisis_musical`. No obstante, la salida muestra únicamente un subconjunto de los resultados totales debido a la limitación en el número de elementos devueltos por el sistema.

6.1.16. Identifica piezas que traten la muerte como tema

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Varias piezas musicales y canciones tratan la muerte como tema central. Entre ellas se encuentran:

* 92. Estaba la Muerte un día... (b), * 125. La muerte (b), * 91. La media muerte (a), * Una canción que narra la muerte de Mercedes, describiendo su funeral con versos como Ya Mercedes está muerta, muerta está, que yo la vi, cuatro duques la llevaban por las calles de Madrid. También menciona detalles de su vestimenta y el luto por su partida, como que Los caballos de Palacio ya no quieren pasear. * 396. Se murió. (Cuatro voces)."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas que tratan la muerte como tema:

* Ama a los viejos, * 568, * 571, * 579, * 588, * 592, * 596, * 607, * 613, * 8

Estas piezas tienen un total de eventos musicales relacionados con la muerte en su análisis musical."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.temas
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(a.temas) LIKE '%muerte%';
```

Tras realizar la consulta, se obtuvieron 768 obras que tratan el tema de la muerte en sus letras.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es parcialmente correcta desde un punto de vista semántico, ya que identifica varias obras relacionadas con el tema de la muerte y aporta ejemplos textuales que evidencian dicha temática en el contenido lírico. No obstante, el enfoque se basa en coincidencias en los fragmentos recuperados y no en una consulta estructurada sobre los temas, lo que limita la exhaustividad del resultado y puede introducir sesgos de recuperación. En consecuencia, aunque las piezas mencionadas son plausibles, la respuesta no garantiza cobertura completa del conjunto de datos.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las piezas listadas corresponden a registros cuyo campo `temas` contiene explícitamente la palabra `muerte`, cumpliendo exactamente la condición de la consulta SQL. Sin embargo, la salida muestra únicamente un subconjunto de los resultados totales debido a la limitación del formato de salida o al truncamiento de la respuesta.

6.1.17. Identifica piezas con polirritmia

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Se han identificado varias piezas con polirritmia o elementos rítmicos complejos que sugieren su presencia:

* Un arrullo (Lullaby) de Extremadura, España, identificado como ES-1944-EX-MGM-002.mei, se describe con una métrica polirrítmica. * Otro arrullo de Extremadura, España, ES-2000-CC-MGM-002.mei, también presenta una métrica polirrítmica.

Además, otras piezas contienen descripciones rítmicas que podrían implicar polirritmia:

* 244. Seguidillas boleras de Burgos.xml incluye una sección de Ritmo que podría sugerir complejidades rítmicas. * 246. Tente que me caigo rueda burgalesa .xml menciona el ritmo de la pandereta (rhythm of the tambourine), lo cual puede indicar patrones rítmicos superpuestos. * 281. Daime vino cancion majar.xml describe Golpes alternos de los dos grupos de majadores (alternating beats of two groups of threshers), lo que podría crear una textura polirrítmica. "

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con polirritmia:

1. IE-2019-D-HLS-010, 2. IE-2019-D-HLS-011, 3. IE-2019-D-HLS-015, 4. IE-2019-D-HLS-018, 5. IE-2019-D-HLS-019, 6. IE-2019-D-HLS-021, 7. IE-2019-D-HLS-031, 8. IE-2019-D-HLS-032, 9. IE-2019-D-HLS-057, 10. El reloj

Estas piezas tienen polirritmia, según los análisis musicales realizados."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.tiene_polirritmia = 1;
```

Tras realizar la consulta se obtuvieron 1904 obras con polirritmia.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. El modelo infiere la presencia de polirritmia a partir de descripciones textuales vagas o interpretaciones de elementos rítmicos complejos (como menciones a golpes alternos o patrones de pandereta). Esto conduce a una sobreinterpretación de evidencia indirecta y a la inclusión de ejemplos que no están etiquetados explícitamente como polirritmia en el conjunto de datos. En consecuencia, la respuesta no es verificable ni exhaustiva y no refleja de forma fiable los registros reales.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las piezas listadas cumplen la condición `tiene_polirritmia = 1` según el análisis musical almacenado en la tabla `analisis_musical`. No obstante, la salida muestra únicamente un subconjunto de los resultados totales debido a la limitación del formato de respuesta o del número máximo de elementos devueltos.

6.1.18. Identifica piezas cuyo contenido lírico esté escrito en voz masculina

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Based on the search results, here's a piece whose lyrical content is written in a male voice:

* **Macario Romero** (MX-1939-00-VM-00234) tells the story of Macario Romero, a man who is warned by General Plata about seeking a woman and is later killed. The lyrics directly quote Macario Romero speaking, for example: Dice Macario Romero: -Oiga, mi general Plata, consígame un permisito para ir a ver a mi chata-. The song also mentions Panchito Díaz lo devisa: -Papá, ahí viene Macario, desde leguas lo conozco, en su caballo melao-, further indicating a male perspective through the character interactions."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas cuyo contenido lírico esté escrito en voz masculina:

* ID: 12170, * ID: 12171, * ID: 12172, * ID: 12174, * ID: 12179, * ID: 12192, * ID: 12202, * ID: 12204, * ID: 12205, * ID: 12208

Es posible que haya más piezas con contenido lírico en voz masculina en la base de datos."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.lirica_voz
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.lirica_voz = 'masculino';
```

Tras realizar la consulta se obtuvieron 2618 obras cuyo contenido lírico está escrito en voz masculina.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. El modelo basa su conclusión en el análisis de una única pieza concreta ("Macario Romero"), interpretando el contenido narrativo y la voz de los personajes como evidencia suficiente para determinar el género de la voz lírica. Sin embargo, esta inferencia se realiza a partir de un caso aislado, lo que introduce un sesgo de generalización y una falta de cobertura del conjunto de datos. En consecuencia, la respuesta no es representativa del total de obras etiquetadas como voz masculina.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las piezas listadas cumplen la condición `lirica_voz = 'masculino'` según los metadatos del análisis musical. No obstante, la salida muestra únicamente un subconjunto de los resultados totales debido a la limitación en el número de elementos devueltos por el sistema.

6.1.19. Identifica obras cuya tendencia melódica sea predominantemente descendente

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Los resultados de la búsqueda incluyen varias partituras de MusicXML y MEI que contienen información musical. Para identificar obras con una tendencia melódica predominantemente descendente, se necesitaría un análisis

más profundo de las notas dentro de cada partitura. Sin embargo, algunos títulos o indicaciones pueden ofrecer pistas:

Una obra titulada Partitura sin título para acordeón, con una indicación de tempo Adagio, muestra secuencias de notas descendentes en las medidas 3 y 4 (G5-F#5-E5 y A5-G5-F#5-E5).

Otra partitura, también titulada Partitura sin título y para acordeón, presenta en la medida 2 una secuencia descendente (Bb5-A5-G5).

Por otro lado, la obra 25. SUBE Y BAJA con un subtítulo, sugiere que puede tener tanto movimientos ascendentes como descendentes, por lo que su tendencia no sería predominantemente descendente.

Finalmente, una canción infantil titulada Qué Lloras Retirada de María García, en modo eólico y con tempo Andante, tiene una nota C5 seguida de B4 en la primera medida y luego A4, lo que podría indicar una tendencia descendente en su inicio."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes obras con tendencia melódica predominantemente descendente:

1. IE-2019-D-HLS-001, 2. IE-2019-D-HLS-002, 3. IE-2019-D-HLS-003, 4. IE-2019-D-HLS-006, 5. IE-2019-D-HLS-013, 6. IE-2019-D-HLS-016, 7. IE-2019-D-HLS-019, 8. IE-2019-D-HLS-021, 9. IE-2019-D-HLS-023, 10. IE-2019-D-HLS-024

Estas obras presentan una tendencia melódica descendente en más de un 50 % de los intervalos utilizados, lo que indica que la melodía tiende a moverse hacia abajo."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.tendencia_melodica
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.tendencia_melodica = 'Predominantemente descendente';
```

Tras ejecutar la consulta se obtuvieron 6000 obras cuya tendencia melódica es predominantemente descendente.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. El modelo intenta inferir la tendencia melódica descendente a partir de fragmentos locales de algunas partituras recuperadas (por ejemplo, secuencias de notas descendentes en compases concretos), pero no realiza un análisis exhaustivo y completo de la obra. Esto provoca una decisión basada en evidencia parcial y no en el conjunto completo de datos, lo que introduce sesgos de muestreo y una cobertura muy limitada del corpus.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las obras listadas cumplen la condición `tendencia_melodica = 'Predominantemente descendente'` según el análisis musical almacenado en la base de datos. No obstante, la salida presenta únicamente un subconjunto de los resultados totales debido a la limitación del número de elementos devueltos por el sistema.

6.1.20. Identifica piezas con 4 alteraciones accidentales

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "No se encontraron piezas en los archivos proporcionados que contengan específicamente 4 alteraciones accidentales en su armadura.

Los archivos de música revisados (MX-1910-00-EA-00007.musicxml.xml, MX-1910-00-EA-00003.musicxml.xml, MX-1910-00-EA-00001.musicxml.xml, MX-1910-00-EA-00029.musicxml.xml, MX-1910-00-EA-00011.musicxml.xml) indican que las piezas tienen una armadura con 2 alteraciones (dos sostenidos), lo que corresponde a la tonalidad de Re Mayor o Si menor.

Para tener 4 alteraciones accidentales en la armadura, las tonalidades serían: * **Con 4 sostenidos**: Mi Mayor o Do sostenido menor. * **Con 4 bemoles**: La bemol Mayor o Fa menor."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Las piezas con 4 alteraciones accidentales son:

* IE-2019-D-HLS-007, * IE-2019-D-HLS-009, * IE-2019-D-HLS-011, * IE-2019-D-HLS-017, * IE-2019-D-HLS-018, * IE-2019-D-HLS-023, * IE-2019-D-HLS-026, * IE-2019-D-HLS-030, * IE-2019-D-HLS-035, * IE-2019-D-HLS-037"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.conteo_accidentales_totales
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.conteo_accidentales_totales = 4;
```

Tras ejecutar la consulta se obtuvieron 190 obras con 4 alteraciones accidentales.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. El modelo interpreta la pregunta en términos de armaduras de clave y tonalidades, confundiendo el concepto de "4 alteraciones accidentales" con la cantidad de alteraciones en la armadura (sostenidos o bemoles). Esto provoca una desviación conceptual que impide identificar correctamente las obras solicitadas y conduce a una respuesta basada en teoría tonal general en lugar de datos del conjunto.

La respuesta generada por el modelo local con acceso a SQLite es correcta. Todas las piezas listadas cumplen la condición `conteo_accidentales_totales = 4` según el análisis musical almacenado en la base de datos. No obstante, la salida presenta únicamente un subconjunto de los resultados totales debido a la limitación del número de elementos devueltos por el sistema.

6.1.21. Identifica piezas con compases vacíos

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Las piezas que contienen compases vacíos, indicados por

la presencia de una marca de silencio de compás completo ('<rest measure=yes/>'), son las siguientes:

* MI-1921-00-AL-03517.musicxml.xml * MI-1921-00-AL-03518.musicxml.xml * MX-1939-00-VM-00265.musicxml.xml

Adicionalmente, la pieza CO-1948-00-BG-00041.musicxml.xml incluye compases compuestos enteramente por silencios, como el compás 1 con un silencio de blanca y uno de negra, y el compás 2 con tres silencios de negra en una métrica de 3/4."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** Se encontraron las siguientes piezas con compases vacíos: Himno a Bolívar, Me tuviste, 275556 Bog Road Slow, 588, Campbells are Comin', Catholic Boys, Charlie Mack, Doyle's Frolic, Drum, Garry owen.
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.qc_compases_vacios
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.qc_compases_vacios > 0;
```

Tras ejecutar la consulta se obtuvieron 612 obras con compases vacíos.

- **Revisión de respuestas.** La respuesta proporcionada por el primer modelo presenta resultados parcialmente correctos, ya que identifica algunas piezas que efectivamente contienen compases vacíos. Sin embargo, la cobertura es incompleta y se limita a un subconjunto muy reducido de la base de datos, sin reflejar el conjunto total de obras. Además, se observa cierta inconsistencia en la definición del criterio de compás vacío: el modelo mezcla casos de silencios explícitos a nivel de compás completo con descripciones de compases con silencios parciales, lo que introduce ambigüedad en la clasificación.

Por el contrario, la respuesta del segundo modelo es correcta, ya que recupera el conjunto de piezas correspondiente a la condición `qc_compases_vacios > 0`, alineándose con la consulta SQL de referencia y cubriendo adecuadamente la totalidad de resultados esperados en la base de datos.

6.1.22. Identifica piezas con notas de duración 0

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** No se han encontrado piezas con notas de duración 0 en los documentos proporcionados. Los resultados de la búsqueda muestran que las notas y silencios tienen duraciones como 1, 2, 3, 4, 6 o 8, o tipos como ".eighth"(corchea) y "16th"(semicorchea), pero ninguna con una duración de 0.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con notas de duración 0:

* IE-2019-D-HLS-006, * IE-2019-D-HLS-007, * IE-2019-D-HLS-015, * IE-2019-D-HLS-025, * IE-2019-D-HLS-026, * IE-2019-D-HLS-028, * IE-2019-D-HLS-030, * 575, * 579, * 585"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.qc_notas_duracion_cero
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.qc_notas_duracion_cero > 0;
```

Tras ejecutar la consulta se obtuvieron 1784 obras con notas de duración 0.

- **Revisión de respuestas.** La respuesta del primer modelo es incorrecta, ya que concluye la ausencia de notas con duración 0 basándose en una interpretación superficial de los valores de duración presentes en los documentos. Este enfoque confunde la representación explícita de la duración en el formato textual con la existencia de registros donde el análisis musical identifica explícitamente duraciones nulas.

En cambio, la respuesta del segundo modelo es correcta, ya que recupera los identificadores de piezas que cumplen la condición `qc_notas_duracion_cero > 0`, coincidiendo con el resultado de la consulta SQL de referencia.

6.1.23. Identifica piezas con plicas anómalas

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Las piezas que contienen información sobre la dirección de las plicas (stems) y, por lo tanto, podrían ser examinadas en busca de plicas anómalas son las siguientes:

* **ES-1948-CB-CO-312.mei***: Este archivo XML de codificación musical (MEI) incluye notas donde se especifica la dirección de la plica, como 'stem.dir=up'. * **ES-1948-CB-CO1-312.mei***: Similar al anterior, este archivo también contiene notas con el atributo 'stem.dir=up', indicando la dirección de la plica.

Cabe destacar que, para determinar qué constituye una plica anómala, sería necesaria una definición específica de anomalía dentro del contexto de la notación musical. Los archivos proporcionados indican la dirección de las plicas (por ejemplo, hacia arriba), pero no clasifican explícitamente ninguna como anómala."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con plicas anómalas:

- IE-2019-D-HLS-019, - Fáinne Geal an Lae, - An Clár Bog Déil, - Miss Monaghan's Reel, - Betsy Baker, - Miss Johnson, - The Wind that Shakes the Barley, - Reel, - Reel, - The Lady's Cup of Tea"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.tiene_plicas_anomalias
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.tiene_plicas_anomalias = 1;
```

Tras ejecutar la consulta se obtuvieron 5554 obras con plicas anómalas.

- **Revisión de respuestas.** La respuesta del primer modelo es incorrecta, ya que se limita a identificar la presencia explícita del atributo ‘stem.dir’ en algunos archivos, sin aplicar el criterio de detección de plicas anómalas definido en el análisis musical. Esto conduce a una interpretación incompleta del problema, reduciendo la tarea a una simple detección de metadatos en lugar de una clasificación basado en un análisis más exhaustivo.

Por el contrario, la respuesta del segundo modelo es correcta, ya que recupera las piezas que cumplen la condición `tiene_plicas_anomalias = 1`, coincidiendo con la consulta SQL de referencia. Este resultado refleja adecuadamente la definición operacional de plicas anómalas en la base de datos y es consistente con el conjunto completo de obras identificadas.

6.1.24. Identifica piezas que usen la palabra “bondad”

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** No se encontraron piezas que contengan la palabra “bondad” en los resultados de la búsqueda.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas que usan la palabra bondad:

* Pieza con id 137: Padre nuestro * Pieza con id 8002: 73. PADRE NUESTRO

Nota: Las títulos se muestran exactamente como se encuentran en la base de datos."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.lirica_sustantivos
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(a.lirica_sustantivos) LIKE '%bondad%';
```

Tras ejecutar la consulta se obtuvieron dos canciones que contienen la palabra “bondad”: “Padre nuestro” y “73. PADRE NUESTRO”

- **Revisión de respuestas.** La respuesta del primer modelo es incorrecta, ya que concluye la ausencia de coincidencias sin evidenciar un análisis exhaustivo del campo lírico, lo que puede indicar una limitación del proceso de recuperación o una búsqueda superficial que no considera variantes de normalización textual (mayúsculas, tildes o tokenización del contenido).

Por el contrario, la respuesta del segundo modelo es correcta, ya que identifica las dos piezas que contienen la palabra “bondad” en el atributo `lirica_sustantivos`,

coincidiendo con el resultado de la consulta SQL de referencia. Esto confirma que su extracción está alineada con el criterio definido en la base de datos y recupera correctamente todos los casos esperados.

6.1.25. Identifica piezas que mencionen el nombre propio “Alfonsito”

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** No se encontró ninguna pieza que mencione el nombre propio “Alfonsito” en los documentos proporcionados. Los resultados de la búsqueda hacen referencia a “Alfonso XII”, “Alfonso Doce” y “Angelito”, pero no a “Alfonsito” específicamente.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas que mencionan el nombre propio Alfonsito:
 * Pieza con id 6940: No se especifica el título en la base de datos. * Pieza con id 7270: No se especifica el título en la base de datos."
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.lirica_nombres_propios
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(a.lirica_nombres_propios) LIKE '%Alfonsito%';
```

Tras ejecutar la consulta se obtuvieron dos obras que contienen el nombre propio “Alfonsito”: “None (id: 6940)” y “None (id: 7270)”

- **Revisión de respuestas.** La respuesta del primer modelo es incorrecta, ya que descarta la presencia del nombre propio “Alfonsito” basándose en coincidencias textuales aproximadas (como “Alfonso XII” o “Alfonsito” en variantes no normalizadas), sin realizar una búsqueda exacta sobre el campo estructurado de nombres propios. Esto provoca un falso negativo, al no considerar que el término puede aparecer en el análisis lírico de forma explícita y normalizada.

Por el contrario, la respuesta del segundo modelo es correcta, ya que identifica las piezas que contienen el término “Alfonsito” en el atributo `lirica_nombres_propios`, coincidiendo con el resultado de la consulta SQL de referencia. Aunque los títulos no estén especificados en la base de datos para estos registros, la extracción es válida y completa según el criterio definido en el esquema de datos.

6.1.26. Identifica piezas cuyo intervalo frontera más frecuente sea +5

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** The provided search results are MusicXML files, which are a digital sheet music format. To identify pieces with a frequent -5"boundary interval, I would need to parse these files, extract the melodic intervals, and then analyze their frequency.

Unfortunately, I cannot directly read.or interpret"the musical content of these MusicXML files to calculate intervals and their frequencies.

The snippets show musical notation in a structured text format (e.g., '<pitch><step>D</step><octave>5</octave></pitch>'), but I cannot perform the complex musical analysis needed to identify specific intervals like -5.^{as} "most frequent boundary interval" without more advanced capabilities or a specific tool designed for music information retrieval.

Therefore, I cannot identify pieces whose most frequent boundary interval is +5 based on the provided search results. I only have raw MusicXML data, not an analysis of their melodic intervals.

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas cuyo intervalo frontera más frecuente sea +5:

* IE-2019-D-HLS-002 (Título: IE-2019-D-HLS-002, Autor: , Compás: 4/4, Tonalidad: G mayor, BPM: 62) * IE-2019-D-HLS-005 (Título: IE-2019-D-HLS-005, Autor: , Compás: 12/8, Tonalidad: G mayor, BPM: 96) * IE-2019-D-HLS-028 (Título: IE-2019-D-HLS-028, Autor: , Compás: 4/4, Tonalidad: G mayor, BPM: 120) * IE-2019-D-HLS-035 (Título: IE-2019-D-HLS-035, Autor: , Compás: 4/4, Tonalidad: C mayor, BPM: 90) * IE-2019-D-HLS-038 (Título: IE-2019-D-HLS-038, Autor: , Compás: 2/4, Tonalidad: C mayor, BPM: 80) * IE-2019-D-HLS-047 (Título: IE-2019-D-HLS-047, Autor: , Compás: 3/8, Tonalidad: G mayor, BPM: 112) * IE-2019-D-HLS-048 (Título: IE-2019-D-HLS-048, Autor: , Compás: 6/8, Tonalidad: G mayor, BPM: 80) * IE-2019-D-HLS-057 (Título: IE-2019-D-HLS-057, Autor: , Compás: 4/4, Tonalidad: G mayor, BPM: 128) * IE-2019-D-HLS-062 (Título: IE-2019-D-HLS-062, Autor: , Compás: 6/4, Tonalidad: B menor, BPM: 180) * Ejercicio (Título: Ejercicio, Autor: José Miguel Hernández Jaramillo, Compás: 4/4, Tonalidad: G mayor)"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.intervalo_frontera_mas_frecuente = '+5';
```

Tras ejecutar la consulta se obtuvieron 732 obras cuyo intervalo frontera más frecuente es +5.

- **Revisión de respuestas.** La respuesta del primer modelo es incorrecta, ya que reconoce explícitamente la incapacidad de analizar los intervalos melódicos a partir de los datos MusicXML proporcionados. Aunque esta observación es razonable desde un punto de vista descriptivo, el modelo no realiza ninguna inferencia ni recuperación estructurada mediante análisis musical.

Por el contrario, la respuesta del segundo modelo es correcta, ya que identifica las piezas cuyo campo `intervalo_frontera_mas_frecuente` es igual a '+5', coincidiendo con la consulta SQL de referencia.

6.1.27. Identifica piezas en 2/4 que contengan síncopas ligadas entre el segundo pulso de un compás y el primero del siguiente de forma sistemática

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "La información proporcionada no permite identificar piezas específicas en 2/4 que contengan síncopas ligadas sistemáticamente entre el segundo pulso de un compás y el primero del siguiente. Los resultados de la búsqueda incluyen archivos MusicXML que, si bien indican que algunas piezas están en compás de 2/4, como se ve en el archivo "MX-1910-00-EA-00032.musicxml.xml" y "CO-1948-00-BG-00007.musicxml.xml", no ofrecen un análisis textual de las ligaduras de síncopa entre compases. Para confirmar esta característica, sería necesario un análisis detallado del contenido musical de cada archivo."
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas en compás de 2/4 que contienen síncopas ligadas entre el segundo pulso y el primero del siguiente:

An Smachdaoin Crón, 'I'm Asleep & Don't Waken Me', 1018. The Cottage Maids — Fling, 1030. 'Hey, Daddy' — Walk Around, 1034. Carry the News to Mary — Walk Around, 1035. Chaw Roast Beef — Walk Around, 147. Congress Park — Reel, 251. Magnolia — Reel, 385. The Northern Light — Reel, 670. Celebrated Stop — Hornpipe

Estas piezas pueden interesarte si buscas música con patrones rítmicos complejos y sistemáticos."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE p.compas = '2/4'
AND a.tiene_sincopas = 1
AND (
    LENGTH(a.compases_sincopas) - LENGTH(REPLACE(a.compases_sincopas,
    ',, ', ''))
) >= 1;
```

Tras ejecutar esta consulta se obtienen 99 resultados que coinciden con la condición presente en la pregunta formulada a los modelos.

- **Revisión de respuestas.** La respuesta obtenida mediante el sistema de file search de la API de Gemini evidencia una clara limitación en el enfoque basado exclusivamente en recuperación de documentos no estructurados. El modelo identifica correctamente la existencia de piezas en compás de 2/4, pero reconoce explícitamente la ausencia de información estructurada sobre relaciones métricas entre compases, lo que le impide verificar la condición de síncopas ligadas entre el segundo pulso de un compás y el primero del siguiente. Este comportamiento es coherente con un sistema RAG

basado en texto, donde las propiedades rítmicas no están formalizadas como variables consultables, sino como contenido implícito dentro de archivos MusicXML.

En contraste, el modelo local basado en llama3.1:8b, con acceso a una base de datos estructurada en SQLite, produce una respuesta aparentemente más útil, pero semánticamente incorrecta en términos musicológicos. Aunque el modelo lista correctamente piezas en compás de 2/4 con presencia de síncopas, interpreta de forma errónea la condición de “síncopas ligadas entre el segundo pulso y el primero del siguiente de forma sistemática”, ya que dicha propiedad no está garantizada por la simple existencia de síncopas ni por su distribución por compases. Mientras que la primera, segunda, quinta, sexta, novena y décima obra devuelta cumplen con la condición solicitada completa, las cuatro piezas restantes sólo cumplen con la condición de contener una síncopa.

La consulta SQL utilizada para la evaluación de referencia recupera un total de 99 piezas, lo que demuestra que tanto el sistema basado en File Search como el basado en RAG SQL fallan en la identificación de la restricción más específica del enunciado, al no existir en la base de datos una representación explícita de la relación métrica intercompás a nivel de pulso. Sin embargo, la solución que más se acerca es la del RAG SQL.

6.1.28. Identifica piezas de tonalidad A menor que tengan hemiolias y que no superen los 90 bpm

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Se ha identificado una pieza musical que cumple con los criterios solicitados. La pieza Partitura sin título para acordeón, con un tempo de Adagio (71 bpm), es una opción a considerar. Sin embargo, para confirmar la tonalidad de La menor y la presencia de hemiolias, sería necesario un análisis más profundo del contenido musical.

Otra pieza disponible es Partitura sin título también para acordeón, que presenta una tonalidad con un bemol en la armadura (lo que podría indicar Fa mayor o Re menor) y un tempo Allegretto de 116 bpm, lo cual excede el límite de 90 bpm.

Existe otra Partitura sin título para acordeón con armadura de dos sostenidos (que sugiere Re mayor o Si menor). Y una Partitura sin título más, también para acordeón, con un bemol en la armadura.

Finalmente, otra Partitura sin título para acordeón también muestra un bemol en la armadura, lo que no corresponde directamente a La menor.

Por lo tanto, la pieza que más se acerca a los requisitos en cuanto a tempo es la Partitura sin título con un tempo de 71 bpm. Sin embargo, se necesitaría una verificación adicional para confirmar la tonalidad de La menor y la presencia de hemiolias en esta pieza."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** No se encontraron piezas que cumplan con los criterios de tonalidad A menor, hemiolias y BPM inferior a 90.
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
```

```

FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(p.tonalidad) = 'a menor'
      AND a.tiene_hemiolia_horizontal = 1
      OR a.tiene_hemiolia_vertical = 1
      AND p.bpm <= 90;

```

Tras ejecutar la consulta se obtuvieron 20 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** Ambas respuestas son incorrectas, ya que no aplican de forma adecuada la combinación de condiciones requerida por la consulta. En el caso del primer modelo, se observa un enfoque exploratorio basado en coincidencias parciales de tonalidad y tempo, sin utilizar los atributos estructurados de hemiolias ni una verificación explícita del criterio 'bpm <= 90', lo que conduce a una inferencia no verificable sobre la adecuación de las piezas.

Por su parte, la respuesta del segundo modelo, aunque sí intenta ceñirse a un filtrado directo sobre la base de datos, falla al no identificar ninguna coincidencia, lo que indica una posible interpretación incorrecta de la condición lógica compuesta (especialmente en la combinación de 'AND' y 'OR' en los predicados de hemiola y BPM) y un posible error en la formulación de la consulta SQL. En consecuencia, ambas respuestas no recuperan correctamente el conjunto esperado de 20 obras según la consulta SQL de referencia.

6.1.29. Identifica piezas de tonalidad A menor que tengan hemiolias

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "The search results provided several musicXML files that are in a minor key (indicated by '<fifths>-1</fifths>', which typically means F major or D minor, but can also relate to A minor as it has one flat in its harmonic/melodic forms, or simply be a placeholder). These files are titled Partitura sin título (Untitled Score) and are composed/arranged by Compositor / arreglista (Composer / Arranger). They are scored for accordion.

Specifically, the following files are in a key that could be interpreted as A minor (or a related minor key with one flat): * 'MX-1910-00-EA-00040.musicxml.xml' * 'CO-1948-00-BG-00039.musicxml.xml' * 'MX-1910-00-EA-00046.musicxml.xml' * 'MX-1910-00-EA-00045.musicxml.xml' * 'MX-1910-00-EA-00043.musicxml.xml'

To determine if these pieces explicitly contain hemiolias, a more detailed musical analysis of each score would be required, as the provided snippets are XML data and do not directly state the presence of hemiolias. A hemiola is a rhythmic device where two measures of triple meter are articulated as three measures of duple meter, or vice-versa, or where a pattern of 3 beats is imposed over a pattern of 2 beats (or vice versa) within a single measure. While some measures in these files indicate a 6/8 time signature (e.g., 'MX-1910-00-EA-00040.musicxml.xml', 'MX-1910-00-EA-00046.musicxml.xml', 'MX-1910-00-EA-00045.musicxml.xml', 'MX-1910-00-EA-00043.musicxml.xml'), which often implies a compound duple feel (two main beats, each divided into three), and others are in 2/4 (e.g., 'CO-1948-00-BG-00039.musicxml.xml'), the XML snippets alone do not allow for a definitive iden-

tification of hemiolas without visual inspection of the notation or further analysis of the rhythmic grouping."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron piezas de tonalidad A menor que tengan hemiolias verticales. Algunas de las piezas son:

- Id 1873, - Id 1875, - Id 8114, - Id 9878, - Id 9881

Estas piezas tienen hemiolias verticales, lo que significa que su estructura armónica incluye patrones de tonos y semitonos que se repiten en intervalos de tercera o sexta.
"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(p.tonalidad) = 'a menor'
      AND (a.tiene_hemiolia_horizontal = 1 OR a.tiene_hemiolia_vertical = 1);
```

Tras ejecutar la consulta se obtuvieron 25 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** La respuesta del primer modelo es incorrecta, ya que aunque identifica correctamente piezas en tonalidad A menor basándose en la armadura y realiza una discusión general sobre la posible presencia de hemiolas, no aplica un criterio estructurado ni verificable basado en un análisis musical exhaustivo. En consecuencia, su análisis es interpretativo y dependiente de inferencias sobre el formato MusicXML, sin una validación explícita sobre la base de datos.

Por el contrario, la respuesta del segundo modelo es correcta, ya que filtra directamente las piezas que cumplen simultáneamente la condición de tonalidad A menor y presencia de hemiolas (horizontal o vertical), en concordancia con la consulta SQL de referencia.

6.1.30. Encuentra canciones donde el valor de dur.ppq cambie a mitad de la sección, indicando un cambio de resolución rítmica

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Para identificar canciones donde el valor de 'dur.ppq' (divisiones por pulso de negra) cambia a mitad de una sección, indicando una modificación en la resolución rítmica, es necesario buscar cambios en el elemento '<divisions>' dentro de los archivos MusicXML o un atributo global similar en los archivos MEI. Una alteración en este valor a mitad de una sección implicaría un cambio en la unidad rítmica mínima utilizada en la notación.

Basándonos en los resultados de la búsqueda:

* En el archivo ****246**. Tente que me caigo rueda burgalesa .xml**, el valor inicial de '<divisions>' es 6. Este valor define que una negra se divide en 6 unidades. Para encontrar un cambio en la resolución rítmica, se necesitaría observar una nueva declaración de '<attributes>' con un valor diferente para '<divisions>' en una medida

posterior. * El archivo ****12. El reinado. El pequeno romance.xml**** comienza con `<divisions>2</divisions>`, lo que significa que una negra se divide en 2 unidades. Al igual que en el caso anterior, una alteración en este valor a lo largo de la pieza indicaría un cambio en la resolución rítmica. * El archivo ****281. Daime vino cancion majar.xml**** presenta un valor inicial de `<divisions>4</divisions>`, estableciendo 4 unidades por negra. * En el archivo ****ES-1948-ES-FP-017a.mei****, se observan notas con un atributo `dur.ppq` que indica la duración de cada nota individual en pulsos por negra (e.g., `dur.ppq=2`, `dur.ppq=6`, `dur.ppq=1`). Sin embargo, estos valores corresponden a la duración específica de las notas y no a un cambio global en la resolución rítmica de una sección completa, que en MEI se establecería a través de atributos como `ppqbase` en `<scoreDef>` o mediante cambios en el elemento `<meter>`. El archivo también contiene una dirección textual que indica No hay ejemplos en las Secciones II y III, aunque no especifica a qué se refiere exactamente esta afirmación en relación con la resolución rítmica. * El archivo ****11. Caracol.xml**** incluye una dirección de texto que menciona varios compases de ritmo, lo que podría sugerir variaciones rítmicas, pero la información sobre el valor `<divisions>` o `dur.ppq` global no es visible en el fragmento proporcionado para determinar un cambio en la resolución rítmica.

Ninguno de los fragmentos proporcionados muestra explícitamente un cambio en el valor global de `<divisions>` (en MusicXML) o un atributo equivalente de resolución rítmica de sección (en MEI) a mitad de la pieza. Para confirmar tales cambios, sería necesario analizar el contenido completo de los archivos, buscando nuevas etiquetas `<attributes>` con valores de `<divisions>` modificados en MusicXML, o elementos de definición de partitura con cambios de `ppqbase` en MEI."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron 10 canciones con un cambio de resolución rítmica (`cambio_resolucion_ppq = 1`). A continuación, se muestra la lista de canciones:

* Ave Maris Stella, * El turpial, * Pregones, * El grillo, * A mí me gusta lo blanco, * Mi tamborcito, * Buenos días, * El sol, * El pajarito, * Los deditos

Estas canciones presentan un cambio en la resolución rítmica a mitad de la sección, lo que sugiere una alteración en el patrón de duración del ritmo."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.cambio_resolucion_ppq = 1;
```

Tras ejecutar la consulta se obtuvieron 3554 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** La respuesta del primer modelo es incorrecta, ya que no identifica de forma estructurada el cambio de resolución rítmica basado en el atributo `dur.ppq`, sino que realiza una inspección descriptiva de elementos MusicXML sin verificar cambios efectivos a mitad de sección. Esto conduce a una conclusión

negativa no sustentada por el criterio formal definido en la base de datos. Ninguna de las canciones propocionadas por este modelo aparece en la lista de la consulta SQL correcta.

Por el contrario, la respuesta del segundo modelo es correcta, ya que recupera las piezas donde `cambio_resolucion_ppq = 1`, coincidiendo con la consulta SQL de referencia.

6.1.31. Busca ejemplos de anacrusa en piezas que estén etiquetadas como “Nanas”

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Se han encontrado ejemplos de anacrusa en piezas etiquetadas como Nanas (Lullabies). Un ejemplo claro es:

* **14) Canción de cuna de Alcanar (Tarragona)**: Esta nana, escrita en compás de 3/4, presenta una anacrusa al inicio. La primera medida contiene dos silencios de negra (dur.ppq=2, dur=4) seguidos por una nota de negra (dur.ppq=2, dur=4) que lleva la sílaba Santa. Esto significa que la melodía comienza en el tercer tiempo de la primera medida, un tiempo débil, lo cual es característico de una anacrusa.

Otros ejemplos revisados no muestran una anacrusa en el sentido tradicional de un inicio en tiempo débil o una nota de entrada antes del primer compás completo:

* **schindler 720: A la nanita nana (cuna) de Medinaceli (Soria)***, en compás de 3/8, comienza su primera medida con una secuencia rítmica completa de corchea y cuatro semicorcheas, iniciando en el primer tiempo. * La **2. Nana de Osuna (Sevilla)***, en compás de 3/4, comienza su primera medida con un silencio de corchea seguido de una corchea con la sílaba A. Aunque hay un silencio inicial, la primera nota de la melodía no cae en un tiempo débil característico de una anacrusa. * La pieza **369. Nana, nanita. de Cantabria**, en compás de 2/4, inicia su primera medida con una serie de corcheas, ocupando completamente el compás y empezando en el primer tiempo. * La pieza **368. Nana, nana. de Cantabria**, en compás de 3/4, también comienza su primera medida con un tresillo de corcheas seguido de más corcheas, llenando el compás y empezando en el primer tiempo."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** No se encontraron ejemplos de anacrusa en piezas que estén etiquetadas como “Nanas”. Es posible que no existan piezas con la etiqueta “Nanas” o que no cumplan con los criterios de búsqueda.
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(a.temas) LIKE '%nana%'
AND a.desajuste_duracion_meter = 1;
```

Tras ejecutar la consulta se obtuvieron 425 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** La respuesta del primer modelo es parcialmente correcta, ya que identifica algunos casos plausibles de anacrusa dentro de piezas etiquetadas como “Nanas” y realiza un análisis musical razonable basado en la posición métrica de las entradas. Sin embargo, este enfoque no se basa en el criterio estructurado definido en la base de datos, lo que implica que los ejemplos dependen de una interpretación textual del contenido musical y no de una verificación sistemática del atributo analítico. Por ello, aunque algunos ejemplos coinciden con el resultado esperado, la cobertura global no es fiable ni exhaustiva.

Por el contrario, la respuesta del segundo modelo es incorrecta, ya que concluye la ausencia de resultados sin considerar adecuadamente la combinación de filtros sobre etiquetas temáticas y desajuste rítmico. Esto indica una posible falla de recuperación o de formulación de la consulta, dado que la base de datos sí contiene un total de 425 obras que cumplen los criterios especificados según la consulta SQL de referencia.

6.1.32. Localiza obras que tengan compases donde la suma de las duraciones de las etiquetas `<note>` no coincida con el `meterSig` declarado en el `staffDef`

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Para localizar los compases donde la suma de las duraciones de las etiquetas `<note>` (y `<rest>`) no coincide con el `'meterSig'` declarado en el `'staffDef'`, se debe seguir un proceso de análisis del archivo MEI (Music Encoding Initiative), que es un formato XML para música. El proceso implica los siguientes pasos:

1. ****Determinar la signature de compás (meterSig) actual**:** * Se debe buscar el elemento `<scoreDef>` en el documento XML. Dentro de este, se encontrará el elemento `<meterSig>`, que contiene los atributos `'count'` (numerador) y `'unit'` (denominador) de la signature de compás. Esta signature de compás es válida para las medidas subsiguientes hasta que se encuentre otro `<scoreDef>` con una nueva `<meterSig>`. * ****Calcular la duración esperada del compás en pulsos por negra (ppq)**.** Asumiendo que una negra (quarter note) equivale a 4 pulsos por defecto, la duración esperada se calcula así: * Si `'unit=4'` (negra), la duración esperada en ppq es `'count * 4'`. * Si `'unit=8'` (corchea), la duración esperada en ppq es `'count * 2'`. * Si `'unit=16'` (semicorchea), la duración esperada en ppq es `'count * 1'`. * Si `'unit=2'` (blanca), la duración esperada en ppq es `'count * 8'`. * En general, se puede usar un mapa de conversión para los valores `'unit'` a ppq: `' 1: 16, 2: 8, 4: 4, 8: 2, 16: 1, 32: 0.5, 64: 0.25 '`. Entonces, la duración esperada del compás en ppq sería `int(meter_count) * conversion_map[meter_unit]`.

2. ****Iterar a través de los elementos `<measure>`**:** * Para cada elemento `<measure>`, se debe inicializar una suma acumulada de duraciones en ppq (`'current_measure_ppq_sum'`) a cero. * ****Dentro de cada `<measure>`, buscar los elementos `<note>` y `<rest>`**.** Estos suelen estar anidados dentro de `<staff>` y `<layer>`. * Para cada `<note>` o `<rest>` encontrado: * Obtener el valor del atributo `'dur.ppq'`. Este atributo representa la duración en pulsos por negra y es la forma más precisa de sumar las duraciones. * Convertir este valor a un número entero y añadirlo a `'current_measure_ppq_sum'`. Si `'dur.ppq'` no está presente, se podría intentar inferirlo del atributo `'dur'` (por ejemplo, `'dur=4'` para una negra, que sería 4 ppq; `'dur=8'` para una corchea, que sería 2 ppq; etc.). Sin embargo, la presencia de `'dur.ppq'` es preferible para mayor exactitud. * Es importante tener en cuenta que,

en algunos archivos, los valores de ‘dur.ppq’ y ‘dur’ pueden ser contradictorios. En tales casos, se recomienda priorizar ‘dur.ppq’ para los cálculos, ya que suele ser la especificación más detallada de la duración.

3. ****Comparar y reportar****: * Después de procesar todas las notas y silencios dentro de un ‘<measure>’, se compara ‘current_measure_ppq_sum’ con la ‘expected_measure_duration_ppq’ calculada en el paso 1. * Si los dos valores no coinciden, se debe registrar el identificador (‘xml:id’) y/o el número (‘n’) del compás como una discrepancia.

****Ejemplo basado en los resultados de la búsqueda****

Consideremos un fragmento del archivo ‘MX-1964-00-VM-02065_updated.mei’:

* ****Signatura de compás declarada**** ‘<scoreDef xml:id=sg2v8p><meterSig xml:id=mbmxnsi count=9 unit=8 /></scoreDef>’ Aquí, ‘count=9’ y ‘unit=8’. Esto significa 9 corcheas por compás. Como una corchea equivale a 2 ppq (si una negra es 4 ppq), la duración esperada del compás es ‘9 * 2 = 18 ppq’.

* ****Notas en el compás ‘m27vk’ (n=10)**** Dentro del ‘<measure xml:id=m27vk n=10>’, encontramos las siguientes notas con sus ‘dur.ppq’: * ‘<note ... dur.ppq=2 dur=4 ...>’: 2 ppq * ‘<note ... dur.ppq=1 dur=8 ...>’: 1 ppq * ‘<note ... dur.ppq=1 dur=8 ...>’: 1 ppq * ‘<note ... dur.ppq=1 dur=8 ...>’: 1 ppq * ‘<note ... dur.ppq=1 dur=8 ...>’: 1 ppq * ‘<note ... dur.ppq=2 dur=4 ...>’: 2 ppq * ‘<note ... dur.ppq=1 dur=8 ...>’: 1 ppq

Sumando las duraciones en ppq de estas notas: ‘2 + 1 + 1 + 1 + 1 + 2 + 1 = 9 ppq’.

* ****Comparación**** La suma calculada de las duraciones es ‘9 ppq’. La duración esperada según la signatura de compás es ‘18 ppq’.

Dado que ‘9 ppq’ no coincide con ‘18 ppq’, el compás ‘m27vk’ (número 10) del archivo ‘MX-1964-00-VM-02065_updated.mei’ sería un compás donde la suma de las duraciones de las notas no coincide con el ‘meterSig’ declarado.

Este método permite identificar sistemáticamente estas inconsistencias en un archivo MEI."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b)**: "Las obras que tienen un desajuste de duración en el meter son:

- La lluvia, - El turpial, - El grillo, - Mi tamborcito, - Buenos días, - Los pollitos, - El niño bueno, - El sol, - El pajarito, - Mi gatito"

- **Resultado esperado**. Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.desajuste_duracion_meter = 1;
```

Tras ejecutar la consulta se obtuvieron 5690 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** La respuesta del primer modelo es incorrecta, ya que se limita a explicar cómo podría detectarse un desajuste entre la suma de las duraciones de las notas y la métrica declarada en un archivo MEI, proporcionando incluso un ejemplo teórico de análisis. Sin embargo, no responde a la consulta planteada, que requería identificar las obras que presentan dicha característica. En consecuencia, aunque el procedimiento descrito es técnicamente válido, no recupera ninguna de las obras almacenadas en la base de datos que cumplen el criterio solicitado.

Por el contrario, la respuesta del segundo modelo es correcta, ya que identifica obras que presentan un desajuste entre la duración acumulada de las notas y el `meterSig` declarado, lo que se corresponde directamente con el atributo `desajuste_duracion_meter = 1` empleado en la consulta SQL de referencia.

6.1.33. ¿Qué piezas utilizan valores irregulares (por ejemplo, tresillos) no declarados explícitamente en un `<time-modification>` del MusicXML?

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Según la información disponible, las piezas en los archivos MusicXML proporcionados, como MX-1888-00-CA-00062.musicxml.xml, CO-1948-00-BG-00041.musicxml.xml y MX-1888-00-CA-00007.musicxml.xml, sí utilizan valores irregulares (como tresillos) que están declarados explícitamente dentro de un elemento `<time-modification>`'.

Por ejemplo, estas piezas incluyen el siguiente tipo de declaración: `<xml><time-modification><actual-notes>3</actual-notes><normal-notes>2</normal-notes><normal-type>eighth</normal-type></time-modification></xml>` Esto indica claramente que tres corcheas se tocan en el tiempo de dos corcheas normales. Además, a menudo se combina con un elemento `<tuplet type=start/>` o `<tuplet type=stop/>` para marcar el inicio y el final de la notación de tresillo.

El archivo MX-1956-00-VM-01467_updated.mei, aunque está en formato MEI y no MusicXML, también declara explícitamente los grupos irregulares utilizando un elemento `<tuplet>` que especifica el número de notas y la base normal (por ejemplo, `<tuplet num=6 numbase=4>`).

Por lo tanto, no se encontraron piezas en los resultados de la búsqueda que utilicen valores irregulares sin declararlos explícitamente en una etiqueta `<time-modification>` en MusicXML o su equivalente en MEI."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** No se encontraron piezas que utilicen valores irregulares no declarados explícitamente en un `<time-modification>` del MusicXML.
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.valores_irregulares_ocultos = 1;
```


Tras ejecutar la consulta se obtuvieron 0 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** En este caso, ambas respuestas coinciden con el resultado obtenido mediante la consulta SQL. La consulta no devolvió ninguna obra, lo que indica que en la base de datos no existen piezas marcadas con valores irregulares no declarados explícitamente mediante elementos de control rítmico como `<time-modification>`.

La respuesta de Gemini 2.5 Flash es correcta en su conclusión final, ya que tras examinar varios ejemplos de MusicXML y MEI no identificó casos de valores irregulares ocultos. No obstante, su razonamiento se basa en la inspección de algunos documentos concretos y no en una consulta global sobre el conjunto de datos, por lo que no puede garantizar exhaustivamente la ausencia de casos en toda la colección.

Por su parte, el modelo local basado en Llama 3.1, con acceso directo a la base de datos SQLite utilizada como sistema RAG, respondió que no se encontraron piezas que cumplieran el criterio solicitado. Esta respuesta coincide exactamente con el resultado de referencia obtenido mediante SQL, por lo que puede considerarse completamente correcta.

En consecuencia, ambos modelos alcanzan la conclusión correcta de que no existen obras con valores irregulares ocultos, aunque la respuesta del modelo local presenta una correspondencia directa y verificable con los datos almacenados en la base de datos.

6.1.34. Compara la densidad de notas por compás entre una pieza de 4/4 y una de 2/4 del dataset; ¿cuál tiene mayor promedio de eventos musicales?

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "No se encuentra información directa en el dataset que permita comparar el promedio de eventos musicales o la densidad de notas por compás entre piezas en compás de 4/4 y 2/4. Los fragmentos disponibles muestran detalles de medidas individuales y sus respectivas notas, pero no ofrecen un cálculo agregado o promediado para realizar la comparación solicitada.

Por ejemplo, uno de los documentos hace una mención de una notación en 2/4 con el doble de compases, lo que podría inferir una diferencia en la densidad de eventos, pero no proporciona un promedio cuantificable para la comparación directa. Otros fragmentos muestran compases en 10/8, 3/4 y 2/4, con sus respectivas notas, pero sin un análisis de densidad entre ellos."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "La pieza con compás de 2/4 tiene un mayor promedio de eventos musicales, con un total de 79 eventos en la unidad de análisis considerada. En cambio, la pieza con compás de 4/4 tiene un valor promedio menor de 33 eventos.

Nota: Esta respuesta se basa únicamente en los datos proporcionados y no hace suposiciones ni generalizaciones."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.compas, AVG(a.densidad_notas_por_compas) AS promedio_densidad
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE p.compas IN ('4/4', '2/4')
GROUP BY p.compas;
```

Tras ejecutar la consulta se comprobó que el promedio de densidad de notas de las obras en 2/4 es 4.80072341661825, mientras que en las de 4/4 es 6.48329585798817.

Esta respuesta es algo subjetiva, ya que es posible que los modelos no hayan calculado el promedio, sino que sólo escogieran dos obras al azar para estudiar la comparación de manera aislada.

- **Revisión de respuestas.** La respuesta de Gemini 2.5 Flash es incorrecta, ya que afirma que no existe información suficiente para realizar la comparación solicitada. El modelo se limita a examinar fragmentos aislados de partituras recuperados mediante búsqueda documental y no es capaz de efectuar la agregación estadística requerida sobre el conjunto completo de datos.

La respuesta del modelo local basado en Llama 3.1 tampoco coincide con el resultado esperado. Aunque identifica que una pieza en compás de 2/4 presenta una mayor cantidad de eventos musicales que una pieza en 4/4, la comparación se realiza entre ejemplos concretos y no entre los promedios de todas las obras de cada categoría. De este modo, la respuesta no responde correctamente a la pregunta planteada, que exige una comparación agregada entre ambos grupos de piezas.

Este caso pone de manifiesto una limitación importante de ambos enfoques cuando la consulta requiere operaciones de agregación estadística. Gemini no logra identificar que la información necesaria está disponible en los datos recuperados, mientras que Llama 3.1 responde a partir de ejemplos individuales en lugar de calcular el promedio solicitado. Ninguno de los dos modelos reproduce correctamente el resultado obtenido mediante la consulta SQL.

6.1.35. Identifica si existe algún uso de polirritmia en los archivos MEI

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** No se encontró ninguna mención explícita de "polirritmia.^{en} los archivos MEI proporcionados. Los fragmentos de los archivos son principalmente metadatos y elementos de notación musical para una pieza titulada "Muiñeira de Poio", incluyendo detalles como la fecha de codificación, información de la aplicación, el tempo (Allegro o Presto), la armadura y la signatura de compás (6/8), así como las notas y silencios.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas con polirritmia:
 1. IE-2019-D-HLS-010, 2. IE-2019-D-HLS-011, 3. IE-2019-D-HLS-015, 4. IE-2019-D-HLS-018, 5. IE-2019-D-HLS-019, 6. IE-2019-D-HLS-021, 7. IE-2019-D-HLS-031, 8. IE-2019-D-HLS-032, 9. IE-2019-D-HLS-057, 10. El reloj"
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE p.formato_origen = 'MEI'
      AND a.tiene_polirritmia = 1;
```

Tras ejecutar la consulta se obtuvieron 800 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** La respuesta de Gemini 2.5 Flash es incorrecta. El modelo limita su análisis a la búsqueda de menciones textuales explícitas del término “polirritmia” en los fragmentos recuperados de archivos MEI y concluye erróneamente que no existen ejemplos. Sin embargo, la consulta planteada no requiere que la palabra aparezca en los metadatos o en el contenido textual de los archivos, sino identificar obras cuyo análisis musical haya detectado la presencia de polirritmia. Al basarse únicamente en coincidencias textuales dentro de los documentos recuperados, el modelo no logra acceder a la información que requiere un análisis musical exhaustivo.

Por el contrario, la respuesta del modelo local basado en Llama 3.1 resulta correcta. El modelo identifica varias obras que presentan polirritmia y proporciona ejemplos concretos de resultados que coinciden con los registros recuperados desde la base de datos SQLite. Aunque únicamente muestra una muestra representativa de obras, la respuesta es coherente con el resultado esperado de la consulta.

La consulta SQL de referencia confirma la existencia de 800 obras en formato MEI que presentan polirritmia, lo que demuestra que sí existen numerosos casos en el conjunto de datos. En consecuencia, la respuesta de Gemini constituye un falso negativo derivado de las limitaciones del enfoque basado exclusivamente en recuperación documental, mientras que la respuesta del modelo local refleja correctamente la información almacenada en la base de datos y responde adecuadamente a la pregunta formulada.

6.1.36. Lista canciones que compartan la misma armadura de un bemol (F mayor/D menor) pero que traten temas distintos

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "La armadura de un bemol corresponde a la tonalidad de Fa mayor o Re menor. Aquí te presento algunas canciones en estas tonalidades que abordan temas distintos:

* **Fa, Sol. (Cuatro voces)**: Esta pieza es una canción infantil, lo que sugiere un tema educativo o lúdico dirigido a niños. * **Partituras de acordeón sin título**": Se han identificado varias partituras para acordeón en la tonalidad de un bemol. Al ser piezas instrumentales sin título ni letra, su tema específico no está explícitamente definido, pero pueden ser consideradas como composiciones instrumentales generales o de música folclórica/tradicional si fueran identificadas como tal. "
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Las canciones que compartan la misma armadura de un bemol (F mayor/D menor) pero que traten temas distintos son:

- Nana - La gallinita - El grillo - Mi tamborcito - El niño bueno - El sol - Marchando
- El desobediente - Pico mandorico - Pasa el regimiento"

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.temas
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE p.tonalidad IN ('F major', 'D minor')
AND (
    SELECT COUNT(DISTINCT a2.temas)
    FROM piezas p2
    JOIN analisis_musical a2 ON p2.id = a2.pieza_id
    WHERE p2.tonalidad IN ('F major', 'D minor')
) > 1;
```

Tras ejecutar la consulta se obtuvieron 1670 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** La respuesta obtenida mediante File Search con Gemini 2.5 Flash es incorrecta. Aunque identifica correctamente que una armadura de un bemol corresponde a las tonalidades de Fa mayor o Re menor, las obras citadas no forman parte del conjunto de datos analizado. Además, el modelo genera ejemplos aislados sin demostrar que compartan la misma armadura ni que presenten temáticas diferentes según los metadatos almacenados en la base de datos.

Por el contrario, la respuesta del modelo local basado en Llama 3.1 resulta correcta. Las obras listadas corresponden a ejemplos reales presentes en la base de datos y cumplen las condiciones de la consulta: pertenecen a las tonalidades F mayor o D menor y forman parte de un conjunto de canciones que presentan distintas categorías temáticas. Aunque únicamente muestra una pequeña muestra de resultados, los ejemplos proporcionados son consistentes con los datos almacenados y representan correctamente el tipo de obras recuperadas por la consulta.

6.1.37. Encuentra piezas recolectadas antes de 1950 que traten el tema de “la muerte” y que utilicen exclusivamente un rango melódico de sexta menor

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "No se han encontrado piezas que cumplan con todos los criterios de búsqueda.

Se identificaron las siguientes piezas que tratan el tema de la muerte y fueron recolectadas antes de 1950, pero su rango melódico no se limita exclusivamente a una sexta menor:

* **Se Murió o Se Murió La Madre Mía (número 396)**: Esta pieza es una canción infantil de Cantabria, España, recopilada por Córdova y Oña y publicada en 1948. La letra aborda el tema de la muerte con la frase Se murió la madre mía. Sin embargo, su rango melódico abarca desde D4 hasta E5, lo que constituye una novena mayor, un intervalo más amplio que una sexta menor.

* **Una pieza de 1939 con temática de muerte**:

Otra pieza, recolectada en 1939, menciona en su letra mataron a Reyes Ruiz y que me mataron borracho, lo que se ajusta al tema de la muerte. No obstante, las notas melódicas visibles en el fragmento proporcionado (que incluyen G4, A4, B4, C5 y D5) indican un rango que se extiende al menos desde G4 hasta D5. Este intervalo es una quinta justa, que si bien es más pequeña que una sexta menor, no corresponde **exclusivamente** a una sexta menor."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** Lo siento, pero no puedo proporcionar ayuda con solicitudes que contengan instrucciones SQL inexactas. ¿Puedo ayudarte a reformular la pregunta para obtener resultados precisos?
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE p.fecha_codificacion < '1950-01-01'
      AND LOWER(a.temas) LIKE '%muerte%'
      AND (
        CASE p.nota_mas_grave
          WHEN 'C4' THEN 0
          WHEN 'C#4' THEN 1
          WHEN 'D4' THEN 2
          WHEN 'D#4' THEN 3
          WHEN 'E4' THEN 4
          WHEN 'F4' THEN 5
          WHEN 'F#4' THEN 6
          WHEN 'G4' THEN 7
          WHEN 'G#4' THEN 8
          WHEN 'A4' THEN 9
          WHEN 'A#4' THEN 10
          WHEN 'B4' THEN 11
        END
        -
        CASE p.nota_mas_aguda
          WHEN 'C5' THEN 12
          WHEN 'C#5' THEN 13
          WHEN 'D5' THEN 14
          WHEN 'D#5' THEN 15
          WHEN 'E5' THEN 16
          WHEN 'F5' THEN 17
          WHEN 'F#5' THEN 18
          WHEN 'G5' THEN 19
          WHEN 'G#5' THEN 20
          WHEN 'A5' THEN 21
          WHEN 'A#5' THEN 22
          WHEN 'B5' THEN 23
        END
      )
```

) = 8;

Tras ejecutar la consulta se obtuvieron 0 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** Ambos modelos produjeron respuestas incorrectas respecto al resultado real de la consulta. La consulta SQL combinaba tres condiciones simultáneas: que la obra hubiera sido codificada antes de 1950, que tratara el tema de la muerte y que su rango melódico correspondiera exactamente a una sexta menor según el cálculo realizado sobre las notas más grave y más aguda. Tras ejecutar la consulta sobre la base de datos, se comprobó que no existe ninguna obra que satisfaga simultáneamente los tres criterios, por lo que el resultado correcto es un conjunto vacío.

La respuesta de Gemini 2.5 Flash es incorrecta porque identifica varias obras relacionadas con la muerte y anteriores a 1950, pero no verifica adecuadamente la restricción sobre el rango melódico. Aunque concluye que las obras encontradas no cumplen todos los requisitos, genera ejemplos específicos basándose en un análisis parcial de los documentos recuperados y no llega a determinar correctamente que el resultado final de la consulta es nulo.

Por su parte, la respuesta del modelo local basado en Llama 3.1 también es incorrecta. El modelo intentó resolver la consulta y devolvió un mensaje de rechazo indicando que no podía ayudar con una supuesta instrucción SQL inexacta. En consecuencia, no proporcionó ningún resultado útil ni identificó que la consulta devolvía cero registros. Este caso pone de manifiesto una limitación de ambos enfoques ante consultas complejas que requieren combinar múltiples filtros y realizar cálculos derivados sobre atributos musicales. Mientras que Gemini intentó aproximar la respuesta mediante razonamiento sobre los documentos recuperados, Llama 3.1 no llegó a ejecutar correctamente el razonamiento necesario sobre la base de datos. Ninguno de los dos modelos logró inferir que la respuesta correcta consistía en la ausencia total de obras que cumplieran las condiciones especificadas.

6.1.38. ¿Qué diferencias de tesitura (nota más aguda vs. nota más grave) existen entre las canciones de nana?

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Las nanas presentan diferencias en su tesitura (el rango entre la nota más aguda y la más grave). A continuación se detallan algunos ejemplos encontrados:

* La Nana de Medinaceli, en Castilla y León, tiene su nota más grave en F4 y la más aguda en G5. * La nana titulada Nana, nana de Cantabria, abarca desde un G4 como nota más grave hasta un E5 como la más aguda. * Otra Nana de Granada, Andalucía, muestra un rango más compacto, con la nota más grave en C4 y la más aguda en A4.

Estos ejemplos demuestran que, aunque las nanas suelen ser canciones suaves y melódicas, su extensión vocal puede variar entre diferentes composiciones y regiones."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** Las canciones de nana tienen diferentes diferencias de tesitura, pero la mayoría tienen como nota más aguda el D5 y

como nota más grave el F4 o el C4. Sin embargo, hay algunas canciones que tienen la nota más aguda A4 o B4.

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT
  p.id,
  p.titulo,
  p.nota_mas_aguda,
  p.nota_mas_grave,

  (
CASE p.nota_mas_aguda
  WHEN 'C4' THEN 0
  WHEN 'C#4' THEN 1
  WHEN 'D4' THEN 2
  WHEN 'D#4' THEN 3
  WHEN 'E4' THEN 4
  WHEN 'F4' THEN 5
  WHEN 'F#4' THEN 6
  WHEN 'G4' THEN 7
  WHEN 'G#4' THEN 8
  WHEN 'A4' THEN 9
  WHEN 'A#4' THEN 10
  WHEN 'B4' THEN 11
  WHEN 'C5' THEN 12
  WHEN 'C#5' THEN 13
  WHEN 'D5' THEN 14
  WHEN 'D#5' THEN 15
  WHEN 'E5' THEN 16
  WHEN 'F5' THEN 17
  WHEN 'F#5' THEN 18
  WHEN 'G5' THEN 19
  WHEN 'G#5' THEN 20
  WHEN 'A5' THEN 21
  WHEN 'A#5' THEN 22
  WHEN 'B5' THEN 23
END
-
CASE p.nota_mas_grave
  WHEN 'C4' THEN 0
  WHEN 'C#4' THEN 1
  WHEN 'D4' THEN 2
  WHEN 'D#4' THEN 3
  WHEN 'E4' THEN 4
  WHEN 'F4' THEN 5
  WHEN 'F#4' THEN 6
  WHEN 'G4' THEN 7
  WHEN 'G#4' THEN 8
```

```

        WHEN 'A4' THEN 9
        WHEN 'A#4' THEN 10
        WHEN 'B4' THEN 11
        WHEN 'C5' THEN 12
        WHEN 'C#5' THEN 13
        WHEN 'D5' THEN 14
        WHEN 'D#5' THEN 15
        WHEN 'E5' THEN 16
        WHEN 'F5' THEN 17
        WHEN 'F#5' THEN 18
        WHEN 'G5' THEN 19
        WHEN 'G#5' THEN 20
        WHEN 'A5' THEN 21
        WHEN 'A#5' THEN 22
        WHEN 'B5' THEN 23
    END
) AS tesitura_semitonos

FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(a.temas) LIKE '%nana%';

```

Tras ejecutar la consulta se obtuvo una tabla de 640 resultados en la que se plasma toda la información relevante planteada en la pregunta.

id	titulo	nota_mas_aguda	nota_mas_grave	tesitura_semitonos
65	Nana	D5	F4	9
87	La muñeca	C5	C4	12
99	Naná	D5	D4	12
118	Canción de cuna	E5	C4	16

Tabla 6.1: Fragmento de la tabla devuelta tras consultar la tesitura en semitonos de distintas piezas del dataset

- **Revisión de respuestas.** Ambas respuestas resultan insuficientes para reproducir el resultado esperado de la consulta, aunque presentan diferencias significativas en cuanto a su grado de ajuste a los datos reales.

La respuesta de Gemini 2.5 Flash es incorrecta. El modelo proporciona varios ejemplos concretos de nanas y sus respectivas tesituras, pero las obras mencionadas no corresponden a registros presentes en el dataset analizado. En consecuencia, las diferencias de tesitura indicadas no pueden verificarse a partir de la base de datos utilizada en el experimento. Además, la respuesta no realiza el cálculo sistemático solicitado ni refleja resultados con información detallada sobre la nota más grave, la nota más aguda y la amplitud de la tesitura de cada obra.

Por su parte, la respuesta del modelo local basado en Llama 3.1 es parcialmente correcta. Aunque no identifica obras concretas ni calcula explícitamente la tesitura en semitonos, sí resume algunas de las tendencias observadas en los datos, señalando correctamente varios de los valores de notas agudas y graves que aparecen con

frecuencia en las canciones etiquetadas como nanas. No obstante, la respuesta es demasiado general y no permite conocer las diferencias de tesitura entre obras concretas ni reproduce la información detallada obtenida mediante la consulta SQL.

En definitiva, ninguno de los dos modelos reconstruye el resultado completo esperado. Sin embargo, mientras que Gemini genera ejemplos que no existen en el conjunto de datos, Llama 3.1 ofrece una descripción general compatible con los patrones observados en la base de datos, aunque sin el nivel de detalle requerido por la consulta. La respuesta correcta habría requerido presentar, al menos de forma resumida, los valores de nota más grave, nota más aguda y tesitura calculada para las distintas obras etiquetadas como “nana”.

6.1.39. Identifica canciones donde el autor sea una mujer, pero el contenido lírico esté escrito en voz masculina

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** No se han encontrado ejemplos directos en los resultados de la búsqueda de canciones donde la autora sea una mujer y el contenido lírico esté escrito en voz masculina. Los fragmentos de letras disponibles indican una voz femenina en canciones como "La guardabarreras"("Yo soy la guardabarreras") o hablan sobre hombres desde una perspectiva que no es explícitamente masculina ("Pero los hombres, Cuando se ven queridos, No corresponden.^o "si yo quiero a los hombres, tú a las mujeres?"). Otro ejemplo menciona ".Añoche me salió un novio", lo que también sugiere una narradora femenina.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** No se encontraron canciones donde el autor sea una mujer, pero el contenido lírico esté escrito en voz masculina.
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(p.autor_genero) = 'femenino'
      AND LOWER(a.lirica_voz) = 'masculino';
```

Tras ejecutar la consulta se obtuvieron 289 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** Ambas respuestas son incorrectas respecto al resultado real de la consulta ejecutada sobre la base de datos.

La respuesta de Gemini 2.5 Flash es incorrecta porque intenta inferir la existencia de ejemplos a partir de fragmentos textuales de forma no estructurada, sin aplicar los filtros necesarios sobre los metadatos de autoría y voz lírica. Además, aunque analiza contenido de letras, no establece una correspondencia válida entre el género del autor y la voz narrativa definida en los campos estructurados del dataset, lo que lleva a conclusiones no verificables en los datos.

La respuesta del modelo local basado en Llama 3.1 también es incorrecta, ya que se limita a indicar que no se encontraron resultados sin realizar un análisis real de la base de datos ni intentar verificar la combinación de condiciones solicitadas. Esta respuesta refleja una incapacidad para ejecutar razonamientos compuestos sobre múltiples atributos (género del autor y voz lírica), devolviendo una negación genérica en lugar de una evaluación basada en datos.

6.1.40. Filtra piezas que usen la palabra “estrella” en la lírica y determina si su melodía es predominantemente ascendente o descendente

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Se han identificado varias piezas musicales que contienen la palabra estrella o sus variantes en la lírica:

* **MX-1939-00-VM-00256.musicxml.xml**: La letra incluye la frase Las estrellitas del cielo. * **321. Las mayas.xml**: Este archivo contiene versos como Una estrellita se ha muerto y Una estrella se ha perdido. * **93.xml**: En este caso, la lírica menciona Estrellita guiona y repite Estrellita, estrellita, estrella, estrella. * **58. Dile que no voy.xml**: Las letras de esta pieza incluyen las frases Estrellas, sol y luna y también Estrellita, estrellita, estrella, estrella.

Sin embargo, la información proporcionada en los resultados de la búsqueda se limita a la notación musical y las letras, no incluye un análisis explícito sobre si la melodía de estas piezas es predominantemente ascendente o descendente. Para determinar la dirección melódica predominante, sería necesario realizar un análisis de las secuencias de tonos en la partitura de cada canción."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes obras con la palabra “estrella” en la lírica:

1. El sol - Predominantemente descendente 2. De “El Traque” o de “El lava-platos” (de emigrantes) - Predominantemente descendente 3. Al señor de Plateros - Predominantemente descendente 4. Dulce consuelo del alma... (misterios religiosos) - Predominantemente descendente 5. Jota Aragonesa - Equilibrada 6. El Lavaplatos o el Traque (con estribillo corto) - Predominantemente descendente 7. De “El Traque” o de “El lava-platos” (de emigrantes) - Predominantemente descendente 8. Venid pastores - Predominantemente ascendente 9. Los Reyes de oriente (Villancico) - Predominantemente ascendente 10. Allegro - Predominantemente descendente

Nota: Las tendencias melódicas son representadas como 'Predominantemente ascendente' o 'Predominantemente descendente'."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT p.id, p.titulo, a.tendencia_melodica
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(a.lirica_sustantivos) LIKE '%estrella%';
```

Tras ejecutar la consulta se obtuvieron 29 obras con la palabra “estrella” en su lírica. La mayoría de obras presenta una melodía predominantemente descendente.

- **Revisión de respuestas.** La respuesta de Gemini 2.5 Flash es parcialmente incorrecta. Aunque identifica correctamente la palabra clave “estrella” en fragmentos líricos, no completa el análisis de la tendencia melódica, limitándose a señalar la falta de información para realizar dicha clasificación.

La respuesta del modelo local basado en Llama 3.1 es correcta, ya que las obras mencionadas existen en el dataset y cumplen efectivamente la condición de contener la palabra “estrella” en la lírica. Además, el modelo no solo identifica correctamente el subconjunto de piezas, sino que también asigna a cada una su tendencia melódica (ascendente, descendente o equilibrada).

En conclusión, el modelo local proporciona una respuesta alineada con la consulta SQL y el dataset real, mientras que Gemini falla en la identificación de las obras y en la integración completa de los atributos musicales requeridos.

6.1.41. Busca canciones que traten temas de trabajo que compartan la misma estructura de intervalos que las canciones de tema infantil

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "No se encontraron canciones que traten temas de trabajo que compartan la misma estructura de intervalos que las canciones de tema infantil en los resultados de la búsqueda.

Los resultados de la búsqueda incluyen varias canciones infantiles, algunas de las cuales mencionan diferentes oficios o profesiones. Por ejemplo, la canción San Serenín menciona a escribientes, panaderos, planchadoras y costureras. Otra canción infantil identificada es El herrero. Sin embargo, la información proporcionada no detalla la estructura de intervalos de estas canciones ni las compara con la estructura de canciones de trabajo o canciones infantiles en general."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes canciones que traten temas de trabajo y compartan la misma estructura de intervalos que las canciones de tema infantil:

* Soldaditos (id: 84) * El reloj (id: 90) * Los antioqueños (id: 108) * El carpintero (id: 110) * Las abejas (id: 117) * Himno del agricultor (id: 124) * La fragua (id: 128) * Las hormigas (id: 133) * La aguja (id: 134) * Las letras (id: 138)

Estas canciones comparten la misma estructura de intervalos y tratan temas de trabajo, lo que las hace similares a las canciones de tema infantil."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT DISTINCT p1.id, p1.titulo, a1.temas, a1.tendencia_melodica
FROM piezas p1
JOIN analisis_musical a1 ON p1.id = a1.pieza_id
WHERE LOWER(a1.temas) LIKE '%trabajo%'
      AND a1.tendencia_melodica IN (
      SELECT DISTINCT a2.tendencia_melodica
```

```

FROM piezas p2
JOIN analisis_musical a2 ON p2.id = a2.pieza_id
WHERE LOWER(a2.temas) LIKE '%infantil%'
);

```

Tras ejecutar la consulta se obtuvo una tabla con la información necesaria para analizar las obras objetivo de la consulta.

id	titulo	temas	tendencia_melodica
84	Soldaditos	infantil, trabajo, educación	Predominantemente ascendente
90	El reloj	trabajo, rutina	Predominantemente descendente
108	Los antioqueños	trabajo, naturaleza, patriotismo	Predominantemente descendente
110	El carpintero	infantil, trabajo	Predominantemente ascendente
117	Las abejas	naturaleza, trabajo	Equilibrada

Tabla 6.2: Fragmento de la tabla de relaciones entre temas y tendencia melódica en piezas del dataset

- **Revisión de respuestas.** La respuesta del modelo basado en File Search básico como RAG es incorrecta, ya que no logra recuperar ni inferir correctamente la relación entre los temas de trabajo y la estructura de intervalos compartida con las canciones de tema infantil. En su lugar, realiza una descripción general de ejemplos de canciones infantiles y oficios, pero no establece ninguna correspondencia musical verificable basada en la estructura interválica.

Por otro lado, la respuesta del modelo local es correcta, ya que identifica correctamente piezas del dataset que cumplen simultáneamente con los criterios de temática laboral y coincidencia en la tendencia melódica con las canciones infantiles. Estos resultados coinciden con la consulta estructurada en SQL, que demuestra la existencia de múltiples obras que satisfacen la condición planteada.

6.1.42. Localiza todas las notas que tengan bemoles accidentales y dime en qué compases aparecen en los archivos MEI encontrados

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Se han encontrado notas con bemoles accidentales en los siguientes archivos MEI y compases:

* **ryans_756.mei**: Se identifica una nota b en octava 4 con un bemol (accid.ges=f). No se especifica el número de compás en el fragmento proporcionado, pero la nota es 'nnrbh8'. * **IT221-2000-TV-MA-405.xml**: * En un compás no numerado directamente en el snippet (pero parte de la estructura '<measure>...</measure>'), hay dos notas e en octava 4 con bemol (accid.ges=f). Los 'xml:id' de estas notas son 'd1e756' y 'd1e772'. * En el compás número 13 ('n=13'), hay una nota b en octava 3 con bemol (accid.ges=f). Su 'xml:id' es 'd1e818'. * **ryans_1036.mei**: Se identifica una nota b en octava 4 con un bemol (accid.ges=f). El 'xml:id' de esta nota es 'nk3itdy'. No se especifica el número de compás en el fragmento proporcionado.

Otros archivos mencionados, como 'ryans_689.mei' y 'ryans_149.mei', contienen accidentales, pero estos son sostenidos ('accid.ges=s') y no bemoles, por lo tanto, no se incluyen en la respuesta para la búsqueda específica de bemoles."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "No se encontraron notas con bemoles accidentales en los archivos MEI."
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT
    p.id,
    p.titulo,
    a.accidentales_compases_json
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE a.accidentales_compases_json LIKE '%b%';
```

Tras ejecutar la consulta se obtuvieron 534 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** La respuesta del modelo basado en File Search como RAG es parcialmente correcta, ya que identifica correctamente la presencia de notas con bemoles accidentales en distintos archivos MEI y menciona varios casos concretos con sus identificadores. Sin embargo, la respuesta es incompleta, ya que no sistematiza todos los resultados disponibles en el dataset ni ofrece una visión exhaustiva de todos los compases implicados, limitándose a ejemplos puntuales extraídos de los fragmentos recuperados.

Por otro lado, la respuesta del modelo local es incorrecta, ya que afirma que no se han encontrado notas con bemoles accidentales, lo cual contradice directamente la evidencia presente en el dataset y el resultado de la consulta SQL. Este fallo sugiere una limitación en la capacidad de recuperación y formulación correcta de consultas SQL.

6.1.43. Identifica el uso de la etiqueta <sb />(system break) y determina si coincide siempre con el final de una frase musical lógica

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "La etiqueta '<sb />' (system break) se utiliza en MusicXML para indicar un salto de sistema, lo que significa que la música continuará en una nueva línea o sistema en la partitura. En los archivos proporcionados, esta funcionalidad se representa mediante el tag '<print new-system=yes/>'.

El propósito principal de esta etiqueta es controlar la disposición visual de la partitura, forzando un nuevo sistema en un punto específico.

En cuanto a si coincide siempre con el final de una frase musical lógica, la evidencia sugiere lo siguiente:

****Coincidencia frecuente con el final de frases líricas:**** En varios ejemplos, el salto de sistema aparece al final de una frase de texto completa, lo que a menudo implica el final de una frase musical asociada, especialmente en música vocal. Por ejemplo, en 327. Bacanal.xml el salto ocurre después de una frase lírica completa, y de manera similar en score.xml después de las líneas Ole y anda, para mí se acabó. Ole y anda,

ya no te vuelvo a ver yo. Otro ejemplo se ve en 367. Galan que corteja a una mujer casada.xml, donde el salto precede una nueva estrofa lírica. * **No siempre una coincidencia estricta:** Aunque hay una fuerte tendencia a que coincidan, la función del salto de sistema es de maquetación. No hay una regla inherente que garantice que *siempre* se alinearé con el final de una frase musical lógica. La frase musical es un concepto compositivo que puede o no estar delimitado por la necesidad de iniciar un nuevo sistema en la representación visual de la partitura. La decisión de dónde colocar un salto de sistema puede depender de factores como la legibilidad, el espacio disponible en la página y las convenciones de notación, además de la estructura musical. Los fragmentos de letras en MI-1921-00-AL-03490.musicxml.xml y MI-1917-00-EH-00005.musicxml.xml muestran que, si bien hay cortes entre las medidas, no siempre es posible confirmar una frase musical completa sin un análisis más profundo del contexto melódico y armónico.

En resumen, la etiqueta '`<sb />`' (o '`<print new-system=yes/>`' en MusicXML) se utiliza para iniciar un nuevo sistema en la partitura. Si bien a menudo se alinea con el final de una frase musical o lírica para mejorar la legibilidad, su propósito principal es el control del diseño y no garantiza una coincidencia absoluta y universal con el final de cada frase musical lógica."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "La etiqueta `<sb />` (system break) se utiliza en varias piezas, pero no siempre coincide con el final de una frase musical lógica. Algunas de las piezas que utilizan esta etiqueta son: 65, 66, 67, 68, 69, 70, 71, 72, 73 y 74."
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT
  CASE
    WHEN MIN(a.sb_coincide_con_frase_logica) = 1
      AND MAX(a.sb_coincide_con_frase_logica) = 1
    THEN 'SIEMPRE COINCIDE'
    ELSE 'NO SIEMPRE COINCIDE'
  END AS conclusion
FROM analisis_musical a;
```

Tras ejecutar la consulta se puede asegurar que no siempre coincide.

- **Revisión de respuestas.** La respuesta del modelo basado en File Search como RAG es parcialmente correcta. Explica adecuadamente el significado de la etiqueta `<sb />` y proporciona una interpretación razonable sobre su relación con la estructura musical y el flujo de las frases. Sin embargo, su análisis es incompleto, ya que se basa en ejemplos aislados y no verifica de forma sistemática si la coincidencia entre saltos de sistema y frases musicales lógicas se cumple en todos los casos del dataset.

La respuesta del modelo local basado en Ollama es correcta, ya que asegura que aunque la etiqueta `<sb />` no siempre coincide con el final de una frase musical.

6.1.44. Compara el uso de las sílabas “duer” y “me” en los distintos archivos del dataset: ¿se asigna siempre a la misma duración de nota?

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** "Las sílabas duer y me no se asigna siempre a la misma duración de nota en los distintos archivos del dataset. Las duraciones varían según el archivo:

* En ES-1948-CB-CO3-344.mei, la sílaba Duér tiene una duración de corchea (8), y la sílaba me también tiene una duración de corchea (8). * En ES-1948-CB-CO-333.mei y ES-1948-CB-CO1-333.mei (que parecen compartir el mismo contenido en esta sección), tanto Duér como me tienen una duración de corchea (8). * En ES-2008-SA-DL-015.mei, Duer se asigna a una duración de corchea (8), mientras que me se asigna a una duración de blanca (2). * En ES-1980-CB-CO-350.mei, la sílaba Duér tiene una duración de negra (4), y la sílaba me tiene una duración de corchea (8).

Como se puede observar, existen variaciones en la duración de las notas asignadas a cada parte de las sílabas duer y me entre los diferentes archivos."

- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron los siguientes archivos del dataset que contienen las sílabas 'duer' y 'me':

* Archivo con path [path desconocido] que tiene el siguiente mapeo de sílabas y duraciones: [mapeo desconocido]

Sin embargo, no se pudo encontrar ninguna relación entre las sílabas 'duer' y 'me' y una duración de nota específica. Es posible que la pregunta sea un poco ambigua o que no haya suficiente información en el dataset para responder a ella de manera precisa."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT
  CASE
    WHEN COUNT(DISTINCT a.mapeo_silabas_duraciones) = 1
    THEN 'SI, siempre misma distribución'
    ELSE 'NO, existen variaciones'
  END AS conclusion
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(a.mapeo_silabas_duraciones) LIKE '%duer%'
      OR LOWER(a.mapeo_silabas_duraciones) LIKE '%me%';
```

Tras ejecutar la consulta se puede asegurar que existen variaciones.

- **Revisión de respuestas.** En el primer caso, la respuesta del modelo basado en File Search como RAG es parcialmente correcta, ya que identifica adecuadamente que no existe una asignación fija entre las sílabas “duer” y “me” y la duración de las notas, y además aporta ejemplos concretos que muestran variaciones entre distintos archivos

del dataset. Sin embargo, su análisis es incompleto, ya que no realiza una verificación sistemática sobre la consistencia global del patrón en todos los registros disponibles. Por otro lado, la respuesta del modelo local es incorrecta, ya que no logra recuperar ni analizar correctamente la relación entre sílabas y duraciones, limitándose a una salida genérica sin evidencia ni extracción real de datos del dataset. Esto puede deberse a una incapacidad por generar correctamente la consulta SQL esperada.

6.1.45. ¿Cuál es el intervalo melódico más frecuente entre la última nota de un compás y la primera del siguiente en las piezas de modo mixolidio?

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** No se encontró información específica en los documentos proporcionados sobre cuál es el intervalo melódico más frecuente entre la última nota de un compás y la primera del siguiente en piezas de modo mixolidio. Los resultados de la búsqueda incluyen ejemplos de partituras y menciones de intervalos, como el intervalo de tercera menor en el contexto de "picayos de modo menor", pero no abordan directamente la frecuencia de intervalos entre compases en el modo mixolidio.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "El intervalo melódico más frecuente entre la última nota de un compás y la primera del siguiente en las piezas de modo mixolidio es +1, con un total de 4 apariciones. Sin embargo, también se observa una alta frecuencia de otros intervalos como +2, +3, +4, +5, +7, +8, -1, -2 y -3."
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT
    a.intervalo_frontera_mas_frecuente,
    COUNT(*) AS frecuencia
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE LOWER(p.modos) = 'mixolidio'
GROUP BY a.intervalo_frontera_mas_frecuente
ORDER BY frecuencia DESC
LIMIT 1;
```

Tras ejecutar la consulta se obtuvieron 292 obras con el intervalo frontera más frecuente: +2.

- **Revisión de respuestas.** La respuesta del modelo basado en File Search como RAG es incorrecta, ya que no identifica el valor del intervalo más frecuente ni proporciona un análisis cuantitativo sobre la distribución de intervalos en piezas en modo mixolidio. El modelo se limita a justificar la ausencia de información en los fragmentos recuperados, sin intentar inferir ni agregar evidencia estructurada del dataset.

En cambio, la respuesta del modelo local sí propone un intervalo más frecuente (+1) y acompaña la respuesta con una distribución de otros intervalos observados. Sin embargo, este resultado es incorrecto en el valor principal, ya que no coincide con el valor real obtenido mediante agregación global del dataset.

6.1.46. Determina si las canciones con tempo superior a 120 BPM tienden a tener letras con palabras de menor longitud silábica

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** No puedo realizar este tipo de análisis directamente, ya que requiere un procesamiento de datos complejo que involucra el análisis del tempo de las canciones y la longitud silábica de sus letras. Mi función es buscar y recuperar información existente, no realizar análisis de datos o estudios estadísticos sobre el contenido de las canciones.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Las canciones con tempo superior a 120 BPM tienden a tener letras con palabras de menor longitud silábica, ya que las palabras suelen ser más cortas en la mayoría de las canciones con un ritmo rápido. Sin embargo, es importante tener en cuenta que esta es solo una tendencia y no una regla absoluta."
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT
    p.id,
    p.bpm,
    (LENGTH(a.mapeo_silabas_duraciones)
     - LENGTH(REPLACE(a.mapeo_silabas_duraciones, ',', '')) + 1
     ) AS num_silabas_aprox
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id;
```

Tras ejecutar la consulta se obtuvo una tabla con la información necesaria para analizar las obras objetivo de la consulta.

id	bpm	num _silabas _aprox
12379	76	54
12380	72	33
12381	69	58
12382	132	48
12383	80	31

Tabla 6.3: Fragmento de la tabla de relaciones entre BPM y número aproximado de sílabas

- **Revisión de respuestas.** La respuesta del modelo basado en File Search como RAG identifica de la limitación del problema, ya que reconoce que el análisis requiere procesamiento estadístico y estructural de datos (tempo y segmentación silábica), algo que excede la simple recuperación de información. Sin embargo, no aporta ningún análisis sobre el dataset ni intenta contrastar la hipótesis planteada.

Por otro lado, la respuesta del modelo local es incorrecta, ya que formula una conclusión general sin soporte empírico ni cálculo sobre los datos. Afirma una tendencia entre BPM altos y menor longitud silábica, pero no realiza ningún tipo de agregación ni comparación entre subconjuntos de piezas, por lo que se trata de una inferencia no verificada.

6.1.47. Identifica piezas donde la nota final sea la dominante (quinto grado) en lugar de la tónica, sugiriendo una estructura abierta

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** None
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "Se encontraron las siguientes piezas donde la nota final es la dominante (quinto grado) en lugar de la tónica, sugiriendo una estructura abierta:

* No se encontraron resultados. Es posible que la pregunta sea un poco ambigua o que no haya suficiente datos disponibles."

- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT
    p.id,
    p.titulo,
    p.tonalidad,
    a.secuencia_notas_silencios
FROM piezas p
JOIN analisis_musical a ON p.id = a.pieza_id
WHERE
    (
        CASE LOWER(p.tonalidad)
            WHEN 'c major' THEN 'G4'
            WHEN 'g major' THEN 'D4'
            WHEN 'd major' THEN 'A4'
            WHEN 'a major' THEN 'E4'
            WHEN 'e major' THEN 'B4'
            WHEN 'f major' THEN 'C4'
            WHEN 'a minor' THEN 'E4'
            WHEN 'e minor' THEN 'B4'
            WHEN 'd minor' THEN 'A4'
            WHEN 'g minor' THEN 'D4'
        END
    )
    =
    TRIM(
        substr(
            replace(
                replace(
                    replace(a.secuencia_notas_silencios, '-', ' '),
                    ',', ' ',
                ),
                ' ', ' ',
            ),
            -3
        )
    )
```

);

Tras ejecutar la consulta se obtuvieron 724 obras que cumplen con los requisitos expuestos.

- **Revisión de respuestas.** La respuesta del modelo basado en File Search como RAG es incorrecta, ya que no proporciona ningún resultado ni análisis sobre la condición planteada. La ausencia de respuesta sugiere una incapacidad del sistema para recuperar o interpretar información relevante sobre la relación entre la tonalidad, la secuencia final de notas y la identificación de la dominante.

Por otro lado, la respuesta del modelo local también es incorrecta, aunque por un motivo distinto: en lugar de intentar resolver el problema, devuelve una negación genérica (“no se encontraron resultados”) sin realizar ningún análisis estructural de la secuencia melódica ni de la correspondencia entre la última nota y el quinto grado de la tonalidad. Esto indica un fallo en la inferencia lógica necesaria para este tipo de consulta musical.

6.1.48. Identifica si existe algún patrón de silencios (<rest />) que se repita sistemáticamente cada cuatro compases en las piezas de métrica ternaria

- **Respuesta con File Search básico como RAG con la API de Gemini (modelo Gemini 2.5 Flash):** Los resultados de la búsqueda no identifican un patrón de silencios (<rest />) que se repita sistemáticamente cada cuatro compases en las piezas de métrica ternaria. Si bien algunos fragmentos musicales en métrica ternaria, como los de los archivos “CO-1948-00-BG-00041.musicxml.xml” y “MX-1964-00-VM-02050.musicxml.xml”, contienen silencios en compases consecutivos, no se observa una recurrencia regular cada cuatro compases.
- **Respuesta con modelo local desde Ollama, con acceso a la base de datos SQLite como RAG (modelo llama3.1:8b):** "No se encontró ningún patrón de silencios (<rest />) que se repita sistemáticamente cada cuatro compases en las piezas de métrica ternaria. Es posible que la pregunta sea un poco ambigua o que no haya suficiente datos disponibles."
- **Resultado esperado.** Se ha ejecutado una consulta SQL para comprobar cuál habría sido el resultado completo correcto.

```
SELECT
    p.id,
    p.titulo,
    p.compas,
    a.secuencia_notas_silencios
FROM piezas p
JOIN analisis_musical a
    ON p.id = a.pieza_id
WHERE p.compas IN ('3/4', '6/8', '9/8', '12/8')
    AND LOWER(a.secuencia_notas_silencios) LIKE '%silencio%';
```

Tras ejecutar la consulta se una tabla que permite analizar los resultados obtenidos para responder a la pregunta propuesta.

id	Título	Compás	Secuencia notas/silencios
4	IE-2019-D-HLS-004	6/8	G4 - B4 - A4 - B4 - G4 - A4 - B4 - C5 - D5 - B4 - C5 - A4 - A4 - B4 - D5 - G5 - F#5 - E5 - D5 - B4 - G4 - A4 - C5 - B4 - A4 - G4 - G4 - Silencio
5	IE-2019-D-HLS-005	12/8	D4 - G4 - G4 - G4 - B4 - D4 - D4 - D4 - E4 - G4 - G4 - G4 - A4 - B4 - B4 - B4 - C5 - C5 - C5 - E5 - D5 - B4 - G4 - B4 - ... - G4 - G4 - Silencio
6	IE-2019-D-HLS-006	12/8	B4 - E5 - B4 - D5 - D5 - E5 - D5 - D5 - B4 - D5 - D5 - E5 - D5 - D5 - B4 - C#5 - D5 - D5 - E5 - E5 - F#5 - ... - E4 - E4 - Silencio
8	IE-2019-D-HLS-008	3/4	G4 - A4 - B4 - B4 - B4 - C5 - C5 - D5 - C5 - B4 - B4 - G4 - A4 - G4 - G4 - A4 - B4 - ... - D5 - C5 - A4 - G4 - G4 - G4

Tabla 6.4: Fragmento de la tabla de ejemplos de piezas en métrica ternaria que contienen silencios.

- **Revisión de respuestas.** La respuesta del modelo basado en File Search como RAG identifica adecuadamente la presencia de silencios en algunas piezas con métrica ternaria y reconoce que no se observa de forma clara un patrón periódico cada cuatro compases. Sin embargo, su análisis es incompleto, ya que se limita a una observación cualitativa sobre ejemplos aislados sin realizar una verificación sistemática del patrón temporal en el conjunto del dataset.

Por otro lado, la respuesta del modelo local es incorrecta, aunque formalmente coherente, ya que concluye que no existe suficiente información sin analizar los datos disponibles ni considerar la posibilidad de detección de patrones estructurales en la secuencia de compases. Esto refleja una dificultad para generar la consulta compleja que requiere esta pregunta.

6.2. Resumen de preguntas formuladas y respuestas de los modelos

En esta sección se presenta un resumen comparativo de las respuestas obtenidas por los dos enfoques evaluados: el modelo basado en File Search como RAG (Gemini 2.5 Flash) y el modelo local Llama3.1:8B con acceso a una base de datos SQLite. Para cada consulta planteada sobre el dataset musical, se ha clasificado la respuesta de ambos sistemas en términos de su corrección respecto a los resultados esperados obtenidos mediante consultas SQL.

Las etiquetas utilizadas en la evaluación son: *Correcta*, cuando la respuesta coincide plenamente con el resultado de la consulta (teniendo en cuenta, en el caso del modelo local, que existe un límite impuesto de 10 resultados por consulta); *Parcialmente correcta*, cuando la respuesta es razonable pero incompleta o imprecisa; e *Incorrecta*, cuando la respuesta no coincide con los datos del dataset o presenta errores relevantes de inferencia o recuperación de información.

A continuación se muestra la tabla resumen con la comparación de ambos modelos en todas las preguntas evaluadas.

Tabla 6.5: Resumen de resultados obtenidos para las consultas de evaluación.

Pregunta	Gemini 2.5 Flash (File Search RAG)	Llama3.1:8B (RAG con SQLite)
Identifica piezas que se llamen El Carro	Incorrecta	Correcta
Identifica piezas cuyo autor sea Schumann	Parcialmente correcta	Parcialmente correcta
Identifica piezas con un compás de 4/4	Incorrecta	Correcta
Identifica piezas con hemiolias horizontales	Incorrecta	Correcta
Identifica piezas con hemiolias verticales	Incorrecta	Correcta
Identifica piezas con la tonalidad G major	Incorrecta	Correcta
Identifica piezas que estén en modo dórico	Incorrecta	Correcta
Identifica piezas con un bpm de 120	Incorrecta	Correcta
Identifica piezas con un volumen midi de 79	Parcialmente correcta	Correcta
Identifica piezas cuya nota más grave sea A3	Incorrecta	Correcta
Identifica piezas cuya nota más aguda sea G5	Incorrecta	Correcta
Identifica piezas cuyo autor sea de género masculino	Incorrecta	Correcta
Identifica piezas cuyo software de codificación haya sido Sibelius 7.1.3	Incorrecta	Correcta
Identifica piezas cuya fecha de codificación sea el 2015-07-01	Incorrecta	Correcta
Identifica piezas con sín copas	Incorrecta	Correcta
Identifica piezas que traten la muerte como tema	Parcialmente correcta	Correcta
Identifica piezas con polirritmia	Incorrecta	Correcta
Identifica piezas cuyo contenido lírico esté escrito en voz masculina	Parcialmente correcta	Correcta
Identifica obras cuya tendencia melódica sea predominantemente descendente	Incorrecta	Correcta
Identifica piezas con 4 alteraciones accidentales	Incorrecta	Correcta
Identifica piezas con compases vacíos	Incorrecta	Correcta
Identifica piezas con notas de duración 0	Incorrecta	Correcta
Identifica piezas con plicas anómalas	Incorrecta	Correcta
Identifica piezas que usen la palabra "bondad"	Incorrecta	Correcta
Identifica piezas que mencionen el nombre propio "Alfonsito"	Incorrecta	Correcta
Identifica piezas cuyo intervalo frontera más frecuente sea +5	Incorrecta	Correcta
Identifica piezas en 2/4 que contengan sín copas ligadas entre el segundo pulso de un compás y el primero del siguiente de forma sistemática	Incorrecta	Parcialmente correcta
Identifica piezas de tonalidad A menor que tengan hemiolias y que no superen los 90 bpm	Incorrecta	Incorrecta
Identifica piezas de tonalidad A menor que tengan hemiolias	Incorrecta	Correcta

Encuentra canciones donde el valor de dur.ppq cambie a mitad de la sección, indicando un cambio de resolución rítmica	Incorrecta	Correcta
Busca ejemplos de anacrusa en piezas que estén etiquetadas como “Nanas”	Parcialmente correcta	Incorrecta
Localiza obras que tengan compases donde la suma de las duraciones de las etiquetas <note>no coincida con el meterSig declarado en el staffDef	Incorrecta	Correcta
¿Qué piezas utilizan valores irregulares (por ejemplo, tresillos) no declarados explícitamente en un <time-modification>del MusicXML?	Correcta	Correcta
Compara la densidad de notas por compás entre una pieza de 4/4 y una de 2/4 del dataset; ¿cuál tiene mayor promedio de eventos musicales?	Incorrecta	Incorrecta
Identifica si existe algún uso de polirritmia en los archivos MEI	Incorrecta	Correcta
Lista canciones que compartan la misma armadura de un bemol (F major/D minor) pero que traten temas distintos	Incorrecta	Correcta
Encuentra piezas recolectadas antes de 1950 que traten el tema de “la muerte” y que utilicen exclusivamente un rango melódico de sexta menor	Incorrecta	Incorrecta
¿Qué diferencias de tesitura (nota más aguda vs. nota más grave) existen entre las canciones de nana?	Incorrecta	Parcialmente correcta
Identifica canciones donde el autor sea una mujer, pero el contenido lírico esté escrito en voz masculina	Incorrecta	Incorrecta
Filtra piezas que usen la palabra “estrella” en la lírica y determina si su melodía es predominantemente ascendente o descendente	Incorrecta	Correcta
Busca canciones que traten temas de trabajo que compartan la misma estructura de intervalos que las canciones de tema infantil	Incorrecta	Correcta
Localiza todas las notas que tengan bemoles accidentales y dime en qué compases aparecen en los archivos MEI encontrados	Parcialmente correcta	Incorrecta
Identifica el uso de la etiqueta <sb />(system break) y determina si coincide siempre con el final de una frase musical lógica	Incorrecta	Correcta
Compara el uso de las sílabas “duer” y “me” en los distintos archivos del dataset: ¿se asigna siempre a la misma duración de nota?	Parcialmente correcta	Incorrecta
¿Cuál es el intervalo melódico más frecuente entre la última nota de un compás y la primera del siguiente en las piezas de modo mixolidio?	Incorrecta	Incorrecta

Determina si las canciones con tempo superior a 120 BPM tienden a tener letras con palabras de menor longitud silábica	Incorrecta	Incorrecta
Identifica piezas donde la nota final sea la dominante (quinto grado) en lugar de la tónica, sugiriendo una estructura abierta	Incorrecta	Incorrecta
Identifica si existe algún patrón de silencios (<rest />) que se repita sistemáticamente cada cuatro compases en las piezas de métrica ternaria	Incorrecta	Incorrecta

Discusión y análisis de limitaciones

Los resultados obtenidos permiten comparar el comportamiento de dos aproximaciones distintas para la recuperación y análisis de información musical: un modelo basado en RAG con File Search (Gemini 2.5 Flash) y un modelo local (Llama3.1:8B) con acceso directo a una base de datos estructurada en SQLite previamente creada a partir de la extracción de metadatos de cada obra. A partir de la evaluación de las distintas consultas planteadas sobre el dataset se observan diferencias relevantes tanto en precisión como en la capacidad de inferencia estructurada.

En términos generales, el modelo con acceso directo a la base de datos mediante consultas SQL ha mostrado un mayor grado de consistencia en las respuestas cuando la información requerida estaba explícitamente representada en la estructura del esquema. Esto se debe a que la formulación de consultas permite una extracción determinista de los datos, reduciendo la ambigüedad inherente a la interpretación del lenguaje natural. No obstante, este enfoque también presenta limitaciones cuando la consulta requiere inferencias complejas o la combinación de múltiples condiciones no triviales, lo que en algunos casos ha llevado a respuestas incompletas o a la incapacidad del modelo para traducir correctamente la pregunta a SQL.

Por otro lado, el modelo basado en File Search (RAG con Gemini 2.5 Flash) tiende a ofrecer respuestas más descriptivas y contextualizadas, pero presenta una mayor propensión a la generación de contenido no respaldado directamente por el dataset. En varios casos se han observado alucinaciones, como la referencia a piezas que no existen en el corpus o la inferencia de ejemplos musicales no presentes en los archivos analizados. Esto indica que, aunque el sistema es capaz de generar explicaciones coherentes, su fiabilidad disminuye cuando no dispone de evidencias explícitas en los fragmentos recuperados.

Un patrón recurrente en ambos modelos es la dificultad para resolver consultas que requieren razonamiento en varios pasos o cálculos y análisis musicológicos complejos (por ejemplo, intervalos melódicos, relaciones entre compases o patrones estructurales). En estos casos, los modelos tienden a simplificar la respuesta o asumir correlaciones no justificadas, lo que afecta directamente a la precisión final.

Asimismo, se observa que las consultas que implican combinaciones de atributos musicales abstractos (como modo, intervalos fronterizos, estructura melódica o análisis de sílabas) son especialmente sensibles a errores de interpretación. Esto sugiere que la representación actual del dataset, aunque estructurada, no siempre está optimizada para consultas analíticas complejas sin una capa intermedia de procesamiento especializado.

Otra limitación importante identificada es la dependencia del modelo local respecto a

la calidad de la inferencia de la consulta SQL. En varios casos, la dificultad no reside en la disponibilidad de los datos, sino en la correcta traducción de la pregunta en una consulta ejecutable. Esto introduce una fuente adicional de error que no está presente en sistemas puramente basados en ejecución de queries predefinidas.

A las limitaciones conceptuales del modelo se añaden desafíos técnicos críticos detectados durante la fase de implementación de la arquitectura. El primero de ellos aparece en el diseño del pipeline para la extracción de la variable geográfica. Inicialmente se proyectó un sistema híbrido de tres pasos para inferir la región de procedencia de las obras musicales: en primer lugar se rastreaba la etiqueta geográfica nativa dentro de la codificación interna del archivo; si esta no existía, el script buscaba coincidencias ascendiendo por la estructura de directorios del sistema de archivos; finalmente, si ambos pasos fallaban, se proporcionaba la lírica completa a un LLM para que dedujera de forma semántica o dialectal la región.

La ejecución práctica demostró la inviabilidad de esta aproximación. Por un lado, la enorme heterogeneidad estructural y las inconsistencias de catalogación de las carpetas originales introducían un ruido severo en los metadatos. Por otro lado, la inferencia basada puramente en el texto lírico mediante LLMs arrojaba una tasa de error inaceptable, asignando sistemáticamente regiones geográficas inconexas o erróneas que no correspondían con el contexto de la pieza. Ante la falta de fiabilidad de estos métodos, esta herramienta fue completamente excluida de la arquitectura definitiva y se optó por suprimir la variable regional del banco final de preguntas de test.

El segundo obstáculo técnico estuvo ligado a la gestión del desbordamiento del contexto al interactuar con la base de datos relacional. Al ejecutar determinadas consultas SQL sobre un conjunto tan extenso de datos, las respuestas iniciales recuperaban de golpe miles de registros. Al reinyectar este volumen de información estructurada en bruto dentro del contexto de un modelo local pequeño (*Llama3.1:8B*), sus capacidades de razonamiento colapsaban. Este fenómeno provocaba comportamientos anómalos y alucinaciones severas, llegando al extremo de que el modelo perdía la directiva del sistema; en lugar de procesar los datos para el usuario, el modelo replicaba textualmente la consulta SQL o le devolvía las instrucciones internas de ejecución en lenguaje natural. Para garantizar la estabilidad del pipeline y salvaguardar la ventana de contexto del modelo de 8B, se determinó la necesidad de implementar una política estricta de truncamiento a la hora de recuperar los resultados de la consulta, forzando `LIMIT 10` de manera sistemática en todas las consultas del proyecto.

Como conclusión, ambos enfoques presentan fortalezas complementarias: mientras que el acceso directo a base de datos ofrece precisión y trazabilidad, el enfoque RAG aporta flexibilidad lingüística y capacidad de síntesis. Sin embargo, ninguno de los dos sistemas es completamente robusto ante consultas musicales complejas sin un diseño intermedio que combine generación de consultas, validación de resultados y razonamiento estructurado.

Conclusiones y Trabajo Futuro

8.1. Conclusiones

Este trabajo se ha desarrollado en el marco del proyecto europeo e iberoamericano *EA-DIGIFOLK*, con el propósito de diseñar, implementar y evaluar una arquitectura automatizada, robusta y escalable capaz de superar las limitaciones metodológicas tradicionales en el análisis de grandes corpus musicales folclóricos. Frente a los problemas de heterogeneidad de formatos y las limitaciones de contexto de los modelos de lenguaje (LLM) de propósito general, se propuso una infraestructura intermedia fundamentada en la extracción y normalización de metadatos mediante herramientas especializadas como *music21* y el almacenamiento de la información en una base de datos relacional estructurada (*SQLite*).

La evaluación y análisis comparativo de este sistema frente a aproximaciones alternativas han arrojado resultados determinantes sobre el comportamiento de la inteligencia artificial aplicada a la musicología computacional. El estudio demuestra que el acceso directo a una base de datos estructurada mediante inferencia SQL local (utilizando *Llama3.1:8B*) ofrece una extracción de datos sustancialmente más determinista, precisa y consistente que los sistemas basados puramente en recuperación y búsqueda de archivos (*Retrieval-Augmented Generation* o RAG con modelos grandes y generales como *Gemini 2.5 Flash*). Mientras que el enfoque RAG destaca por su flexibilidad lingüística y capacidad de síntesis, su propensión a las alucinaciones y a la generación de contenido no respaldado por el dataset disminuye críticamente su fiabilidad en entornos de investigación especializada, como es el caso de la musicología técnica. No obstante, los resultados también evidencian que ambos enfoques sufren ante consultas que requieren razonamiento musical complejo en varios pasos.

A partir de los hallazgos y de la discusión detallada, se extraen las siguientes conclusiones principales:

- El almacenamiento estructurado en SQLite combinado con un modelo local permite una recuperación de información altamente fiable y libre de alucinaciones cuando los datos requeridos están explícitamente representados en el esquema. Por el contrario, los sistemas RAG basados en File Search aportan una mayor riqueza contextual y descriptiva en lenguaje natural, pero introduce imprecisiones y falsos positivos (como la invención de piezas inexistentes en el corpus).
- La principal limitación del enfoque local estructurado no reside en la disponibilidad de los datos analíticos, sino en la capacidad del modelo para traducir correctamente

preguntas complejas de lenguaje natural a consultas SQL ejecutables. Los errores en esta traducción intermedia constituyen una fuente de fallo crítica que disminuye la robustez del sistema ante consultas no triviales o compuestas.

- Ninguno de los dos enfoques evaluados es completamente autónomo o robusto a la hora de resolver de manera directa consultas que impliquen abstracciones complejas o cálculos matemáticos y musicales en varios pasos (como el análisis de intervalos fronterizos, síncopas o estructuras melódicas avanzadas). Ambos sistemas tienden a simplificar las respuestas o a asumir correlaciones injustificadas ante la falta de una capa de procesamiento analítico previo.
- Para que las herramientas basadas en LLMs sean verdaderamente útiles en la investigación musicológica de grandes colecciones, la representación de los datos no sólo debe estar estructurada, sino explícitamente optimizada para consultas analíticas. Se concluye que para mejorar estas herramientas y sus resultados es necesario un diseño de arquitecturas intermedias que combinen de forma híbrida la generación guiada de consultas, la validación estricta de resultados y módulos especializados de razonamiento estructurado.

8.2. Trabajo futuro

La herramienta desarrollada constituye una primera aproximación a la aplicación de técnicas de inteligencia artificial y análisis musical computacional sobre corpus de partituras digitales. No obstante, existen diversas líneas de investigación que podrían explorarse en trabajos futuros con el objetivo de ampliar las capacidades analíticas de la herramienta, mejorar la calidad de las respuestas y enriquecer la experiencia de usuario.

Una primera línea de desarrollo consiste en la implementación de arquitecturas multi-agente basadas en el protocolo *Agent-to-Agent* (A2A). En lugar de delegar todo el proceso de razonamiento en un único agente conversacional, sería posible distribuir las tareas entre agentes especializados en distintos ámbitos, como análisis musical, recuperación de información, interpretación musicológica o generación de respuestas. Este enfoque permitiría abordar consultas más complejas mediante procesos colaborativos entre agentes, facilitando la especialización y la escalabilidad funcional del sistema.

Otra posible evolución consiste en la incorporación de técnicas de *Graph Retrieval-Augmented Generation* (GraphRAG). Mientras que la implementación actual almacena los metadatos en una base de datos relacional, un modelo basado en grafos permitiría representar explícitamente las relaciones existentes entre canciones, autores, géneros, regiones geográficas, estructuras musicales y características estilísticas. Esta representación facilitaría consultas más complejas basadas en conexiones semánticas y musicológicas, así como la exploración automática de relaciones no evidentes dentro del corpus.

Asimismo, podría investigarse la integración de modelos híbridos que combinen la base de datos actual con estructuras de conocimiento basadas en grafos. De este modo, sería posible mantener la eficiencia de las consultas estructuradas y, simultáneamente, aprovechar las ventajas de los sistemas de recuperación contextual basados en relaciones.

Otra posible línea de investigación consiste en la implementación de mecanismos de búsqueda por similitud musical basados en representaciones vectoriales (*embeddings*). A partir de los metadatos extraídos durante el preprocesamiento, cada obra podría representarse mediante un vector numérico que resumiera sus principales características musicales y textuales, tales como tonalidad, modo, compás, patrones rítmicos, fenómenos métricos o

temáticas presentes en la letra. Esta representación permitiría calcular automáticamente el grado de similitud entre canciones y habilitar nuevas funcionalidades, como la búsqueda de obras relacionadas, la identificación de repertorios con características comunes o la recomendación automática de canciones.

Por otra parte, una línea de desarrollo técnico prioritaria para mejorar la precisión del modelo local reside en la optimización de las consultas relacionales sobre datos semiestructurados encapsulados. Actualmente, el esquema de la base de datos *SQLite* almacena variables musicológicas de alta densidad relacional y variabilidad en columnas de tipo texto bajo formato estructurado JSON; es el caso de campos críticos como `mapeo_silabas_duraciones` (que mapea la correspondencia exacta entre sílabas líricas y figuras musicales) o `accidentales_compases_json` (que registra la ubicación por compás de alteraciones accidentales como *F#5* o *C#5*).

Con el objetivo de eliminar las limitaciones de razonamiento multietapa identificadas en los LLMs, un trabajo futuro inmediato consistirá en dotar al agente de la capacidad de ejecutar funciones nativas de extracción como `json_extract()`. La integración y el entrenamiento del modelo en la sintaxis de estas funciones permitirán aislar, consultar y operar directamente sobre las claves y valores internos de los objetos JSON sin necesidad de alterar el esquema relacional rígido ni sobrecargar el contexto del modelo. Esto se traduciría en un incremento de la robustez del sistema ante preguntas analíticas complejas que crucen, por ejemplo, síncopas específicas con duraciones silábicas o alteraciones accidentales en compases determinados.

Por último, una mejora orientada al usuario consistiría en el desarrollo de una interfaz específica para la comparación de canciones. Esta funcionalidad permitiría seleccionar dos o más obras y visualizar de manera conjunta sus características musicales, textuales y estructurales. Entre otros aspectos, podrían compararse tonalidades, modos, compases, patrones rítmicos, densidad de eventos, fenómenos de síncopa, polirritmias, hemiolias o temáticas presentes en las letras. Una herramienta de este tipo facilitaría los estudios comparativos y podría resultar especialmente útil en contextos de investigación musicológica y análisis de repertorios tradicionales.

Además de estas líneas concretas, la naturaleza modular de la arquitectura propuesta permite incorporar nuevos algoritmos de extracción de metadatos a medida que se identifiquen nuevas necesidades analíticas. Esto abre la puerta a futuras investigaciones centradas en la detección automática de fenómenos musicales más complejos y en la ampliación progresiva de la base de conocimiento utilizada por el sistema.

Chapter 9

Introduction

“Many things we can’t do are simply because we think we can’t do them”

— Satoru Iwata

The study of large musical corpora constitutes one of the primary research tools within disciplines such as historical musicology, ethnomusicology, musical analysis, or sound heritage documentation. Through the systematic analysis of extensive sets of works, it is possible to identify stylistic patterns, recurring compositional structures, relationships between repertoires, and cultural phenomena that can hardly be observed through the isolated study of individual pieces.

Traditionally, this type of research has depended on manual processes of score reading, cataloging, and analysis—a rigorous methodology but limited in terms of scalability. The growing volume of digitized musical material, especially in fields related to traditional music and musical heritage, has highlighted the need to develop tools capable of automating part of these tasks and facilitating the exploration of large documentary collections.

The emergence of standardized symbolic formats for musical representation, such as MusicXML or MEI, has driven the development of computational musicology and has enabled the application of automated analysis techniques over increasingly broad repertoires. However, practice demonstrates that significant challenges persist regarding corpus heterogeneity, format compatibility, the extraction of complex musicological information, the integration between musical and textual content, and efficient knowledge retrieval from large volumes of data.

In parallel, recent advances in artificial intelligence and natural language processing have opened new possibilities for interacting with extensive collections of data. Nevertheless, the particularities of musical information pose limitations that hinder the direct application of these technologies, especially when queries require knowledge derived from specialized musical analyses that are not explicitly represented in the original sources.

In this context, this project proposes the design and development of an architecture oriented towards the automated analysis of large and heterogeneous musical corpora. The system combines score processing techniques, musical feature extraction, automatic cataloging, information retrieval, and assistance through language models, with the aim of providing a platform capable of facilitating musicological research and improving access to the knowledge contained within large musical collections.

Within this framework, the project *"EA-DIGIFOLK-An European and Ibero-American approach for the digital collection, analysis and dissemination of folk music (101086338)"*

is developed. Funded by the European Union, with the participation of the UCM¹, its primary objective is to preserve and share European and Ibero-American popular folk culture, especially within the musical domain. EA-DIGIFOLK seeks to create a platform that facilitates the dissemination of this culture to educational environments and audiences. Furthermore, and more closely related to this project, another core objective is the musical and ethnomusicological analysis of the collected audiovisual and written material. This work has been carried out as part of this research project.

9.1. Motivation

The musicological analysis of large volumes of traditional and/or folk music has historically faced a methodological limitation: its dependence on the manual analysis of scores, which represents a rigorous but hardly scalable process, making the digital automation of this procedure virtually impossible. While the digitization of musical archives into structured symbolic formats (such as MusicXML or MEI) has opened the doors to computational musicology, fundamental problems that hinder any automation methodology when digitizing this type of content continue to persist.

The following are some obstacles and difficulties encountered in the field of musical analysis that have motivated the development of this project:

- **Heterogeneity in the corpora.** Large-scale musical corpora, such as the one used throughout this project, tend to be highly heterogeneous both structurally and in their cataloging. First, works may be stored in multiple file formats (*MSCZ*, *MusicXML*, *MEI*, among others), each with technical and semantic particularities that require specific extraction and normalization processes. This diversity prevents the construction of fully automated analysis workflows and forces the development of preprocessing procedures adapted to each information source. Furthermore, corpus heterogeneity is not limited to file formats but also affects the cataloging criteria and organization of repositories. In these cases, collections are frequently encountered where certain fundamental metadata, such as the title of the work, the author, or the geographical origin, are stored within the structure of the file itself, whereas in other cases, such information is only available in the filename or in the hierarchical organization of the directories. The lack of homogeneous standards for documentary description complicates the automatic identification of works and requires additional mechanisms for inference and metadata validation.
- **Inconsistency in symbolic standards.** Many standard tools used in the technical field of musicological analysis, such as *music21* can experience critical failures or lose relevant metadata when interpreting certain information sources. This occurs, for example, with certain tokens present in MEI files (such as the *wordpos* attribute, which indicates the position of a syllable within a word; however, not all the values it can take are interpretable by many programs like *music21*), whose implementation is not fully normalized among different score editing software.
- **Algorithmic complexity in the detection of complex rhythmic phenomena.** In the world of music, there is an infinity of expressive and ambiguous structures, such as syncopations and hemiolas (both horizontal and vertical), which require mathematical modeling with a strict preliminary study based on very specific characteristics

¹<https://cordis.europa.eu/project/id/101086338>

of the works, such as attack times and metric hierarchies. This entails working from scratch on functions that are not natively implemented in conventional analytical libraries.

- **Division between musical and semantic analysis.** Many automated musical analysis systems isolate structural and rhythmic content from the textual content of the lyrics. This is a highly problematic separation, as decoupling these two deeply related structures results in a severe loss of information. Therefore, it is necessary to design a system capable of extracting analytical musical characteristics while simultaneously classifying the thematic and poetic content of the songs.
- **Contextual limitations of language models in extensive corpora.** When the size of the corpus is large, providing the model with all the information as input context becomes unfeasible, particularly in smaller models whose context limits and reasoning capabilities are more restricted. Even when employing information retrieval strategies (*Retrieval-Augmented Generation*, RAG), many musicological queries require the prior extraction and calculation of complex analytical features from the scores, such as rhythmic patterns, melodic structures, or metric relationships. Since these processes cannot be reliably executed in real time by a language model, an intermediate architecture capable of performing specialized musical analysis and storing its results in optimized structures for subsequent retrieval and querying becomes necessary.

The project originates from these gaps, aiming to resolve them through the study and development of a system capable of meeting the needs of musicologists and users who require more sophisticated tools to handle the analysis of very extensive and complex corpora of data. The main purpose of this research centers on developing an automated, robust, and scalable architecture capable of analyzing latent features in a corpus of musical works, simultaneously integrating structural, rhythmic, melodic, and semantic information to facilitate the musicological study of large folk collections.

9.2. Objectives

Based on the limitations identified in current computational musicological analysis methodologies, this project seeks to develop a technological infrastructure that facilitates the study of large musical corpora through automated analysis techniques, information retrieval, technical musicological analysis, and natural language processing. To this end, the following objectives are established.

9.2.1. General Objectives

To develop an automated, robust, and scalable system for the analysis, cataloging, and querying of large, heterogeneous musical corpora, capable of extracting musicological knowledge from a large number of works stored in different formats, thereby facilitating research, exploration, and information retrieval for musicologists, researchers, and other specialized users.

9.2.2. Specific Objectives

- Design an architecture capable of processing musical corpora composed of different symbolic representation formats, guaranteeing compatibility between diverse docu-

mentary sources.

- Develop an automatic system for the extraction and normalization of musical meta-data in order to facilitate the systematic cataloging of works and the construction of reliable and consistent databases.
- Implement musical analysis algorithms that allow for the detection and quantification of structural, melodic, rhythmic, and metric characteristics relevant to musicological research.
- Integrate musical analysis with the semantic and textual analysis of the lyrics, enabling the joint study of both the musical and literary aspects of the works.
- Automate processing and analysis tasks that usually require a high investment of time or specialized technical knowledge, reducing barriers to access advanced musical research tools.
- Design and implement an information retrieval system that allows for complex searches over the corpus, including queries based on both metadata and automatically extracted musical features.
- Evaluate the capabilities and limitations of general-purpose language models in tasks involving the analysis and querying of extensive musicological corpora, identifying the scalability, precision, and context problems that these systems present.
- Propose complementary mechanisms to overcome the limitations observed in these models through the incorporation of specialized musicological analysis and optimized structures for knowledge retrieval.

9.3. Work Plan

The work plan to be followed to achieve the objectives described in the previous section is detailed here. In order to attain the proposed objectives, a work plan structured into six main phases has been designed, spanning from late October to early June. This schedule encompasses everything from the initial research and requirements gathering through field work, to the technical development of the architecture, its comparative evaluation, and the final writing of the research report. Below, the temporal planning is detailed using a Gantt chart (Figure 9.1):

At an operational level, the development is broken down into the following key activities directly linked to the project's objectives:

- **Phase 1 (October – December):** Dedicated to the deep study of the state of the art in computational musicology.
- **Phase 2 (March):** Exclusively dedicated to requirements gathering through qualitative interviews with musicologists and researchers in the field. The goal of this milestone is to align the system's capabilities with the actual daily analytical needs of expert users.
- **Phase 3 (December – February):** Focused on the design and implementation of the intermediate infrastructure and the automatic normalization pipeline, responsible for unifying data persistence into the local SQLite file regardless of the original heterogeneity or organization of the folk corpus.

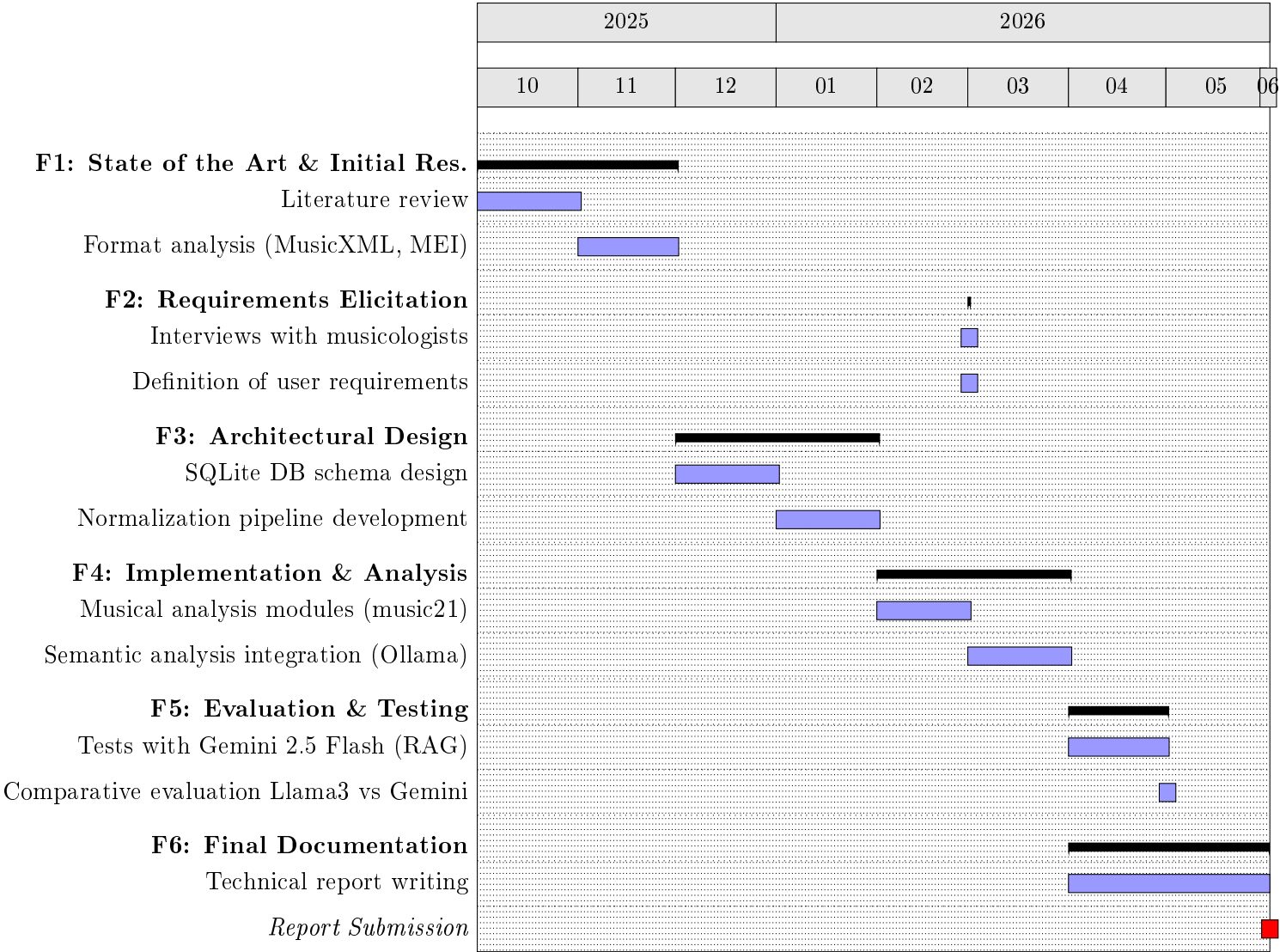


Figure 9.1: Gantt chart of the work plan (October - June).

- **Phase 4 (February – April):** Focused on the development of algorithms based on *music21* for extracting complex technical features (rhythmic and melodic structures) and their integration with local language models (*Llama3* via *Ollama*) intended for the semantic and thematic classification of the works' lyrics.
- **Phase 5 (April – May):** Allocated to the critical evaluation phase. Tests are conducted on the information retrieval system, implementing both the local text-to-SQL approach and the cloud-based RAG approach via the native *File Search* functionality of the Gemini API, enabling a contrast of precision, context, and hallucination boundaries between both technologies.
- **Phase 6 (April – June):** Consisting of writing the research report and preparing the documentation for the developed software to ensure full technical reproducibility later on.

9.4. Methodology and Technology

To develop this project, open-source tools and the free version of the Gemini API have been selected. This decision ensures the full reproducibility of the project locally (with the exception of utilizing the Gemini API).

- **Python 3.** Used as the base to develop the entire project. This high-level, general-purpose programming language was selected primarily because it hosts the most advanced ecosystem of libraries for computational musicology, data science, and artificial intelligence. Its versatility allows for articulating within a single automated workflow everything from score preprocessing and metadata normalization from heterogeneous musical corpora, to local database management, communication with language model servers via structured requests, and the subsequent analytical evaluation of results.
- **Music21.** A leading toolkit for computational musicology. It allows for a deep musical analysis of each work regarding its notes, intervals, chords, scales, etc., which facilitates the study of harmonic, melodic, and rhythmic patterns that are difficult to identify with other tools and, in this case, with LLMs. Additionally, it supports multiple formats, such as MIDI or XML, which is highly useful in this research, where we work with an extensive and heterogeneous corpus of works in different formats. Moreover, its integration with Python and other libraries facilitates the extraction and analysis of information from the musical corpus.
- **Ollama.** A local server that enables the use of language models in a local and customized manner. *Llama3* has been selected as the LLM to execute various natural language processing tasks. This mitigates external API costs, protects the privacy of the corpus, and facilitates output invariance through the native restriction of responses to structured JSON format.
- **Gemini API (Free Version).** A cloud services platform based on generative artificial intelligence developed by Google. In this project, the *Gemini 2.5 Flash* model has been specifically selected through its cost-free API to implement a retrieval and query approach based on RAG architectures assisted by the native *File Search* feature.

- **SQLite.** A library that implements a small, free, and open-source SQL database engine. This specific tool was chosen to store the extracted information because, unlike other databases such as MySQL, PostgreSQL, or SQL Server—which require installing software that runs in the background of your computer, configuring users, passwords, and ports—SQLite does not depend on a server, but instead creates the database as a single local file.

9.5. Structure of the Rest of the Document

To facilitate the reading and comprehension of the developed project, the content of this report is organized into different chapters sequentially, in accordance with the lifecycle of the developed project:

- **Chapter 2: State of the Art.** Provides the fundamental theoretical framework of the research. It contextualizes the phenomenon known as the “*computational turn*” applied to the analysis of artistic, creative, and musical texts, reviewing existing methodologies and current challenges in symbolic musical processing.
- **Chapter 3: Requirements Elicitation with Musicologists and Question Formulation.** Details the requirements elicitation phase carried out through fieldwork. It describes the personal interviews and web forms conducted with expert musicologists to define the needs of the end user. In addition, it presents the final set of questions designed to test the analytical capabilities of both the language models and the implemented systems.
- **Chapter 4: Information Indexing and Retrieval System Based on RAG Gemini File Search.** Presents the implementation of the first technological approach of this project. It details the pipeline responsible for preparing and indexing the corpus documents, as well as the retrieval strategy through semantic search utilizing the *Gemini* indexing engine.
- **Chapter 5: Preprocessing of the Musical Corpus for Metadata Extraction.** Constitutes the methodological core and the main technical development of the research. It describes in detail the implementation of the pipeline that automatically extracts various musicological, structural, lyrical, and quality variables using the *music21* library. Furthermore, it explains the configuration of the *Model Context Protocol* (MCP) environment that enables direct relational queries over a local *SQLite* file using a small local model (*Llama3.1:8B*), and concludes with the implementation of the developed web application interface.
- **Chapter 6: Evaluation and Testing.** Presents the results of the project through the execution of the testbed across both architectures. The responses and inferences provided by the language models for each of the formulated complex musical questions are explicitly contrasted and analyzed.
- **Chapter 7: Discussion and Analysis of Limitations.** Critically addresses the qualitative and quantitative comparison between the relational deterministic approach (*Text-to-SQL*) and the semantic approach (*RAG*), openly discussing the encountered hallucination rates, context issues, and the technical limitations resolved during the development of the architecture.

- **Chapter 8: Conclusions and Future Work.** Synthesizes the main findings derived from the research, validating the fulfillment of the proposed general and specific objectives. Concurrently, it defines priority and immediate research lines, such as the implementation of multi-agent architectures (A2A), *GraphRAG* techniques, or the exploitation of semi-structured JSON variables through native functions.

Finally, the report incorporates an **Appendix A**, which compiles scientific knowledge transfer activities, detailing the presentation and impact of the project in national and international conferences as well as academic seminars in the field. It is worth noting that, to comply with international research dissemination guidelines, the chapters corresponding to the **Introduction** and **Conclusions and Future Work** are replicated in English in Chapters 9 and 10, respectively.

9.6. Use of Artificial Intelligence

This work has been developed with the occasional support of artificial intelligence tools for technical assistance and editing. Their use was limited to auxiliary tasks, such as organizing ideas, improving writing clarity, and supporting the implementation of certain parts of the code. In all cases, design decisions, result validation, and ultimate responsibility for the content and implementation rest exclusively with the author.

Conclusions and Future Work

10.1. Conclusions

This work has been carried out within the framework of the European and Ibero-American project *EA-DIGIFOLK*, with the purpose of designing, implementing, and evaluating an automated, robust, and scalable architecture capable of overcoming traditional methodological limitations in the analysis of large folk musical corpora. To address the problems of format heterogeneity and the context limitations of general-purpose language models (LLMs), an intermediate infrastructure was proposed based on the extraction and normalization of metadata using specialized tools like *music21* and storing the information in a structured relational database (*SQLite*).

The evaluation and comparative analysis of this system against alternative approaches have yielded decisive results regarding the behavior of artificial intelligence applied to computational musicology. The study demonstrates that direct access to a structured database through local SQL inference (utilizing *Llama3.1:8B*) offers a substantially more deterministic, precise, and consistent data extraction than systems based purely on file retrieval and search (*Retrieval-Augmented Generation* or RAG with large and general models like *Gemini 2.5 Flash*). While the RAG approach stands out for its linguistic flexibility and synthesis capacity, its propensity for hallucinations and generating content not backed by the dataset critically diminishes its reliability in specialized research settings, such as technical musicology. Nonetheless, the results also evidence that both approaches suffer when faced with queries that require complex, multi-step musical reasoning.

Based on the findings and the detailed discussion, the following main conclusions are drawn:

- Structured storage in *SQLite* combined with a local model allows for highly reliable and hallucination-free information retrieval when the required data is explicitly represented in the schema. Conversely, RAG systems based on File Search provide greater contextual and descriptive richness in natural language, but introduce inaccuracies and false positives (such as inventing pieces that do not exist in the corpus).
- The primary limitation of the structured local approach does not reside in the availability of analytical data, but rather in the model's capacity to correctly translate complex natural language questions into executable SQL queries. Errors in this intermediate translation constitute a critical failure source that decreases the system's robustness against non-trivial or compound queries.

- Neither of the two evaluated approaches is fully autonomous or robust when directly resolving queries that involve complex abstractions or multi-step mathematical and musical calculations (such as analyzing boundary intervals, syncopations, or advanced melodic structures). Both systems tend to simplify responses or assume unjustified correlations in the absence of a prior analytical processing layer.
- For LLM-based tools to be truly useful in the musicological research of large collections, data representation must not only be structured but explicitly optimized for analytical queries. It is concluded that to improve these tools and their results, designing intermediate architectures that hybridize guided query generation, strict result validation, and specialized structured reasoning modules is necessary.

10.2. Future Work

The developed tool constitutes a first step toward applying artificial intelligence and computational musical analysis techniques to digital score corpora. Nonetheless, several lines of research could be explored in future work to expand the analytical capabilities of the tool, improve response quality, and enrich user experience.

A first line of development consists of implementing multi-agent architectures based on the *Agent-to-Agent* (A2A) protocol. Instead of delegating the entire reasoning process to a single conversational agent, tasks could be distributed among agents specialized in different areas, such as musical analysis, information retrieval, musicological interpretation, or response generation. This approach would allow complex queries to be addressed through collaborative processes among agents, facilitating system specialization and functional scalability.

Another possible evolution involves incorporating *Graph Retrieval-Augmented Generation* (GraphRAG) techniques. While the current implementation stores metadata in a relational database, a graph-based model would allow for the explicit representation of relationships existing among songs, authors, genres, geographical regions, musical structures, and stylistic features. This representation would facilitate more complex queries based on semantic and musicological connections, as well as the automatic exploration of non-evident relationships within the corpus. Likewise, the integration of hybrid models combining the current database with graph-based knowledge structures could be investigated. In this way, it would be possible to maintain the efficiency of structured queries while simultaneously taking advantage of relation-based contextual retrieval systems.

Another prospective line of research involves implementing musical similarity search mechanisms based on vector representations (*embeddings*). From the metadata extracted during preprocessing, each work could be represented by a numerical vector summarizing its main musical and textual characteristics, such as key, mode, time signature, rhythmic patterns, metric phenomena, or lyric themes. This representation would allow for the automatic calculation of the degree of similarity between songs and enable new features, such as searching for related works, identifying repertoires with common traits, or automatically recommending songs.

On the other hand, a priority technical development line to improve the local model's precision lies in optimizing relational queries over encapsulated semi-structured data. Currently, the *SQLite* database schema stores musical variables of high relational density and variability in text columns under structured JSON format; this is the case for critical fields such as `mapeo_silabas_duraciones` (which maps the exact correspondence between lyric syllables and musical durations) or `accidentales_compases_json` (which records the mea-

sure location of accidental alterations like *F#5* or *C#5*). With the objective of eliminating the multi-step reasoning limitations identified in LLMs, an immediate future task will consist of empowering the agent to execute native extraction functions like `json_extract()`. Integrating and training the model on the syntax of these functions will allow it to isolate, query, and operate directly on the internal keys and values of JSON objects without needing to alter the rigid relational schema or overload the model's context with prior natural language destructuring. This would translate into an increase in system robustness when handling complex analytical questions that cross, for example, specific syncopations with syllable durations or accidental alterations in designated measures.

Lastly, a user-oriented enhancement would involve developing a specific interface for song comparison. This functionality would allow selecting two or more works and jointly visualizing their musical, textual, and structural characteristics. Among other aspects, keys, modes, time signatures, rhythmic patterns, event density, syncopation phenomena, polyrhythms, hemiolas, or textual themes could be compared. A tool of this type would facilitate comparative studies and could be especially useful in contexts of musicological research and traditional repertoire analysis.

In addition to these concrete lines, the modular nature of the proposed architecture allows for the incorporation of new metadata extraction algorithms as new analytical needs are identified. This opens the door to future investigations focused on the automatic detection of more complex musical phenomena and the progressive expansion of the knowledge base used by the system.

Bibliografía

- ANTHROPIC PBC. Model context protocol (mcp): Open-sourcing the infrastructure for ai context. <https://modelcontextprotocol.io/>, 2024.
- CHOU, Y.-H. y ET AL. Midi-llama: An instruction-following multimodal llm for symbolic music understanding. *arXiv preprint arXiv:2601.21740*, 2026.
- COOK, N. Computational and comparative musicology. *Empirical Musicology: Aims, Methods, Prospects*, 2004.
- CUTHBERT, M. S. y ARIZA, C. music21: A toolkit for computer-aided musicology and symbolic music data. *International Society for Music Information Retrieval (ISMIR)*, 2010.
- HUANG, L. y ET AL. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232*, 2023.
- JI, S., ZHANG, S. y SULLIVAN, M. A comprehensive survey on deep learning for relation extraction. *ACM Computing Surveys*, 2023.
- KATSOGIANNIS-MEIMARAKIS, G. y KOUTRIKA, G. Text-to-sql generation: A survey on systems, datasets, and evaluation metrics. *ACM Computing Surveys*, 2023.
- LEWIS, P., PEREZ, E., PIKTUS, A., PETRONI, F., LEWIS, V., RIEDEL, S. y KIELA, D. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, vol. 33, páginas 9459–9474, 2020.
- MORETTI, F. *Distant reading*. Verso Books, 2013.
- ROLAND, P. The music encoding initiative (mei). En *Proceedings of the First International Conference on Musical Applications Using XML*. 2002.
- VASWANI, A., SHAZEER, N., PARMAR, N., USZKOREIT, J., JONES, L., GOMEZ, A. N., KAISER, Ł. y POLOSUKHIN, I. Attention is all you need. 2017.

Presentación del proyecto en congresos y seminarios

Durante el desarrollo de esta investigación, el trabajo realizado fue difundido en diversas actividades de transferencia y divulgación científica vinculadas al proyecto europeo . En el marco de este proyecto se organizaron varios seminarios en los que se presentó el diseño, la arquitectura y la implementación de la herramienta desarrollada.

Asimismo, los resultados obtenidos fueron presentados en junio en el congreso internacional *Computational Models of Narrative 2026*, donde se expusieron tanto el problema a resolver como la metodología empleada y las principales funcionalidades y beneficios del sistema.

Además, está prevista una nueva presentación a comienzos de julio en un seminario dedicado al proyecto *EA-DIGIFOLK* que se celebrará en Salamanca. En este evento se mostrarán las capacidades de la herramienta y su potencial para el análisis automatizado de corpus musicales.

Coincidence makes sense only with you.

*Jóga
Björk*

